Chapter 1: Introduction

Chien Chin Chen

Department of Information Management
National Taiwan University

What is an Operating System? (1/2)

A ***program*** that acts as an intermediary between a ***user*** of a computer and the ***computer hardware***.

Various operating system goals:

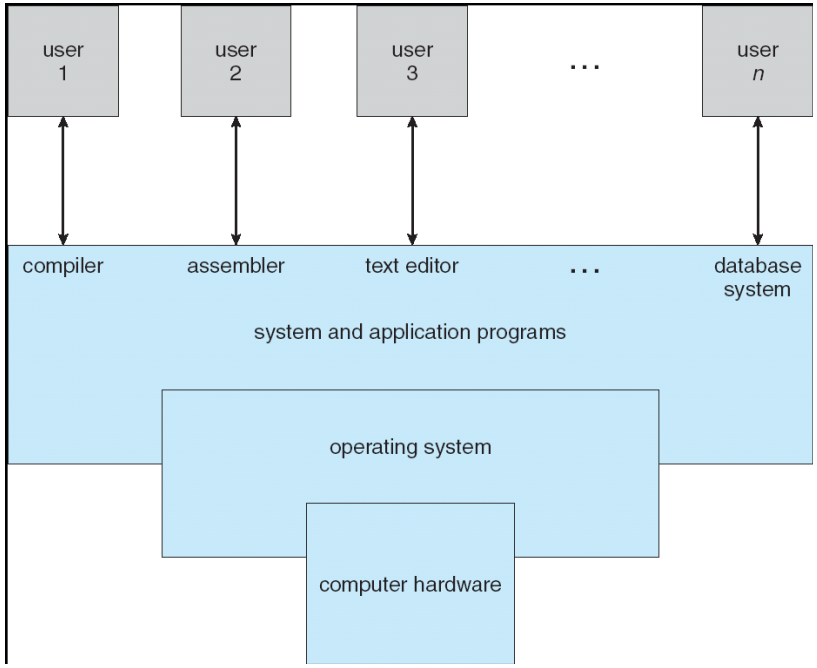
- ***Mainframe*** operating systems: to optimize utilization of hardware.
- ***PC*** operating systems: to support complex games, business applications …
- ***Handheld computers***: to help users easily interface with the computer to execute programs.

What is an Operating System? (2/2)

Some operating systems are designed to be ***convenient***, others to be ***efficient***, and others some ***combination of the two***.

Four Components of a Computer System (1/2)

Computer system can be divided into four components: ***hardware***, the ***operating system***, the ***application programs***, and the ***users***.



Four Components of a Computer System (2/2)

Hardware – provides basic **computing resources**.

- CPU, memory, I/O devices.

Operating system – **controls** and **coordinates** use of hardware among various applications and users

Application programs – define the ways in which the system resources are used **to solve the computing problems of the users**.

- Word processors, compilers, web browsers, database systems, video games.

Users – people, machines, other computers.

An operation system is similar to a ***government***. It provides an environment within which other (user) programs can do useful work.

Viewpoints From Users (1/2)

PC: the OS is designed for ***one user only***.

- Resources are monopolized.
- The goal is to maximize the work of the user.
- The OS is generally designed for ***ease of use***, with some attention paid to performance and non paid to resource utilization.

Mainframe: the OS is designed for ***multiple users*** – accessing the same computer through terminals.

- These users share resources.
- The OS is designed to ***maximize resource utilization*** – to assure that all available CPU time, memory …

Viewpoints From Users (2/2)

Handheld computer: are **standalone** units for individual users.

- The OS is designed mostly for individual usability.
- But performance per amount of battery life is important as well.

Computer with little (or no) user view: embedded home devices.

- The OS is designed to run without user intervention.

Viewpoints From Computers

OS for computer is the program involved with the hardware.

OS is a ***resource allocator***.

- Manages all resources.
- Decides between **conflicting requests** for efficient and **fair** resource use.

OS is a ***control program***.

- Controls execution of programs to prevent errors and improper use of the computer.

Operating System Definition (1/3)

What is an operating system?

- No universally accepted definition.

Bare computer (hardware) alone is not easy to use, so application programs are developed.

- These programs require certain ***common operations***, such as those controlling the I/O device.
- These common functions of ***controlling*** and ***allocating*** resources are then brought together into one piece of software: the *operating system*.

Operating System Definition (2/3)

OS is …

- "Everything a vendor ships when you order an operating system"

  But varies wildly, for example, text/graphic mode.

A more common definition:

- "The one program running at all times on the computer" is the ***kernel***.
- Everything else is either a system program or an application program.

Operating System Definition (3/3)

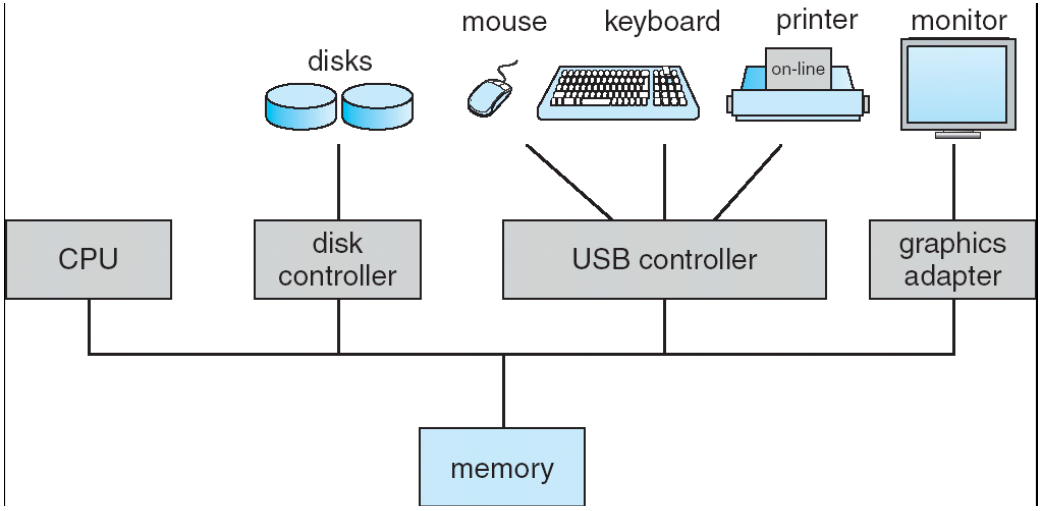
The matter of what constitutes an operating system has become increasingly important.

- Antitrust of Microsoft windows.
- Microsoft included too much functionality in its operating systems and thus prevented application vendors from competing.

Computer System Organization (1/2)

Computer-system operation:

- One or more ***CPUs***, ***device controllers*** connect through common ***bus*** that provides access to shared memory.
- Each device controller is in charge of a specific type of ***device*** (disk drives, audio/video devices).



Computer System Organization (2/2)

Each device controller has a **local buffer** and a set of special-purpose **registers**.

- E.g., your SATA hard disk may contains 8M buffer.

CPU moves data from/to main memory to/from local buffers.

I/O devices and the CPU can execute **concurrently**.

Concurrent I/O is from the device to local buffer of controller.

Computer System Operation (1/2)

When a computer is **powered up** or **rebooted** …

- A ***bootstrap program*** is loaded to Initialize all aspects of system, from CPU registers to device controllers to memory contents.

  Typically stored in ROM (read-only memory) or EEPROM, (erasable programmable read-only memory) generally known as **firmware**.

  Load into memory the operating-system kernel.

- The operating system then starts executing the first process and waits for some **event** to occur.

- **Events** are usually signaled by an ***interrupt*** from either the hardware or the software.

  Hardware :I/O operations — disk drive access, keystroke, …

  Software: system calls.

Computer System Operation (2/2)

- Why interrupt? — I/O operations ***without*** interrupt as an example.
  - The *handshaking* process of a host (a program) reads data through a port (device):
    - The controller does the I/O to the device (hardware operations).
    - The host ***repeatedly*** read the `busy` bit (of the device) until that bit becomes clear.
    - The host reads data from the device controller.
    - The I/O is finished.
  - In step 2, the host is ***busy-waiting*** or ***polling***.
    - It is in a *loop*, reading the `busy` bit over and over until the bit becomes clear.
    - The wait may be long, the host should probably switch to another task.
    - **Interrupt**: the *hardware mechanism* that enables a device to ***notify*** the CPU when it is ready for service.

Interrupt Mechanism (1/3)

The CPU hardware has a wire called the ***interrupt-request line*** that the CPU senses after executing every instruction.

When the CPU detects that a controller has asserted a signal on the line, the CPU performs a **state save** and **transfers control to a generic routine**.

The generic routine examine the interrupt information and calls the **interrupt-specific handler**.

- Polling all the devices to see which one raised the interrupt.

Ideally, interrupts must be handled quickly (data overflow on keyboard controller).

- Fortunately, only a predefined number of interrupts is possible.
- The modern interrupt mechanism accepts an ***address*** (index) in accordance with a ***interrupt vector*** (table) to provide fast interrupt service.

Interrupt Mechanism (2/3)

The ***interrupt vector*** contains the addresses of all the specific interrupt handle routines.

The ***address*** (index) is an offset in the interrupt vector.

This vectored interrupt mechanism can reduce the need for a single interrupt handler to search all possible sources of interrupts.

Operating systems as different as Windows and Unix dispatch interrupt in this manner.

vector number	description
0	divide error
1	debug exception
2	null interrupt
3	breakpoint
4	INTO-detected overflow
5	bound range exception
6	invalid opcode
7	device not available
8	double fault
9	coprocessor segment overrun (reserved)
10	invalid task state segment
11	segment not present
12	stack fault
13	general protection
14	page fault
15	(Intel reserved, do not use)
16	floating-point error
17	alignment check
18	machine check
19–31	(Intel reserved, do not use)
32–255	maskable interrupts

Used for device-generated interrupts

The design of the interrupt vector for the Intel Pentium processor.

Interrupt Mechanism (3/3)

The interrupt mechanism must also **save the address of the interrupted instruction**.

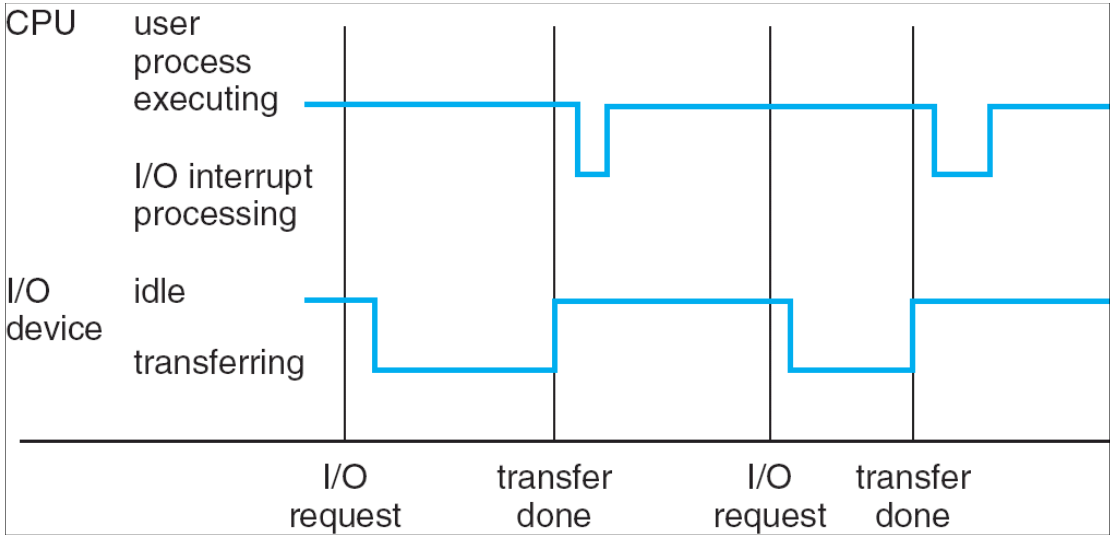
- It may also save the **state** (processor register values).

After the interrupt is serviced, the saved address (and saved state) is loaded, and the interrupted computation resumes as though the interrupt had not occurred.

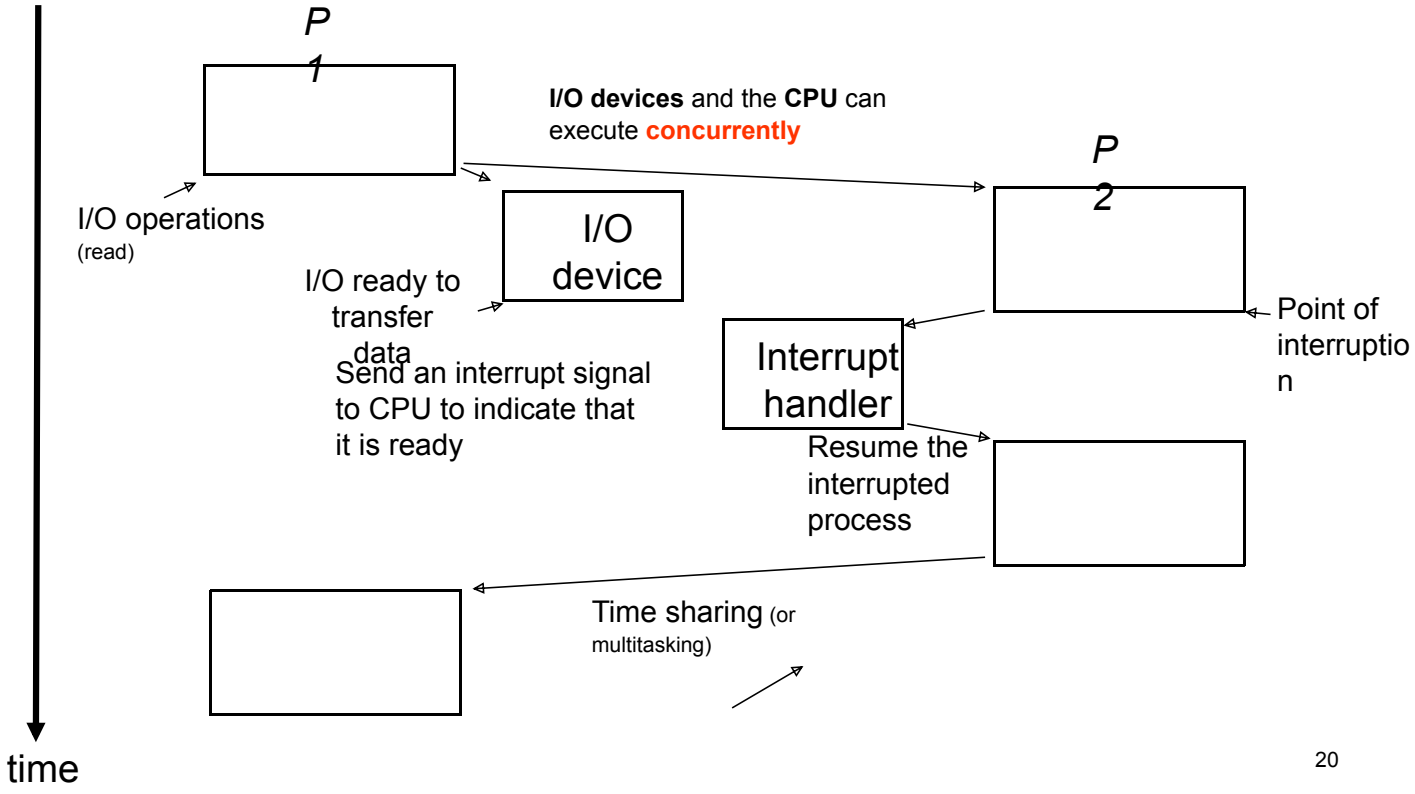
Note:

- Incoming interrupts are *disabled* while another interrupt is being processed to prevent a *lost interrupt*.
- A ***trap*** is a ***software-generated interrupt*** caused either by an error or a user request (system call).

Interrupt Timeline



Example of Interrupt



Storage Structure (1/4)

Main memory and the ***registers*** are the only storage that the CPU can access directly.

Registers:

- Are built into the CPU.
- CPU can decode instructions and perform simple operations on register contents.

Main memory:

- Computer programs must be in main memory to be executed.
- Is the **only large storage area** that the CPU can access directly.

Storage Structure (2/4)

A typical (instruction) execution cycle first ***fetches*** an instruction from memory.

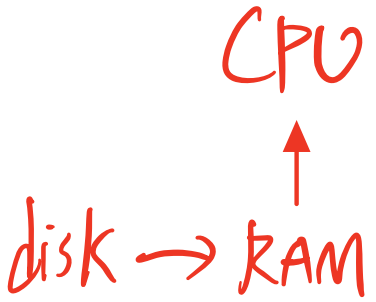
- The instruction is stored in the ***instruction register***.

CPU then ***decodes*** the instruction.

- May cause operands to be fetched from memory and stored in some ***internal register***.

After the instruction on the operands has been executed, the result may be stored back in memory.

Memory contains information about **data** and **programs**.



Storage Structure (3/4)

Ideally, we want the **programs** and **data** to reside in main memory permanently.

- **Impossible!!**

  Main memory is too **small** to store all needed programs and data.

  Main memory is a **volatile** storage device that loses its contents when power is turned off.

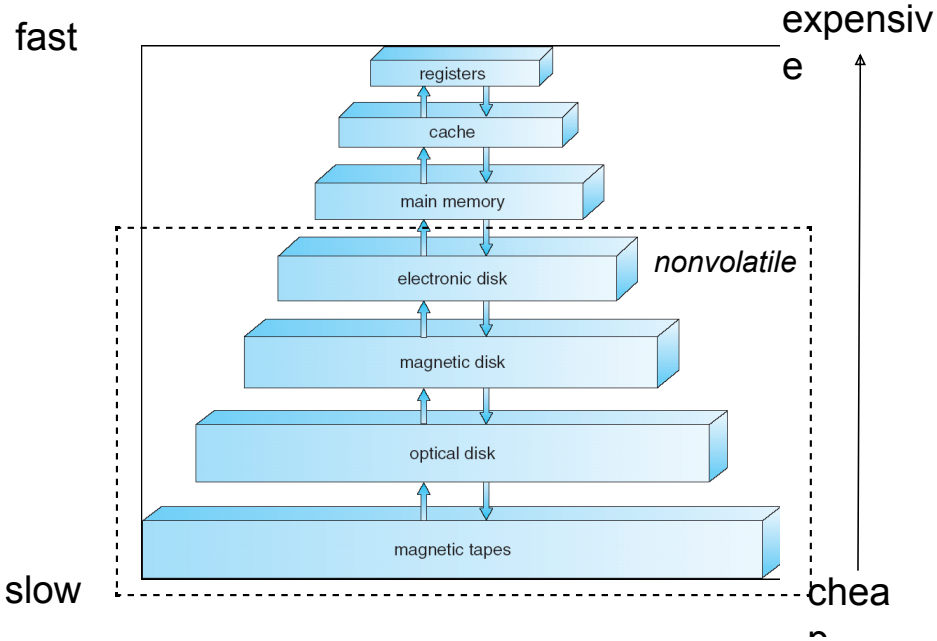
Secondary storage – extension of main memory that provides large nonvolatile storage capacity.

- Magnetic disks – The most common secondary storage.

Storage Structure (4/4)

Other storage units: CD-ROM, tapes, and so on.

Storage systems organized in ***hierarchy*** according to **speed** and **cost**.



I/O Structure (1/7)

A large portion of operating system code is dedicated to managing I/O.

- Importance to the reliability and performance of a system.
- The varying nature of the devices.

A general-purpose computer system consists of ***CPUs*** and multiple ***device controllers*** that are connected through a common ***bus***.

- Each device controller is in charge of a specific type of device (e.g., disk controller and disks).

I/O Structure (2/7)

Hard disk and disk controller

https://www.technipages.com/wp-content/uploads/2022/08/hard-disk-controller.jpg

I/O Structure (3/7)

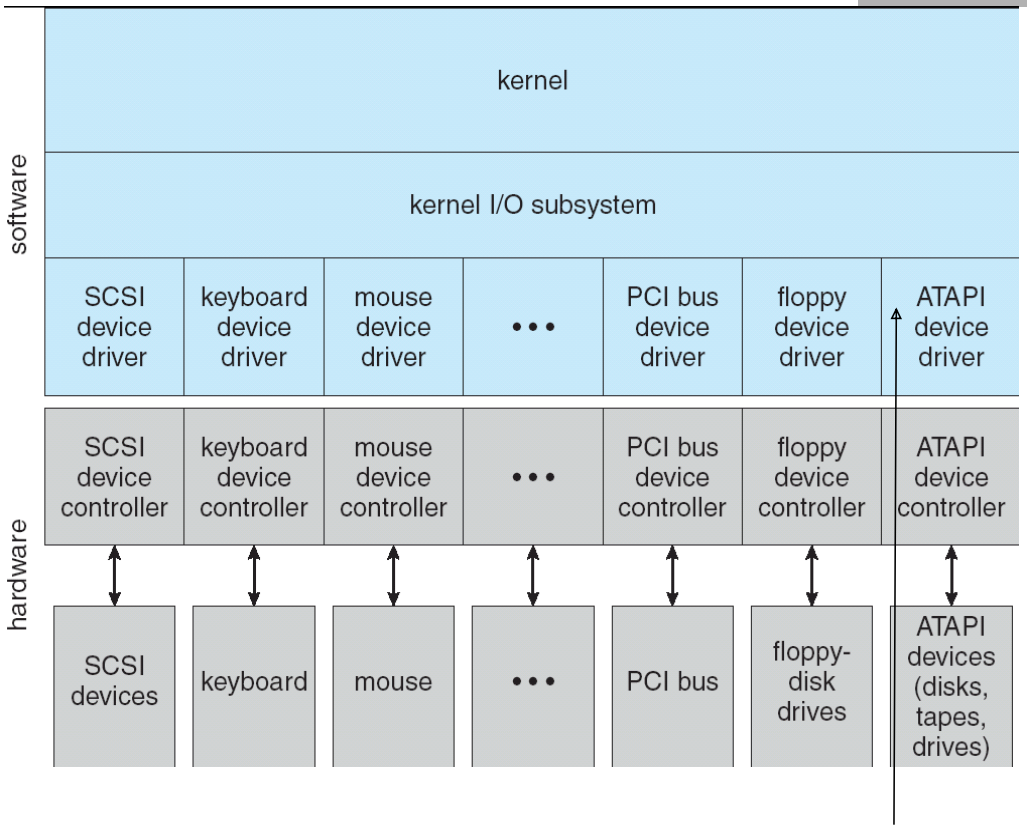
A device controller maintains some local ***buffer*** storage and a set of ***registers***.

- Controller is responsible for moving the data between the devices that it controls and its local buffer storage.

Operating systems have a ***device driver*** for each device controller.

- The device driver understands the device controller and presents a **uniform interface** to the device to the operating system.

I/O Structure (4/7)

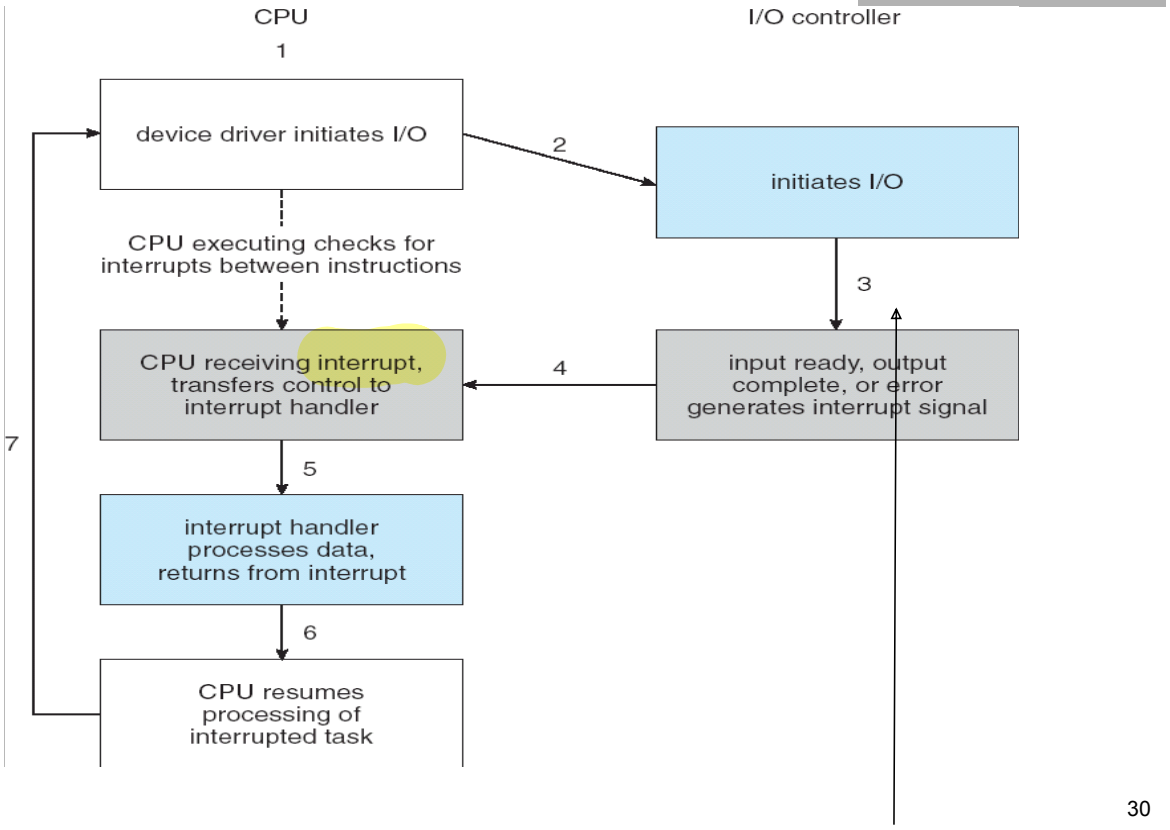


I/O Structure (5/7)

To start an I/O operation (such as “read a character from a disk device”):

- The **device driver** loads the appropriate registers with the device controller.
- The **device controller**, in turn, examines the contents of these registers to determine what action to take.
- The controller starts the transfer the data from the device to its local buffer.
- Once the transfer the data is complete, the device controller informs the device driver via an **interrupt**.
- The device driver then returns control (the received data) to the operating systems.

I/O Structure (6/7)



I/O Structure (7/7)

The purpose of device driver is to hide the differences among device controllers from the I/O subsystem of the kernel.

- An I/O `read()` can read data from SATA, ATA, or SCSI hard disks by using specific device drivers.

Device drivers are **internally custom-tailored** to each device but **export standard interfaces** to the I/O subsystem of the kernel.

- Benefit:
  - Simplify the job of the operating system developer.
  - New devices are easy attached to a computer system without waiting for the operating-system vendor to develop support code.

Computer System Architecture (1/5)

Computer systems can be categorized according to the number of general-purpose processors used.

Single-processor systems:

- There is *one main CPU* capable of executing a general-purpose instruction set.
- Almost all systems have other *special-purpose processors* as well.
  - Device-specific processors (e.g., graphics controllers).
  - Do not run user processes directly and are managed by the operating system.
  - Relieve the overhead of the main CPU.

Computer System Architecture (2/5)

Multiprocessor Systems:

- As known as ***parallel systems***.
- Have **two or more processors** in close communication.
- Share the same computation resources (memory, bus, …).

- Main advantages:

  <u>Increased throughput</u>:

  - We expect to get more work done in less time.
  - The speed-up ratio with *N* processors is not *N*, however; rather, it is less than *N*.
  - A certain amount of overhead is incurred in keeping all the parts working correctly.

Computer System Architecture (3/5)

- **Economy of scale**:
  - Cost less than equivalent multiple single-processor systems, because they share computation resources.
    - Store data on the same disk vs. many copies of the data.
- **Increased reliability**:
  - If functions can be distributed properly among several processors, then the failure of one processor will not halt the system, only slow it down.

Computer System Architecture (4/5)

The multiple-processor systems in use today are of two types:

- **Asymmetric multiprocessing** (master-slave relationship):
  - Each processor is assigned a specific task.
  - A **master** processor controls the system.
  - The master processor schedules and allocates work to the **slave** processors.
- **Symmetric multiprocessing** (SMP):
  - All processors are **peers**, performs all tasks within the operating system.
  - Operating system must be written carefully to avoid unbalanced resource arrangement.
    - One processor may be sitting idle while another is overloaded.
  - Virtually all modern operating systems (windows, Mac OSX, Linux) now provide support for SMP.

Computer System Architecture (5/5)

A recent trend in CPU design is to include multiple compute **core** on a single chip.

- In essence, these are multiprocessor chips.
- These multi-core CPUs look to the operating system just as *N* standard processors.
- Advantages:
  - On-chip communication is faster
  - One chip with multiple cores uses significantly less power than multiple single-core ships

Operating System Structure (1/7)

Operating systems vary greatly in their makeup, since they are organized along many different lines.

- However, there are many commonalities.

One of the most important aspect of operating systems is the ability to ***multiprogramming*****.**

- User cannot keep CPU and I/O devices busy at all times.
  - Jobs have to wait for some task, such as an I/O operation, to complete.
  - The CPU would sit idle.
- Multiprogramming organizes jobs so **CPU always has one to execute**.

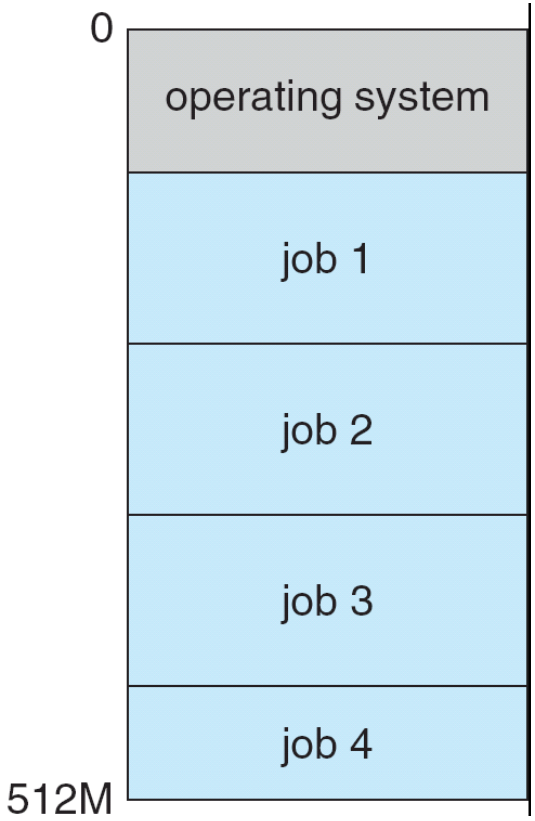
Operating System Structure (2/7)

Multiprogramming:

- A subset of total jobs in system is kept in memory.
- One job is selected and run.
- When it has to wait (for I/O for example), OS switches to another job.
- When that job needs to wait, the CPU is switched to another job, and so on.
- Eventually, the first job finishes waiting and gets the CPU back.

- As long as at least one job needs to execute, **the CPU is never idle**.

Operating System Structure (3/7)



Operating System Structure (4/7)

Time sharing (multitasking):

- Is logical **extension** **of multiprogramming**.
- CPU switches jobs **frequently**.

Time sharing is frequently used in ***interactive computer system***.

- For example, windows systems, which provide direct communication between the user and the system (using keyboard or mouse).

  The response time (of each job) should be short!!
- The operating system must switch rapidly from one job (for one user) to next, such that each user is given the impression that the entire computer system is dedicated to his use.

Operating System Structure (5/7)

Time sharing and multiprogramming require several jobs to be kept simultaneously in memory.

- A program loaded into memory and executing is called a ***process***.

Since main memory is too small to accommodate all jobs, the jobs are kept initially on the disk in the ***job pool***.

- The pool consists of all processes residing on disk awaiting allocation of main memory.

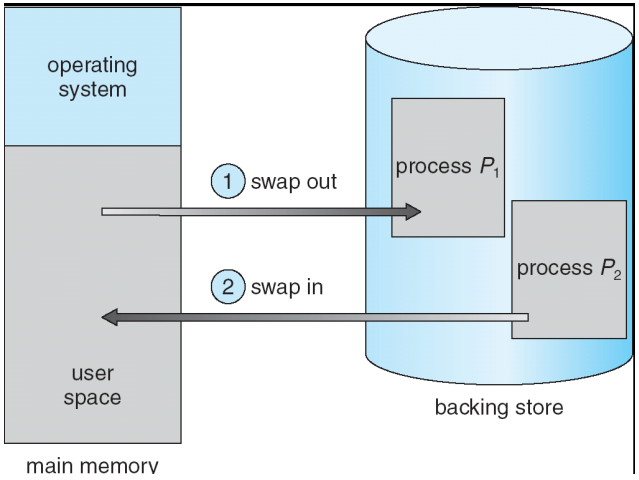
Job scheduling (chapter 5) selects jobs from the pool and loads them into memory for execution.

If several jobs ready to run at the same time ***CPU scheduling*** (chapter 5) chooses among them.

Operating System Structure (6/7)

If processes don't fit in memory, ***swapping*** moves them in and out to run (chapter 9).

- Swap to a backing store (e.g., hard disks).
- For example a higher-priority process arrives and wants service, the memory manager can swap out the lower-priority process and then load and execute the higher-priority process.



Operating System Structure (7/7)

Virtual memory allows execution of processes not completely in memory (chapter 10).

Dual-Mode Operation (1/4)

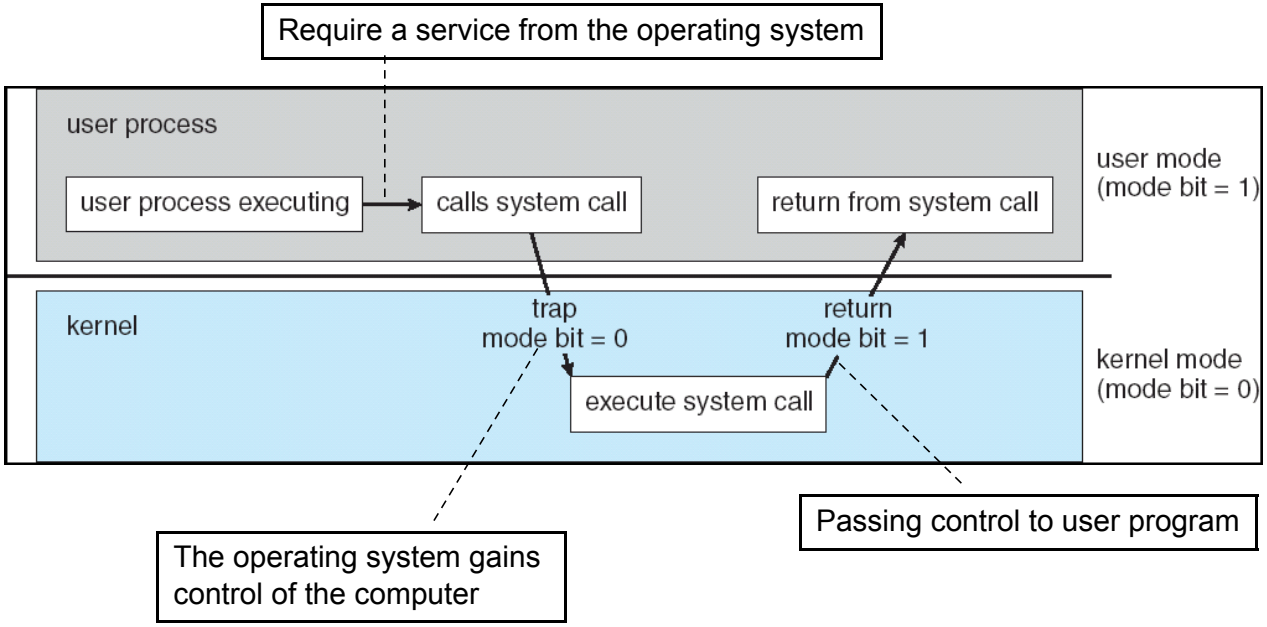
To ensure the proper execution of the operating system, we must be able to **distinguish** between the execution of **operating-system code** and **user-defined code**.

- Computation resources **can only** be managed by operating-system code.
- User-defined code can not cross the line.
- Supported by hardware mechanism.

Dual-mode: ***user mode*** and ***kernel mode***.

- **Mode bit** provided by hardware, kernel (0) or user (1).
  Recent versions of the Intel CPU do provide dual-mode.
- When a user application requests a service from operating system, it must transition from user to kernel mode to fulfill the request.
- Request only through ***system call***.

Dual-Mode Operation (2/4)



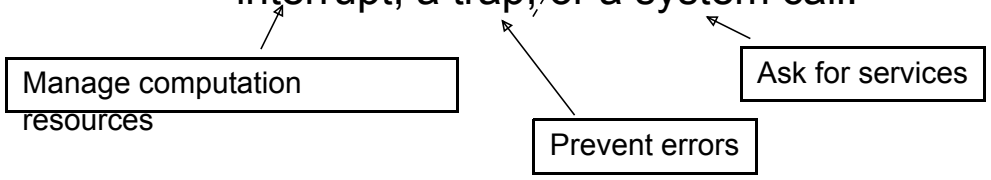
Dual-Mode Operation (3/4)

The dual mode protects the operating system from errant users.

- We designate some machine instructions that may cause **harm** as ***privileged instructions***, and only executable in kernel mode.

  I/O control, interrupt management … are examples of privileged instruction.

- Generally, control is switched to the operating system via an interrupt, a trap, or a system call.



Dual-Mode Operation (4/4)

More description of system call:

- When a system call is called, it is treated by the hardware as a software interrupt.
- The mode bit is set to kernel mode.
- Control passes through the interrupt vector to a service routine in the operating system.

  The kernel examines the parameter of the interrupt to determine what type of service the user program is requesting.
- The kernel verifies and executes the request.
- And returns control to the instruction following the system call (user mode).

Caching (1/2)

Important principle of computer systems.

Information is normally kept in some storage system (large, slow, and cheap).

As it is used, it is copied into a faster (small, and expensive) storage system — the ***cache***.

When we need a particular piece of information, we **first** check faster storage (cache) to determine if information is there.

- If it is, information used directly from the cache (fast).
- If not, **data copied to cache** and used there.

Because cache is smaller than storage being cached, **cache management** is an important design problem.

- Careful selection of the **cache size** and **replacement policy** (chapter 10).

Caching (2/2)

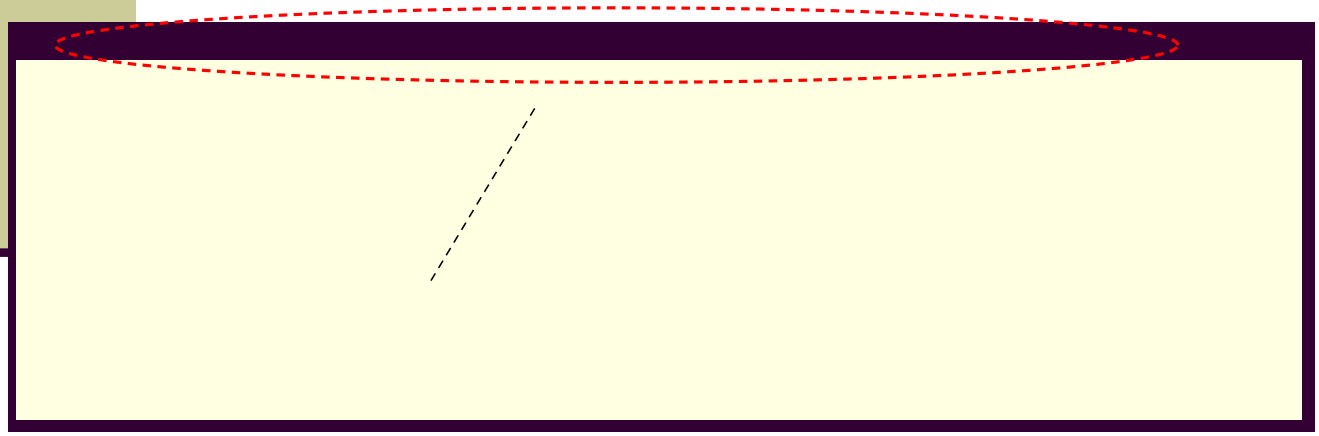
Performance of Various Levels of Storage:

Level	1	2	3	4
Name	registers	cache	main memory	disk storage
Typical size	< 1 KB	> 16 MB	> 16 GB	> 100 GB
Implementation technology	custom memory with multiple ports, CMOS	on-chip or off-chip CMOS SRAM	CMOS DRAM	magnetic disk
Access time (ns)	0.25 – 0.5	0.5 – 25	80 – 250	5,000.000
Bandwidth (MB/sec)	20,000 – 100,000	5000 – 10,000	1000 – 5000	20 – 150
Managed by	compiler	hardware	operating system	operating system
Backed by	cache	main memory	disk	CD or tape

Movement between levels of storage hierarchy can be **explicit** or **implicit**:

- For example, transfer of data from disk to memory is usually controlled by the operating system (explicit).

End of Chapter 1



Chapter 2: System Structures

Chien Chin Chen

Department of Information Management
National Taiwan University

Objectives

- To describe the **services** an operating system provides to users, processes of the system.
- To discuss the various ways of **structuring** an operating system.
- To explain how operating systems are installed and customized and how they boot.

Operating System Services (1/4)

- One set of operating-system services provides functions that are **helpful to the user** (or user processes):
  - **User interface** - almost all operating systems have a user interface (UI).
    - ***Command-line interface*** (CLI): require a program to allow entering and editing of text commends.
    - ***Graphics user interface*** (GUI): a window system with a pointing device and a keyboard to enter commends.
    - ***Batch***: commands and directives are entered into files to be executed.
  - **Program execution** - the system must be able to load a program into memory and to run that program, end execution, either normally or abnormally.

Operating System Services (2/4)

- **I/O operations** - a user program may require I/O.
  - For efficiency and protection, users cannot control I/O devices directly.
  - The operating system must provide a means to do I/O.
- **File-system manipulation** - user programs need to read/write/create/delete/search files and directories.
  - The operating system provides permission management to allow or deny access to files or directories.
- **Communications** – user processes may exchange information, on the same computer or between computers over a network.
  - Communications may be via *shared memory* or through *message passing*.
- **Error detection** – the operating system needs to be constantly aware of possible errors.
  - And fix errors generated from hardware (disk fail) or software (arithmetic error).

Operating System Services (3/4)

- For systems with **multiple users** (processes), another set of operating-system functions exists for ensuring **the efficient operation of the system itself**.
  - **Resource allocation** - when multiple users or multiple jobs running concurrently, resources must be allocated to each of them.
    - CPU, memory, file storage …
    - Operating systems have *CPU-scheduling routines* to determine the best way to use the CPU.
  - **Accounting** - To keep track of which users use how much and what kinds of computer resources.
    - Usage statistics may be a valuable tool for researchers who wish to reconfigure the system to improve computing services.

Operating System Services (4/4)

- **Protection and security** - a multiuser or networked computer system may want to control use of user information.
  - **Concurrent** processes should not interfere with each other.
  - **Protection** involves ensuring that all access to system resources is controlled.
  - **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices (e.g., network adapters) from invalid access attempts.

User Operating-System Interface – **CLI** (1/4)

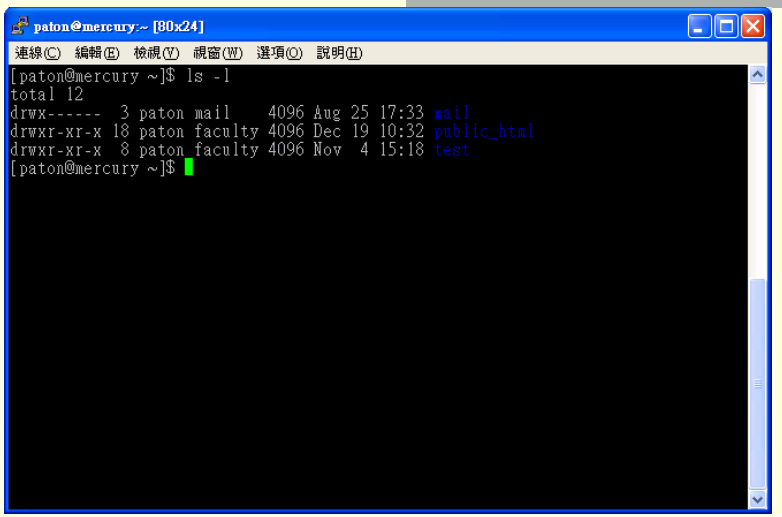
- ***Command-line interface***

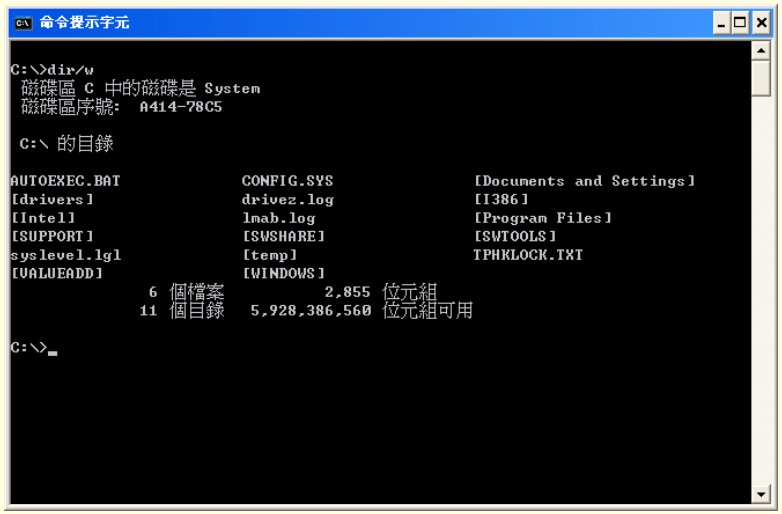
  (interpreter):

  - Primarily **get** and **execute** the next **user-specified command**.

  - Many of the commands are to manipulate files/directories.

    - Create/delete/list/print/ copy/execute…





User Operating-System Interface – CLI (2/4)

- Two ways to implement commands and command interpreters:
  - The command interpreter contains the code to execute the command.
    - For example, a command to delete a file may cause the interpreter to jump to a section of its code that makes the appropriate system call.
    - The number of commands that can be given determines the size of the interpreter.
  - An alternative approach implements **most** commands through system programs.
    - If the interpreter does not understand the command ...
    - It **identify** the command file and **load** it into memory for **execution**.

User Operating-System Interface – CLI (3/4)

- (cont.)
  - An UNIX example: `rm file.txt`
    - The interpreter search for a file called `rm` (`/bin/rm`).
    - **Load** it into memory and **execute** it with the parameter `file.txt`.
    - The function `rm` is completely defined by the code in the file `/bin/rm`.
  - Programmers can add new commands to the system easily.
  - The interpreter program can be **small**, and does not have to be changed for new commands to be added.
  - Used mostly among operating system, e.g., UNIX.

User Operating-System Interface – CLI (4/4)

- CLI is sometimes implemented in kernel, sometimes by systems program.
- An operating system can have multiple interpreters to choose from, known as ***shells***.
  - For example, on UNIX and Linux systems, there are *Bourne/C/Korn* …shell.
  - The name 'shell' originates from shells being an outer layer of interface between the user and the innards of the operating system (the kernel).
  - Most shells provide similar functionality with only minor differences; most users choose a shell based upon personal preference.
    - E.g., the syntax of *shell script*.

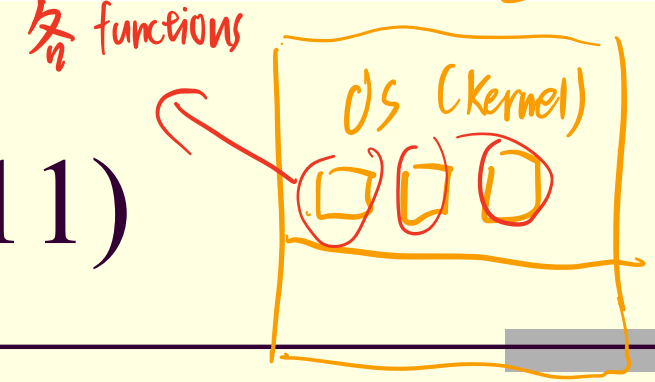
User Operating System Interface – **GUI** (1/2)

- A GUI provides a ***desktop*** metaphor interface.
  - ***Icons*** represent files, programs, actions, etc.
  - A ***mouse click*** can invoke a program, select a file …
- GUI Timeline:
  - Experimentally appeared in the early 1970s.
  - Became widespread by Apple Macintosh computer (Mac OS) in the 1980s.
  - Dominated by Microsoft Windows (3.1, NT, 95, 98, 2000, XP, Vista).
- UNIX systems have been dominated by command-line interface.
  - Although there are various GUI interface available.
    - X-Windows systems, K Desktop Environment (KDE) by GNU project (open source – source code is in the public domain).

User Operating System Interface – GUI (2/2)

- Preference:
  - Many UNIX users prefer a command-line interface.
  - Most Windows user are pleased to use the Windows GUI and never use the MS-DOS shell interface.
- Nevertheless, many systems now include both CLI and GUI interfaces.

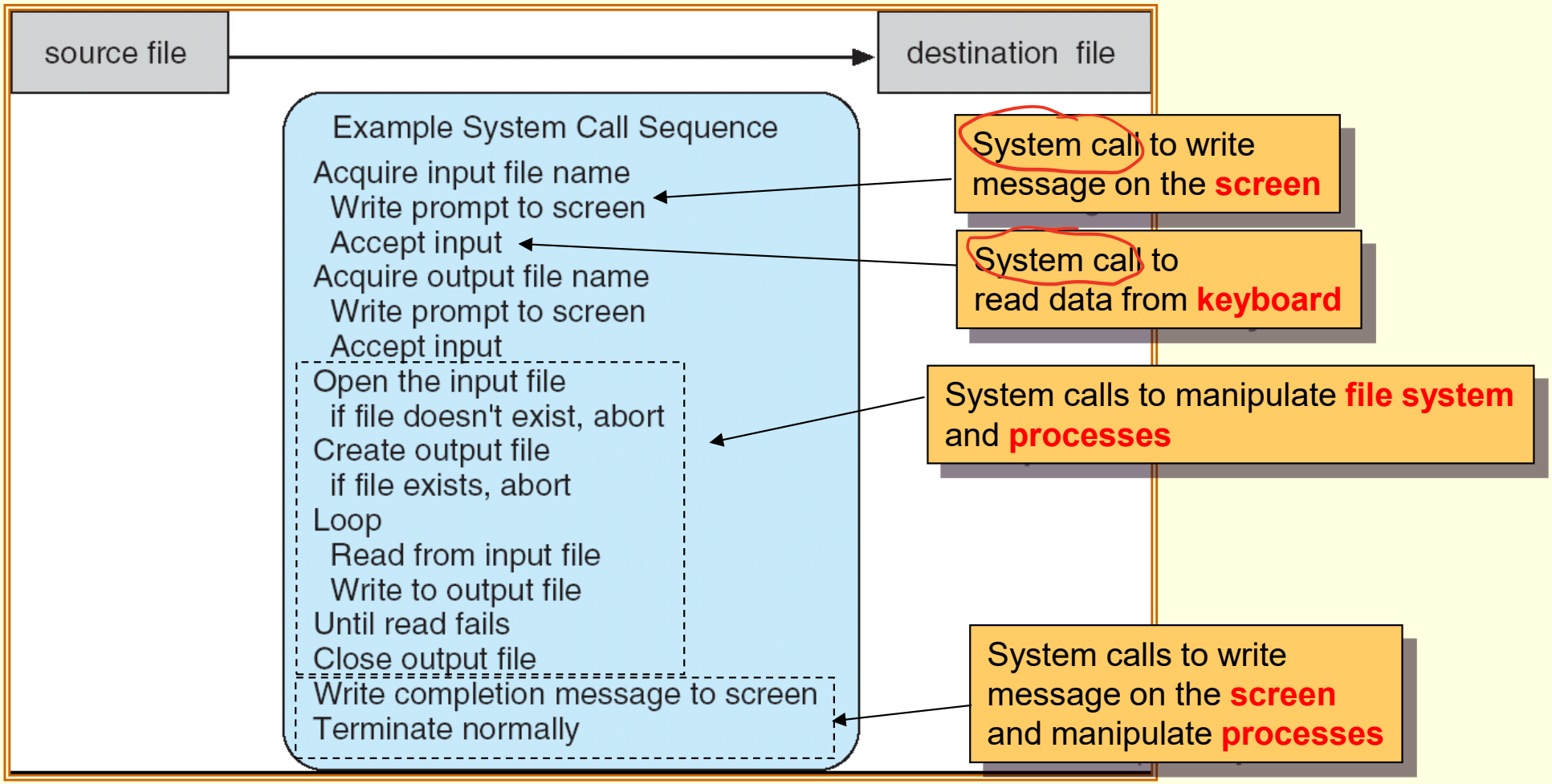
System Calls (1/11)



- Can be regarded as a ***programming interface*** ***to the services provided by the OS***.
  - Called by user applications.
- Are generally available as ***routines***, typically written in a high-level language (C or C++).
  - Also in low-level assembly language (for accessing hardware).

System Calls — An Example Program of File Copy (2/11)

- System call sequence to copy the contents of one file to another file.



Even a simple program makes heavy use of system calls!!

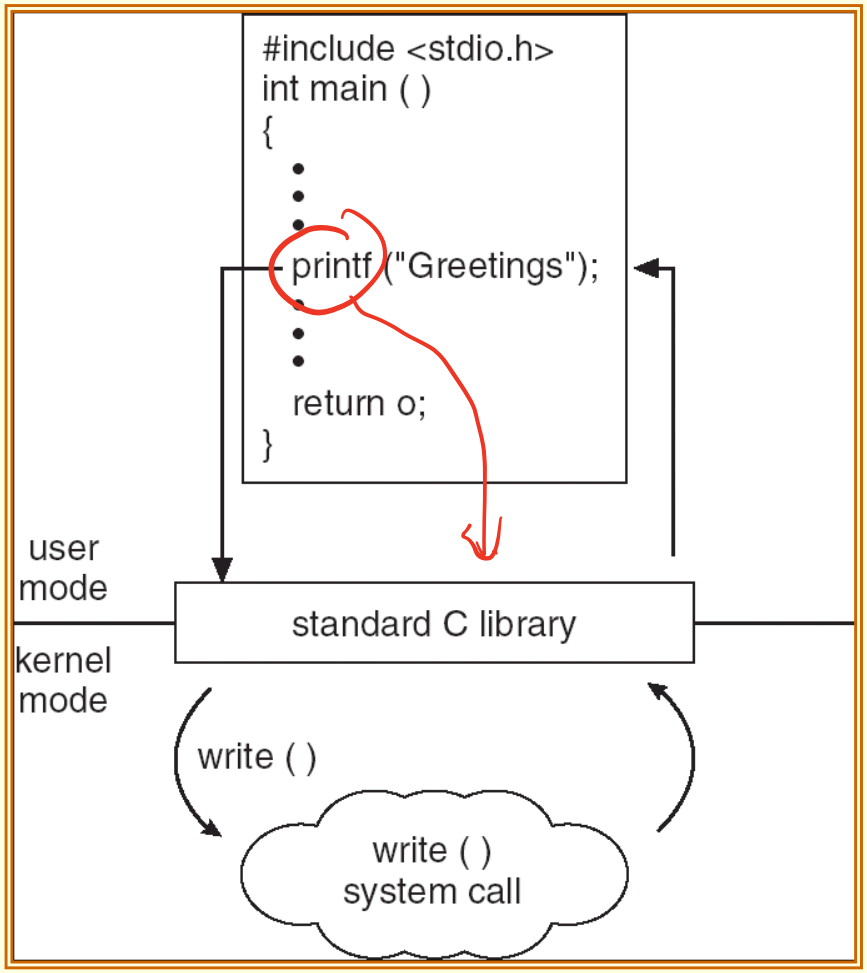
System Calls (3/11)

- Programs mostly access system services via a high-level ***application program interface*** (API) rather than using system call directly.
  - API specifies a set of functions (specifications) that are available to programmers.
  - **The functions of the API invoke the actual system calls on behalf of the programmer**.
- Three most common APIs:
  - ***Windows API*** for Windows.
  - ***POSIX API*** for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X).
  - ***Java API*** for the Java virtual machine (JVM).

Portable Operating System Interface

System Calls (4/11)

- C program invoking `printf()` library call, which calls `write()` system call.



System Calls (5/11)

- Why use APIs rather than system calls?
  - **Program portability** — a program using an API can be expected to compile and run on any system that supports the same API.
    - E.g., your Windows programs on various versions of Windows.
  - Programming with API is more **simpler** than using system calls.

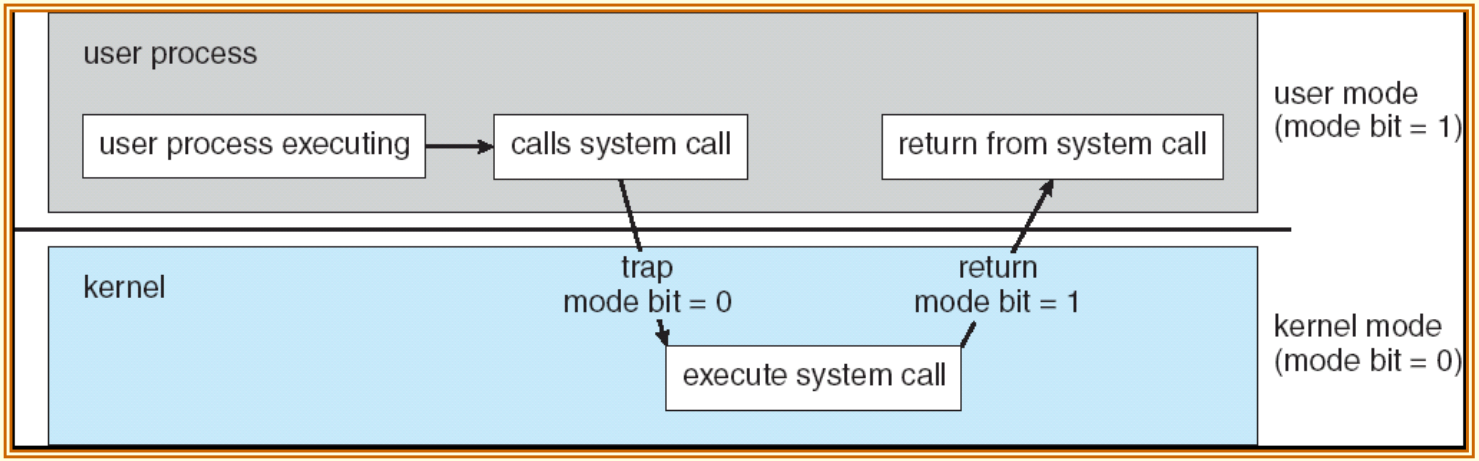
software interrupt

System Calls (6/11)

至此四次 ← ①call總機 ②pointer to program

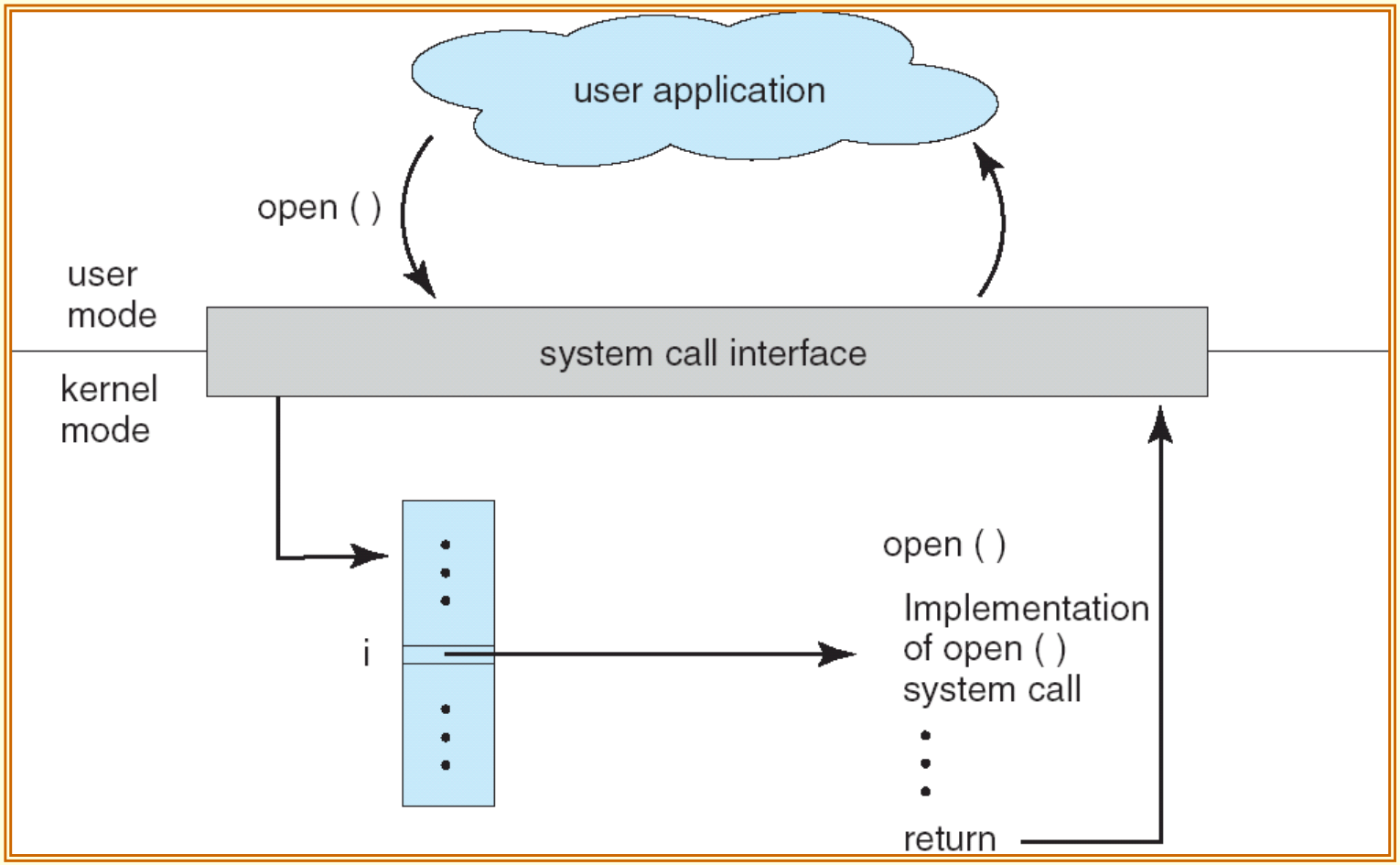
- As system calls are routines in kernel space, using it causes a **change in privileges**.
- How ?
  - Via software interrupt (e.g., `INT 0x80` assembly instruction on Intel 386 arch of Linux system).
- But before that …
  - Similar to hardware interrupt, we need a number (index) to indicate the required system call, which is store in the `EAX register`.
- System contains a table of code pointers.
  - Using the system call number, we jump to the address of the system call for execution.

API helps us wrap all the details by simply invoking a library function.



System Calls (7/11)

- How the operating system handles a user application invoking the `open()` system call through API.



System Calls (8/11)

- To **link** system call made available by the operating system:
  - The **run-time library** for most programming languages provides a **system-call interface**.
  - Typically, a **number** associated with each system call.
  - System-call interface (or kernel) maintains a **table** indexed according to these numbers.
  - The system call interface invokes intended system call in operating system kernel and returns status of the system call and any return values.

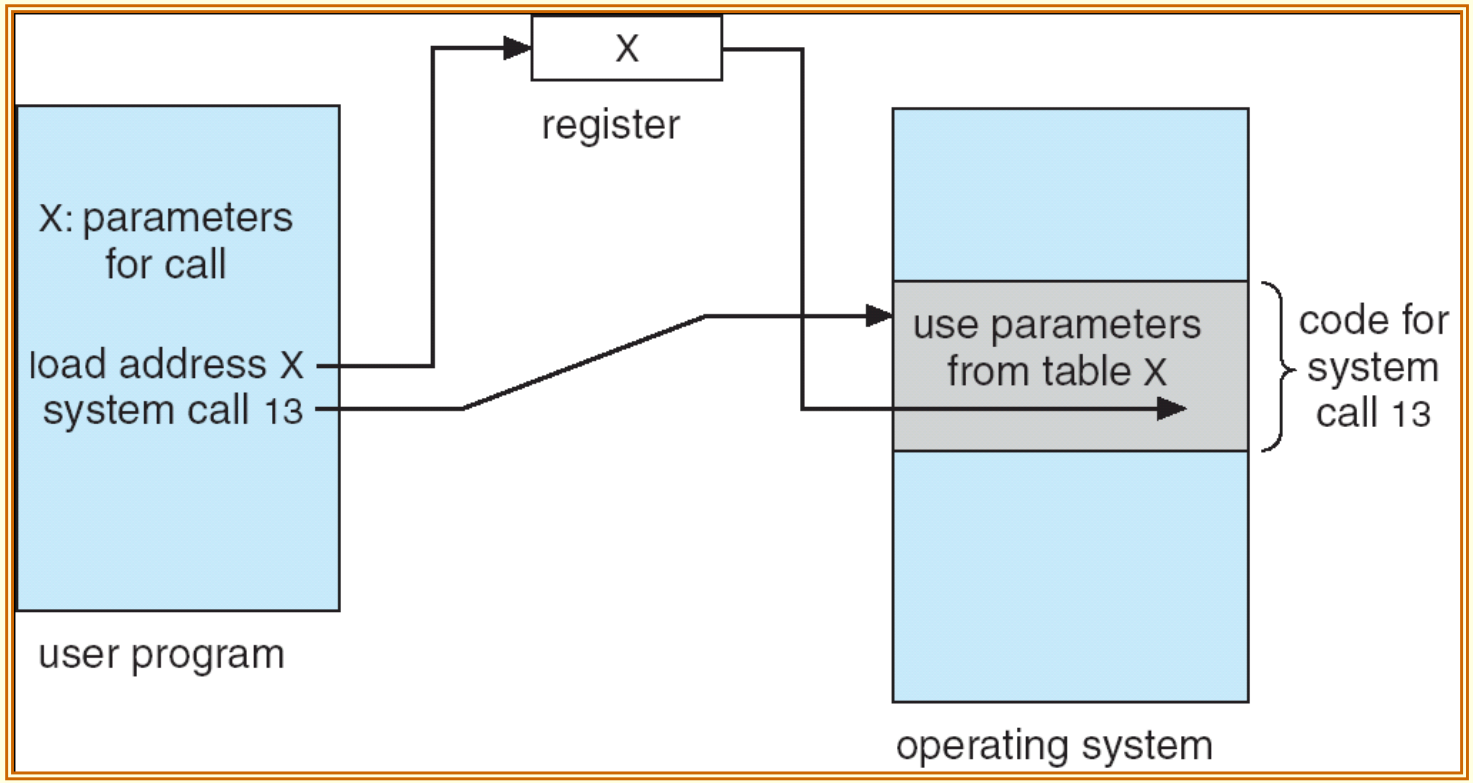
System Calls (9/11)

- With the help of API:
  - The caller need know nothing about how the system call is implemented.
  - Just needs to obey API and understand what the operating system will do as a result of that system call.
  - Details of the operating system are hidden from programmer.

System Calls (10/11)

- Often, more information is required than simply identity of desired system call
  - E.g., system call parameters.
- Three general methods used to pass parameters to the OS:
  - Simplest: pass the parameters in *registers*
    - In some cases, may be more parameters than registers.
  - Parameters are stored in a *block* in memory, and address of block passed as a parameter in a register.
    - This approach taken by Linux and Solaris.
  - Parameters are *pushed* onto a ***stack*** by the program and *popped* off the stack by the operating system.
  - Block and stack methods are popular because they do not limit the number or length of parameters being passed.

System Calls (11/11)



Types of System Calls (1/8)

- Many of today's operating system have hundreds of system calls.
  - Linux has 319 (or more) different system calls.
- Those system calls can be grouped roughly into five major categories:
  - Process control.
  - File management.
  - Device management.
  - Information maintenance.
  - Communications.

Types of System Calls — **Process Control** (2/8)

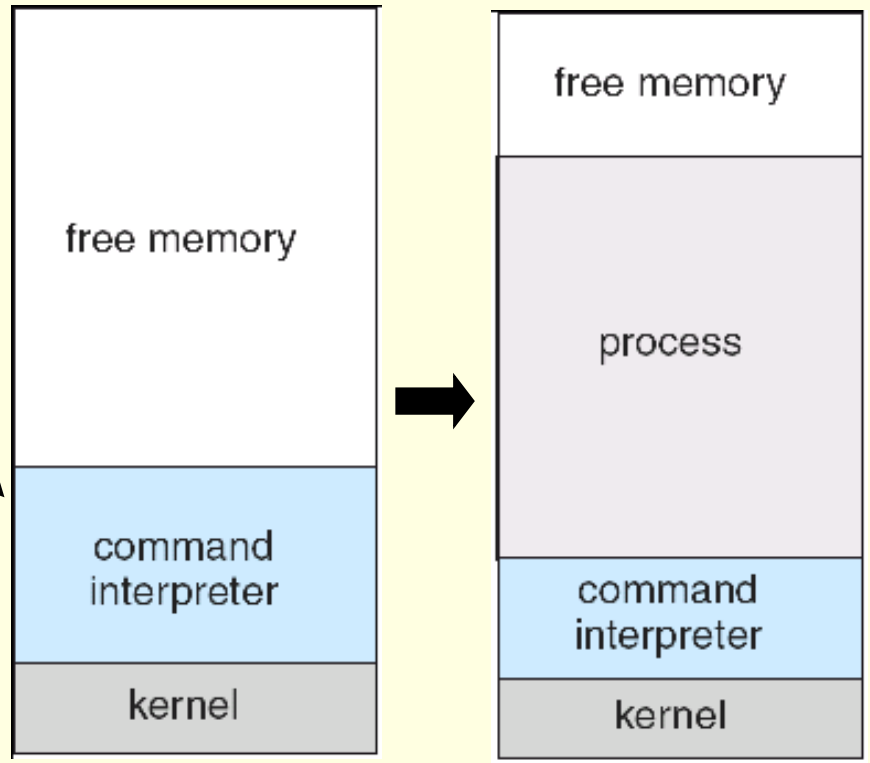
- `end` and `abort`:
  - A running program needs to be able to halt its execution either normally or abnormally.
  - The operating system then transfers control to the invoking command interpreter to read the next command.

- `load` and `execute`:
  - A process executing one program may want to load and execute another program.
  - The existing process can be lost, saved, or allowed to continue execution concurrently with the new process.
  - Chapter 6 (synchronization) discusses coordination of concurrent processes in great detail.

- `wait time/event`:
  - Having created new processes, we may need to wait for them to finish their execution.
  - We may want to wait for a certain amount of time to pass.
  - We may want to wait for a specific event to occur.

Types of System Calls — Process Control (3/8)

- Two popular variations in process control:
  - **Single-tasking system**: the MS-DOS operating system.



It has a command interpreter that is invoked when the computer started.

The command interpreter loads the program into memory and run the process.

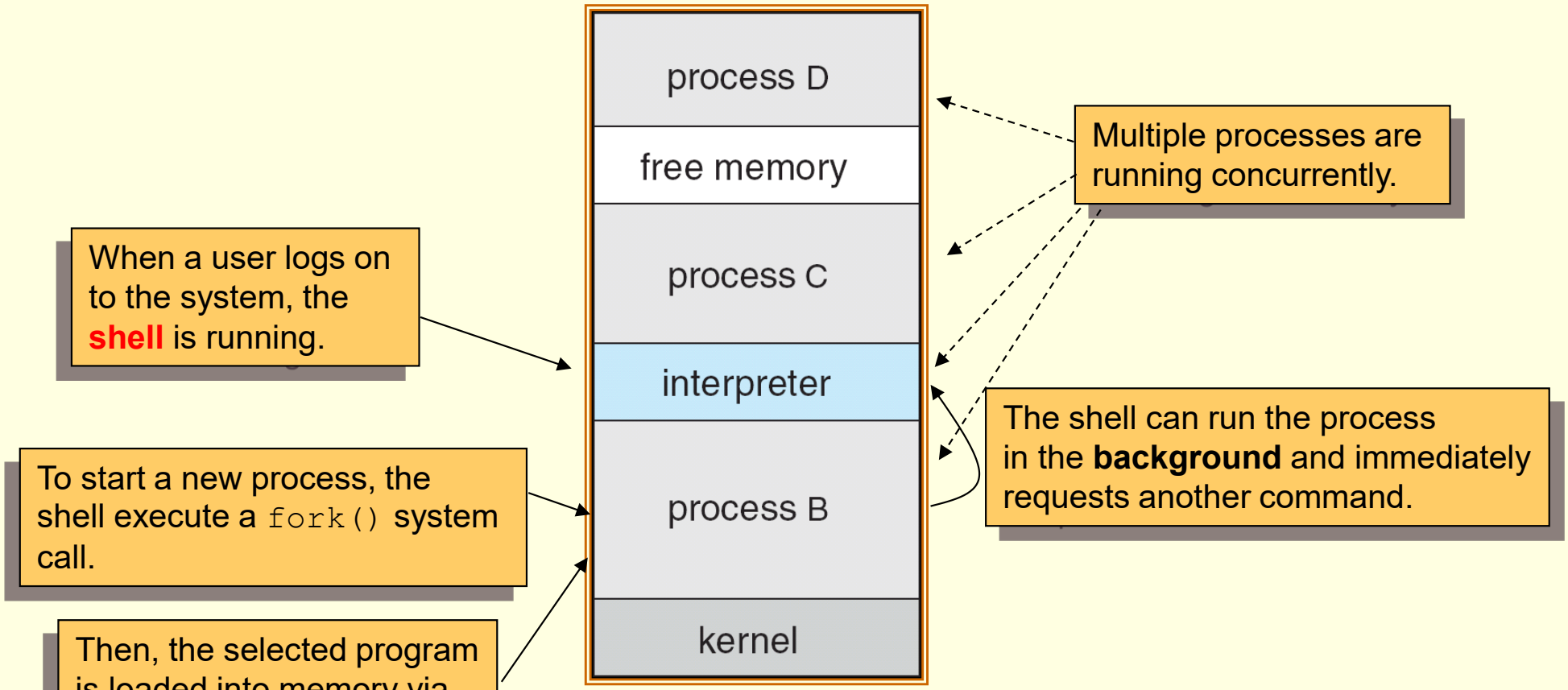
May writing over most of itself to give the process as much memory as possible.

When the process terminates, the interpreter reloads itself from disk and wait for the next user commands.

Only one process and can not create a new process

Types of System Calls — Process Control (4/8)

- **A multitasking system**: the FreeBSD (derived from Berkeley UNIX).



Chapter 3 discusses the `fork()` and `exec()` system calls

Types of System Calls — **File Management** (5/8)

- `create` and `delete`:
  - Able to create and delete files/directories.
- `open` and `close`:
  - Able to open and close a file.
- `read`, `write`, and `reposition`:
  - Able to read, write, or skipping to the end/head of the file.
- Other system calls for obtaining/setting file/directory attributes.

Types of System Calls — **Device Management** (6/8)

- The various **resources** (memory, disks, file, …) controlled by the operating system can be thought of as **devices**.
- To access a resource, a process has to:
  - First `request` the device, to ensure **exclusive** use of it.
  - Then we can `read`, `write`, and `reposition` the device.
  - After we are finished with the device, we `release` it.
  - The similarity between I/O devices and files is so great that many operating systems (UNIX) merge the two into a combined file-device structure.
    - A set of system calls is used on files and devices.
    - Sometimes, I/O devices are identified by special file names.

Types of System Calls — **Communication** (7/8)

- There are two common models of interprocess communication:
  - **Message-passing model**:
    - The communicating processes (may be on different computers) exchange messages with one another to transfer information.
    - Client and server (daemon) architecture.
      - Client: ask for connecting communication.
      - Server: wait for connection.
    - Require system calls to `build up`/`terminate` connection, `read`, and `write` messages.
  - **Shared-memory model**:
    - Processes use `shared memory create/attach` system calls to create and gain access to regions of memory owned by other processes.
    - They can then exchange information by reading and writing data in the shared areas.

Types of System Calls — Communication (8/8)

- Message passing:
  - Is useful for exchanging smaller amounts of data, because no conflicts need be avoided.
  - Is easy to implement.

- Shared memory:
  - Allows maximum speed of communication, since it can be done at memory speeds when it takes place within a computer.
  - However, problems exist in the areas of protection and synchronization between the processes sharing memory.

System Programs (1/3)

- A perspective of operating systems is a collection of **system programs**.
  - System programs provide a convenient environment for program execution and development.
  - Most users' view of the operation system is defined by system programs, not the actual system calls.
  - **Some of them are just user interfaces to system calls!!**
- Categories of system programs:
  - **File manipulation**:
    - Create, delete, copy, rename, print, dump, list, and generally manipulate files and directories

System Programs (2/3)

- **File modification**:
  - Text editors to create and modify files.
- **Programming-language support**:
  - Compilers, assemblers, debuggers and interpreters sometimes provided.
- **Program loading and execution:**
  - Loaders to load assembled or compiled programs into memory for execution.
- **Communications:**
  - Provide the mechanism for creating virtual connections among processes, users, and computer systems.

System Programs (3/3)

- In addition to system programs, application programs are supplied to solve common problems or perform common operations.
  - Web browsers, word processors, database systems, games …
- The view of the operating system seen by most users is defined by the application and system programs, rather than the actual system calls.

Operating System Design (1/2)

- The first is to define **requirements** and **specifications**.
- However, there is no unique solution to the problem of defining the requirements for an operating system.
  - Requirements can be affected by choice of hardware, type of system.
    - Handheld devices vs. PCs.
    - Single process vs. multitasking.
- The requirement can be divided into *user goals* and *system goals*.
  - User goals – operating system should be convenient to use, easy to learn, reliable, safe, and fast.
  - System goals – operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient.

Operating System Design (2/2)

- One important principle of system design is **the separation of *policy* from *mechanism***.
  - **Policy: What** will be done?
  - **Mechanism: How** to do it?
  - Example: *timer* is a **mechanism** for ensuring CPU protection, but deciding how long the timer is to be se is a a **policy** decision.
- Flexibility of the separation:
  - Policies are likely to change across places or over time.
  - The separation enables a change in policy to redefine certain policy parameters rather than changing the underlying mechanism.
- Most of Windows services mix mechanisms with policies to enforce a global look and feel.

Operating System Implementation (1/2)

- Traditionally, operating systems have been written in assembly language.
- Now, they are most written in higher-level languages such as C or C++.
  - The code can be **written faster** and is **easier to understand** and **debug**.
  - System is **easier to port** (to move to some other hardware).
    - The Linux operating system is written mostly in C and is available on a number of different CPUs, including Intel 80x86, Motorola 680X0, …
- Previous comments on higher-level languages: reduced speed and increased storage requirements.
  - Modern compiler techniques can perform complex analysis and optimizations that produce excellent code.

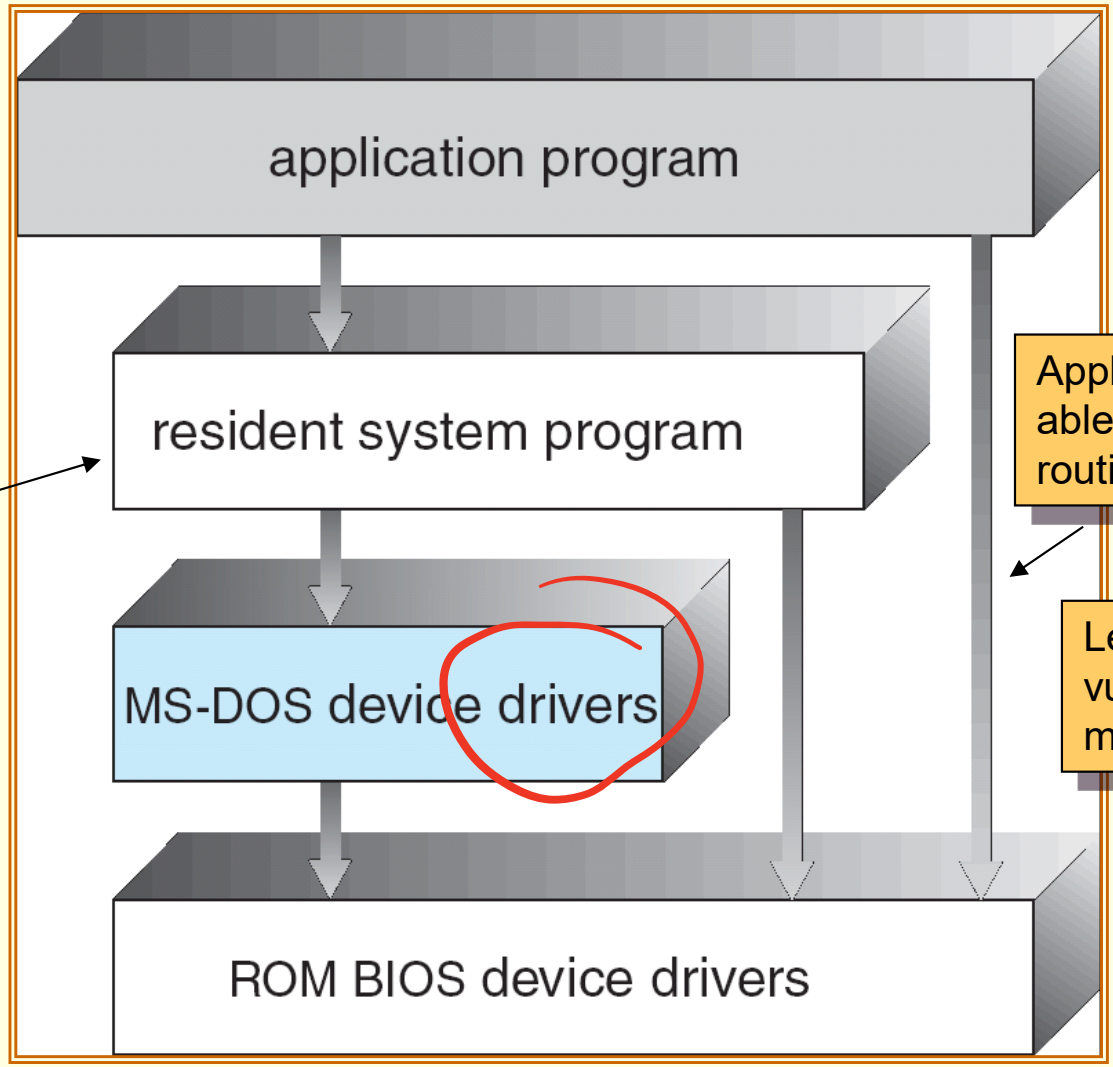
Operating System Implementation (2/2)

- Moreover, major performance improvements in operating systems (and other systems) are more likely to be the result of better **data structures** and **algorithms** than of excellent assembly-language code.
- Should pay more attentions on the **memory manager** and the **CPU scheduler**.
  - They are probably the most critical routines.

Operating-System Structure
Simple Structure (1/11)

- A common approach to implement an operating system is to partition the task into small components.
  - **Rather than have one monolithic system!!**
  - These components are interconnected and meld into a kernel.
  - But…
- **Simple structure**:
  - Many commercial system do not have well-defined structures initially.
    - Started as small, simple, and limited systems and then grew beyond their original scope.
    - For example, MS-DOS.

Operating-System Structure
Simple Structure (2/11)



The interfaces and levels of functionality are not well separated.

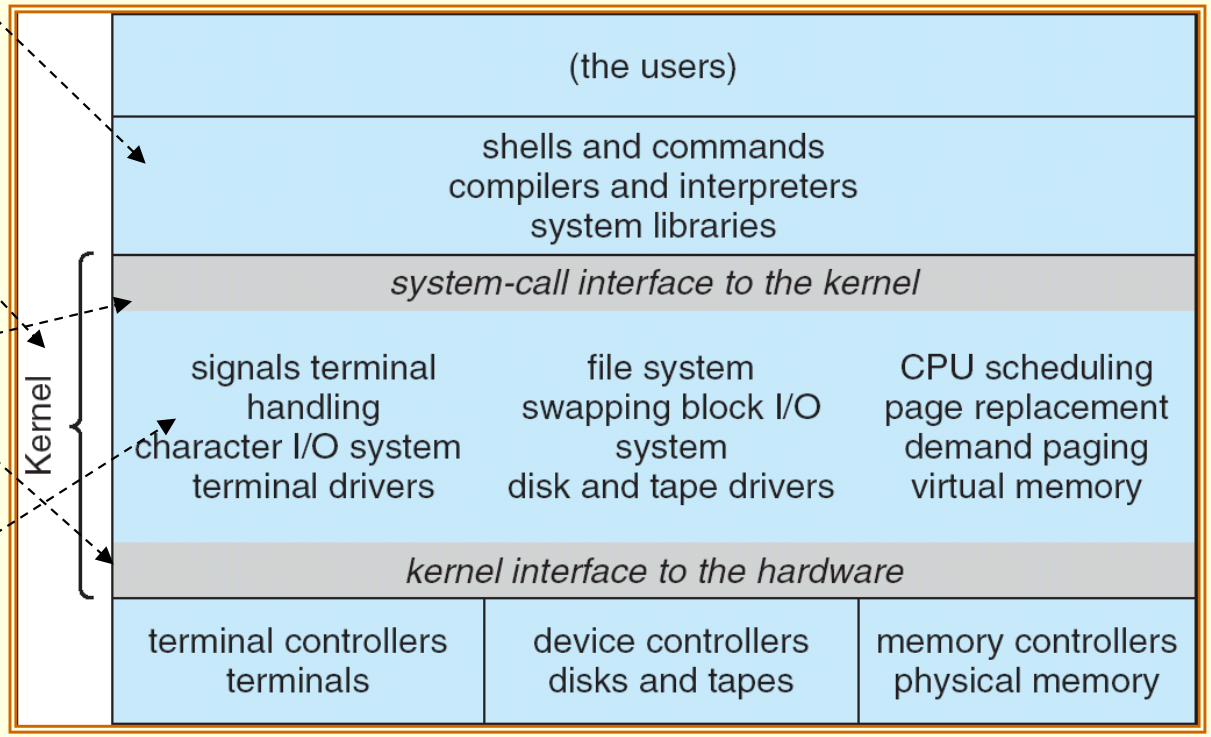
Application programs are able to access the basic I/O routines to access devices

Leave MS-DOS vulnerable to malicious programs

MS-DOS layer structure.

Operating-System Structure
Simple Structure (3/11)

- UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring.
- The UNIX OS consists of two separable parts:
  - **System programs**.
  - **The kernel**.



Interfaces to the bare hardware and system (application) programs

The kernel provides a lot of services, combined into **one monolithic level**.

Very difficult to implement and maintain.

Operating-System Structure

Layered Approach (4/11) (模組化/效率低)

- With the improvements of hardware and programming techniques, operating systems can be broken into pieces of components.
  - That is **modular** operating systems.
    - Information hiding: hide the internal implementation detail of modules and provide external access **interfaces**.

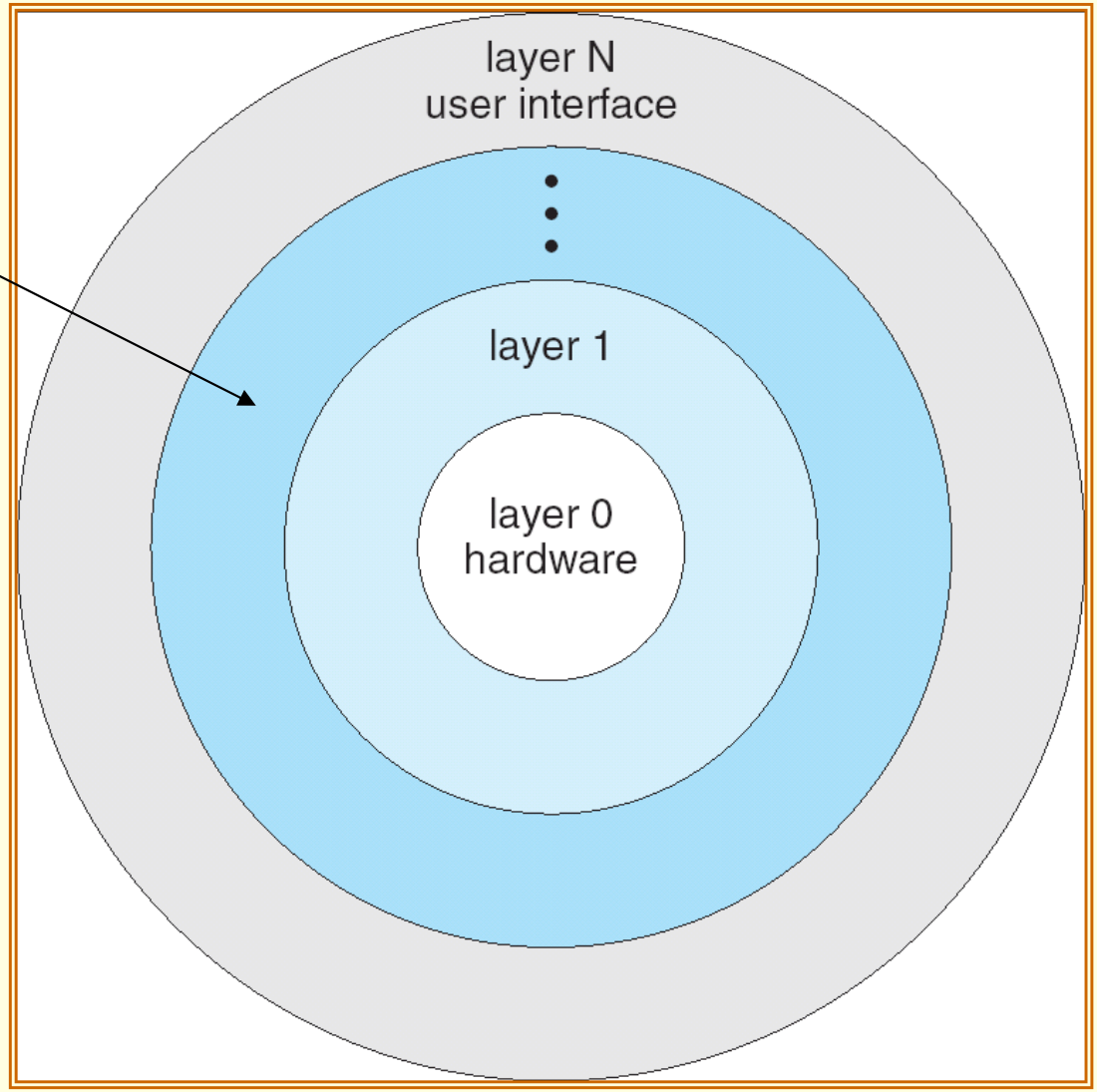
- One way of modular system: **layered approach**.
  - The operating system is divided into a number of layers (levels), each built on top of lower layers.
  - The bottom layer (layer 0), is the hardware.
  - The highest (layer N) is the user interface.

Operating-System Structure
Layered Approach (5/11)

好 maintain
只能讀前一層/呼叫
下一層

Layer *M* consists of data structures and a set of routines that can be invoked by higher-level layers.

Layer *M*, in turn, can invoke operations on lower-level layers.

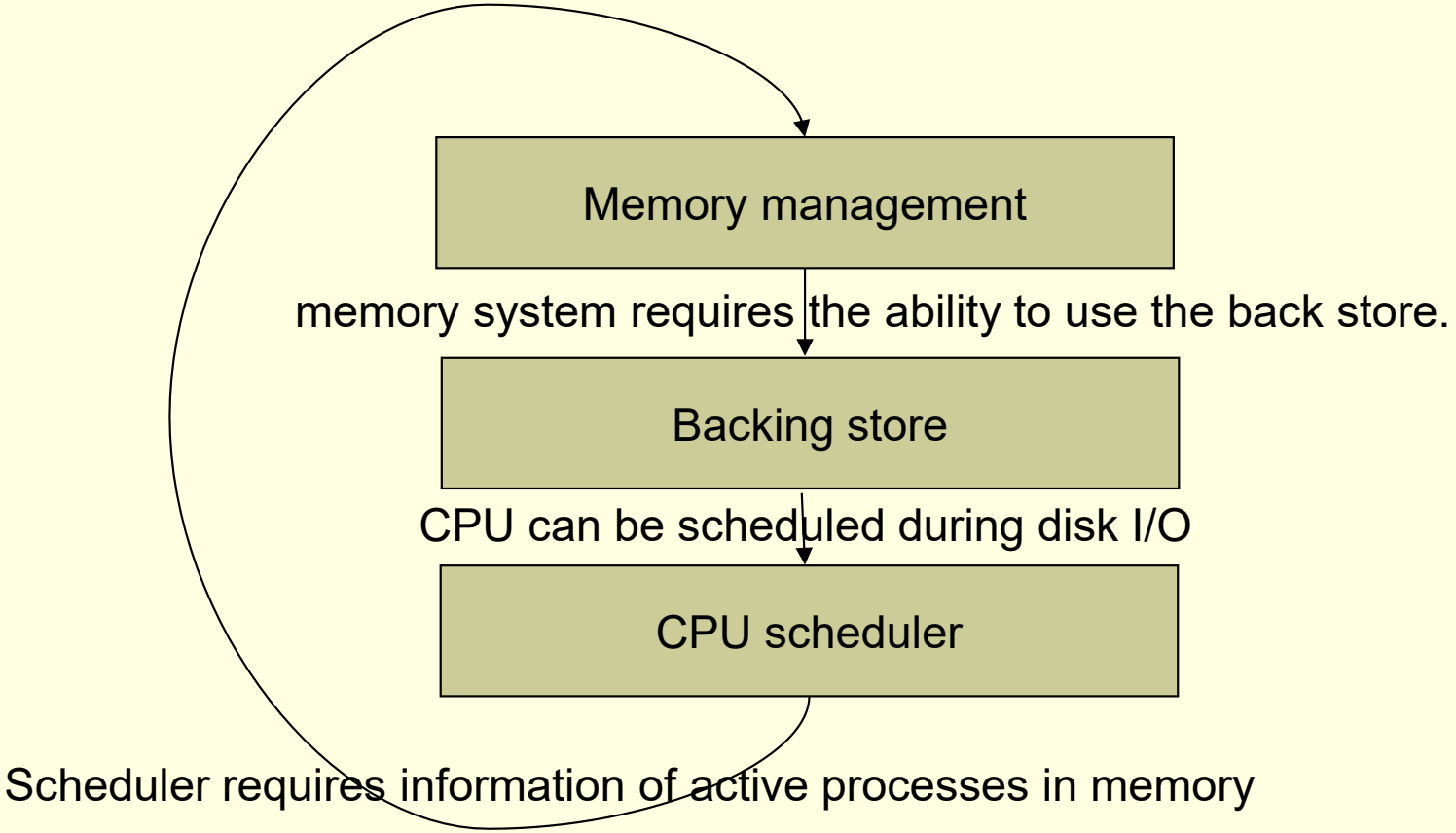


Operating-System Structure Layered Approach (6/11)

- The main **advantage** of the layered approach:
  - Simplicity of construction and **debugging**.
  - Layer-by-layer debugging, starting from layer 0.
  - If an error is found during the debugging of a particular layer, the error must be on that layer.
- The major **difficulty** of the layered approach:
  - Because only lower layers operations can be invoked, appropriately defining the various layers is difficult.
    - **System services usually tangle together.**
  - Layered implementation tend to be less efficient.
    - A function call on the top layer can lead to many lower-layer calls.
    - Function calls need to pass (redundant) parameters.
- Recently, fewer layers with more functionality are being designed.
  - Providing the advantages of modularization.
  - Avoiding the difficulties of layer definition and interaction.

Operating-System Structure Layered Approach (7/11)

- Example of tangled layers:



Operating-System Structure
Microkernels (8/11)

interprocess communication 須經過 kernel，效率低

- As operating systems expanded, the kernel became large and difficult to manage.
- In the mid-1980s, CMU developed an operating system called **Mach** that modularized the kernel using the **microkernel** approach.
  - Micro → removing all nonessential components from the kernel and implementing them as system and user-level programs (servers).
  - Typically, microkernels provide **process** and **memory** management, and a **communication** facility.
  - The client program and services communicate indirectly by exchanging message with the microkernel.

Operating-System Structure
Microkernels (9/11)

- **Benefits**:
  - Easier to include new operating system services to a microkernel.
    - Do not require modification of the kernel.
  - The small kernel makes it easier to port to new hardware architectures.
  - More reliable and secure (less code is running in kernel mode).
- **Problems**:
  - Performance overhead of user space to kernel space communication.
  - Initial Windows NT (a micorkernel organization) → Windows NT 4.0 (moving layers from user space to kernel space).

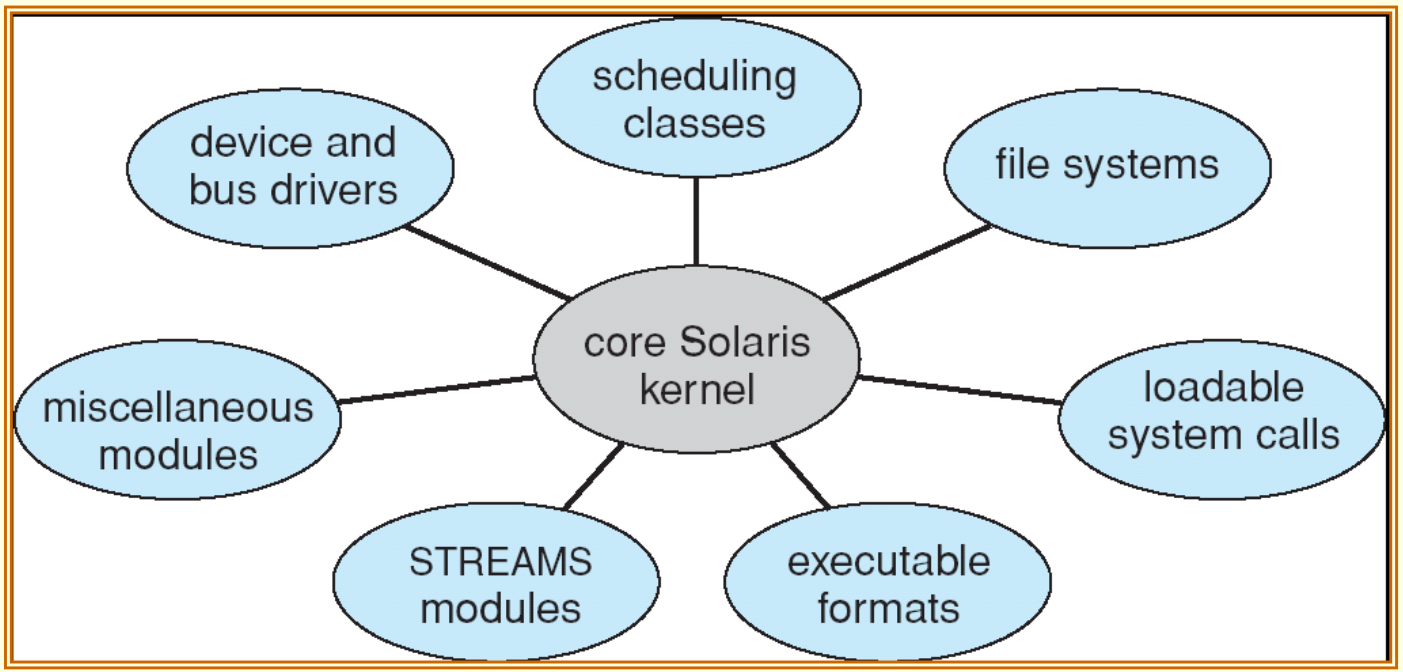
↳ 為了快一點

Operating-System Structure
Modules (10/11)

主流

- A better methodology for operating-system design involves using object-orient programming techniques to create a **modular kernel**.
  - Consists of a **core kernel**, and system service as **kernel modules**.
    - Each module talks to the others over known **interfaces**

比 layered 自由



比 micro 效率高

Operating-System Structure
Modules (11/11)

- Moreover, modules (system services) can be linked into the system either during boot time or **during run time** (that is, loaded as needed within the kernel).
  - Load different file system (ext2fs, FAT32 or NTFS) as needed, to save main memory.
- The module structure is similar to layered (communicate with interfaces) and microkernel approaches (a core), but with more flexible.
  - Any module can call any other module, but the layered approach can not.
  - Is efficient than microkernel approach because modules are in the kernel space and do not need to invoke message passing to communicate.
- The strategy of dynamically loadable modules is very popular in modern UNIX-based operating systems, such as Linux.

Operating System Generation (1/3)

- Operating systems are normally distributed on disk or CD-ROM.
  - They are designed to run on **any** class of machines **with different hardware configurations**.
- To generate a system for each specific computer site, a special program – **`SYSGEM`** – is needed.
  - It determines computer components by:
    - Reading a given file.
    - Asking the operator of the system for hardware information.
    - Probes the hardware directly.

Operating System Generation (2/3)

- The information must be determined:
  - **CPU**:
    - What CPU is to be used?
    - Number of CPUs.
    - Has extended instruction sets or floating point arithmetic.
  - **Memory**:
    - Size.
  - **Devices**:
    - Type and model.
    - Interrupt number.
  - **Operating-system options**:
    - Maximum number of processes to be supported.
    - CPU-scheduling algorithm.

Operating System Generation (3/3)

- Once the information is determined …
  - Source code of the operating system can be **modified** and completely **compiled** to produce a tailored operating system.
    - System generation is slower.
    - But more specific to the underlying hardware.
  - Or, the description can cause the selection of **modules** from a **precompiled library**, which are linked together to form the operating system.
    - Because the system is not recompiled, system generation is faster.
    - The resulting system may be general.
    - Easy to modify the generated system as the hardware configuration changes (such as, add a new hardware).

System Boot (1/2)

- The generated operating system must be made available by the hardware.
- How does the hardware know where the kernel is and how to load that kernel??
- ***Booting*** – the procedure of starting a computer by loading the kernel.
  - Power up or reset.
- Need a ***bootstrap program*** to*:*
  - Locate the kernel on the disk.
  - Load it into memory.
  - Start its execution.
  - A simple code stored in ROM or EPROM.
    - At a fixed location so that can be loaded and executed when computer is on.
  - But before that, it first initializes all aspects of the system:
    - CPU registers, device controller, the contents of main memory …

System Boot (2/2)

- Some computer systems (such as PCs) use a two-step booting process:
  - A simple bootstrap loader fetches a more complex boot program from disk.
  - Which in turn loads the kernel.
- The boot program stored in the ***boot block*** (a fixed location on disk) is usually sophisticated and modifiable and is to load an (or different) operating system into memory and begin its execution.
  - Then the operating system is said to be **running**.

End of Chapter 2

Chapter 3: Processes Concept

Chien Chin Chen

Department of Information Management
National Taiwan University

Outline

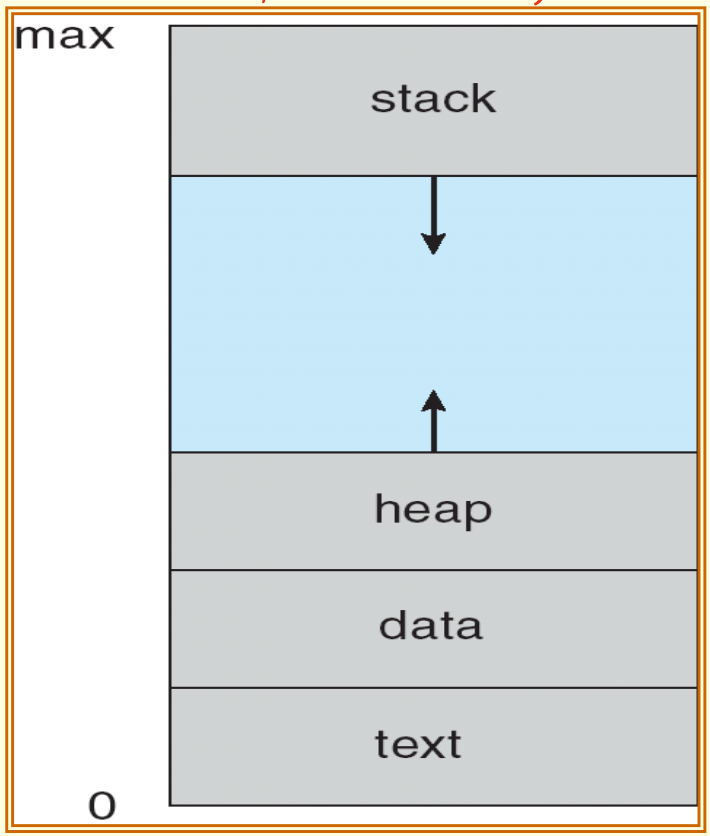
- Overview
- Process Scheduling
- Operations on Processes
- Interprocess Communication
- Examples of IPC Systems
- Communication in Client-Server Systems

Process Concept (1/3)

- The name of CPU activities varies in different systems:
  - Batch system – *jobs*.
  - Time-shared systems – *user programs* or *tasks*.
- Textbook uses the terms ***job*** and ***process*** almost interchangeably, but prefers *process*.
  - However, much of operating-system theory and terminology was developed during the batch era.
  - *Job* (such as job scheduling) would be used to avoid misunderstanding.

Process Concept (2/3)

- Process: 正在執行中的program，比program大多了 (program只有code)
  - A program in execution.
  - Execution must progress in **sequential** fashion.
- A process in memory includes:
  - ***Text section***: the program code.
  - ***Stack***: contains temporary data (local variable, function parameters …).
  - ***Data section***: contains global variables.
  - ***Heap***: used for dynamical memory allocation.
  - ***Program counter***: the address of next instruction.
  - ***Register status***.



Process in memory

Process Concept (3/3)

- A program is not a process:
  - A program is a ***passive*** entity; a list of instruction stored on disk.
  - A process is an ***active*** entry.
    - A program becomes a process when an executable file is loaded into memory.
- Two (or more) processes may be associated with the same program.
  - For example, a user may invoke many copies of the web browser program.
  - Each of these is a separate process.
    - **While the text sections are equivalent, the data, heap, and stack sections vary.**

Process State (1/2)

- The ***state*** of a process is defined by the current activity of that process.
- Each process may be in one of the following states:
  - ***New***: the process is being created. 在硬碟上有旦尚未載入記憶體時
  - ***Running***: instructions are being executed.
  - ***Waiting***: the process is waiting for some event to occur.
    - Such as an I/O completion. 變數還沒到手,還不能給CPU跑
  - ***Ready***: the process is waiting to be assigned to a processor. 有資格使用CPU,被挑到會變成running
  - ***Terminated***: the process has finished execution.
- **Only one process can be running on any processor at any instance.**
  - Many processes may be *ready* and *waiting*.

Process State (2/2)

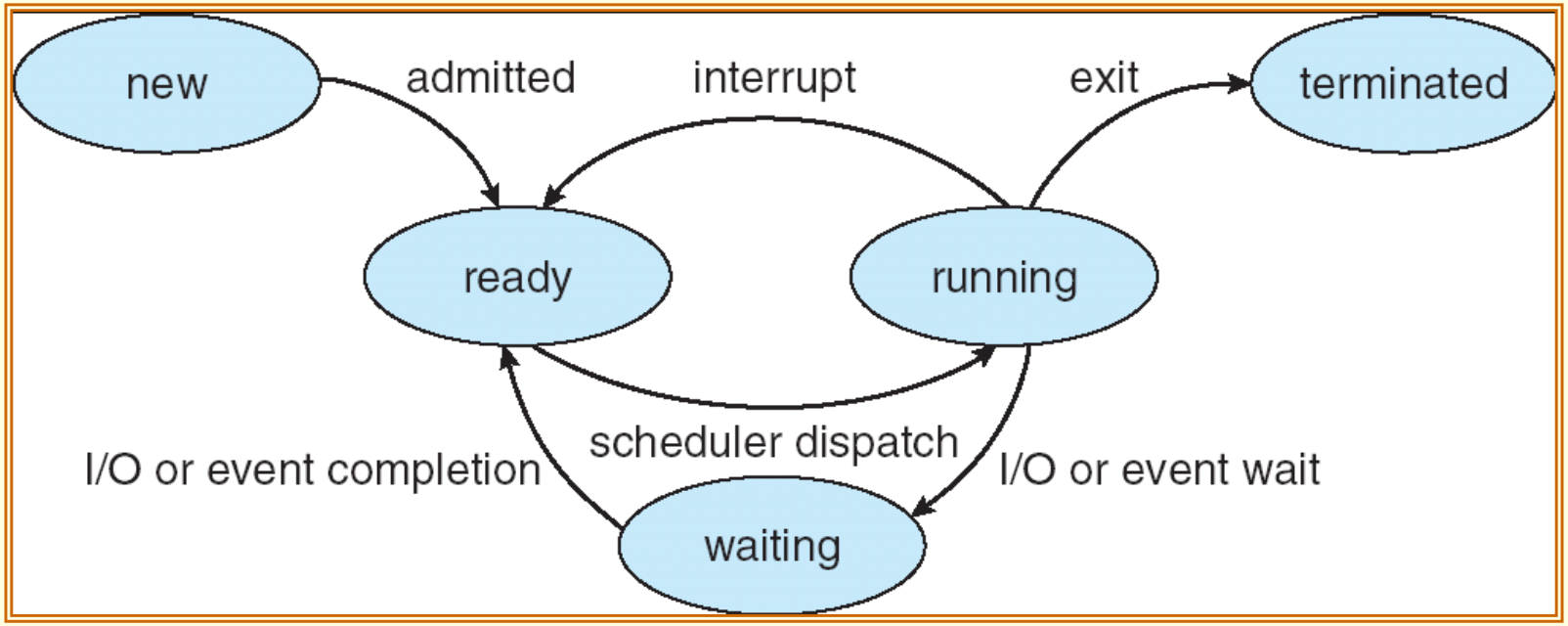
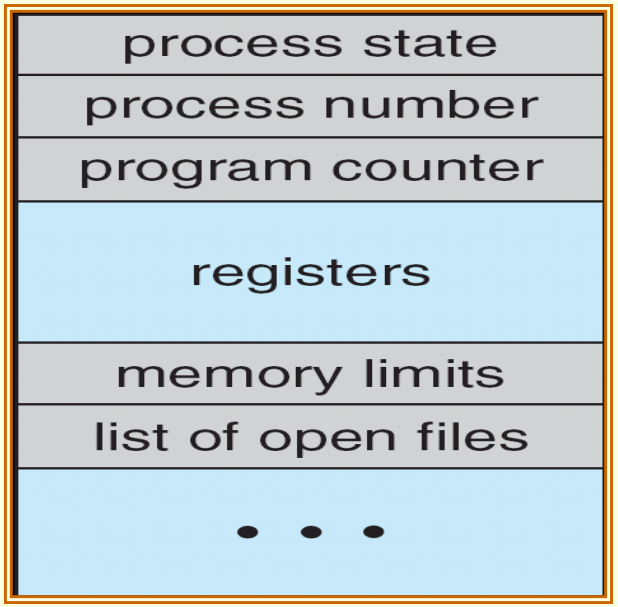


Diagram of Process State

Process Control Block

- ***Process control block*** (PCB): a representation of a process in the operating system.
- Information associated with each PCB:
  - **Process state**.
  - **Program counter**.
    - The address of next instruction.
  - **CPU registers**.
  - **CPU scheduling information**.
    - Process priority (chapter 5).
  - **Memory-management information** (chapter 8).
  - **Accounting information**.
    - E.g., amount of CPU time.
  - **I/O status information**.
    - Allocated devices, files, and so on.



Different processes have different PCB information.

Process Scheduling

- Multiprogramming:
  - To have some process running at all time.
  - To maximize CPU utilization.
- Time sharing:
  - To switch the CPU among processes so frequently.
  - Users can interact with each program while it is running.
- To do so … we need …
  - ***Process scheduler***:
    - Select an available process for execution on the CPU.
    - The rest have to wait until the CPU is free and can be rescheduled.

Scheduling Queues and Schedulers (1/7)

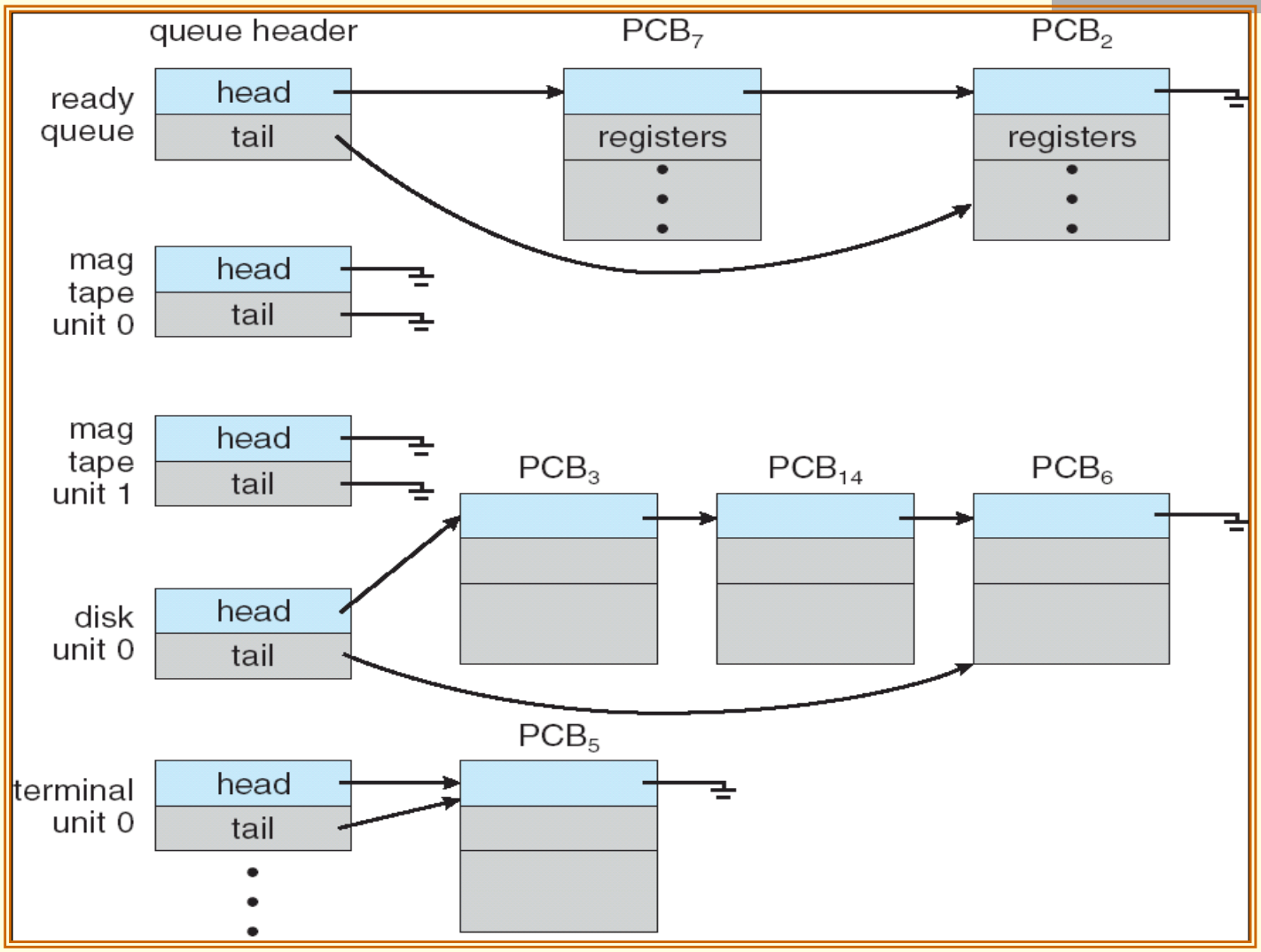
- The operating system usually contains a lot of **queues**.
  - Contain pointers to the first and final PCBs in the list (queue).
  - Each PCB includes a pointer field that points to the next PCB in the queue.
  - ***Job queue***: (舊時代的)
    - Processes are initially spooled to a mass-storage device (e.g., a disk), where they are kept for later execution.
    - Job scheduler later selects processes from this pool and load them into memory for execution.
    - Often in batch system. **On some operating systems, job queue and job scheduler may be absent**.
      - Such as UNIX and MS Windows.
      - Just put every new process in memory for execution.

Scheduling Queues and Schedulers (2/7)

- ***Ready queue***:
  - Contain the processes residing in main memory.
  - Those processes are **ready** and waiting to execute.

- Operating systems also include other queues.
  - ***Device queue***:
    - A list of processes waiting for a particular I/O device.
      - For instance, there are many processes make requests to a disk.

Scheduling Queues and Schedulers (3/7)

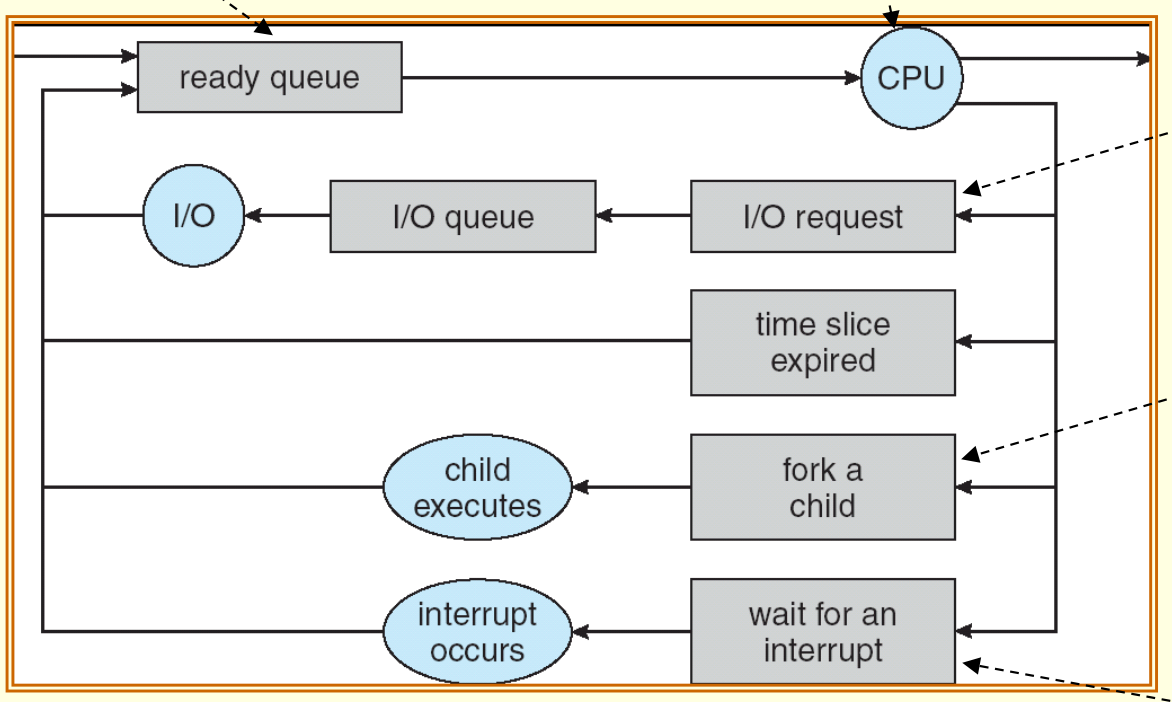


Ready Queue And Various I/O Device Queues

Scheduling Queues and Schedulers (4/7)

A new process is initially put in the ready queue (**ready** state).

Eventually, it will be selected (dispatched) for execution (**running** state).



The process may issue an I/O request, and switches to the **waiting** state.

The process may create a new sub-process and wait for the sub-process's termination (**waiting** state).

The process may be removed from the CPU by an interrupt, and be put back in the ready queue (**ready** state).

Processes migrate among the various queues.

When the process terminates, it is removed from all Queues and has its PCB and resources deallocated.

Scheduling Queues and Schedulers (5/7)

- A process migrates among the various scheduling queues throughout its lifetime.
  - The operating system must select process from these queues in some fashion.
- Two main schedulers:
  - ***Job scheduler*** (or **long-term** scheduler) – selects which processes should be brought into the ready queue.
  - ***CPU scheduler*** (or **short-term** scheduler) – selects which process should be executed next and allocates CPU.
- The primary difference between the schedulers: **frequency of execution**.
  - Often, the short-term scheduler executes at leas once every 100 ms.
  - The long-term scheduler executes much less frequently.

Scheduling Queues and Schedulers (6/7)

- The short-term scheduler must be **fast**.
  - If it takes 10 ms to decide to execute a process for 100 ms, then 10 / (10 + 100) = 9% of the CPU is being wasted for scheduling the work.
- However, because of the longer interval between executions, the long-term scheduler can afford to take more time to decision.
- The long-term scheduler controls the degree of multiprogramming.
- In general, processes can be described as either ***I/O bound*** or ***CPU bound***.
  - A **I/O-bound process** spends more of its time doing I/O.
  - A **CPU-bound process** uses more of its time doing computations.
  - A good long-term scheduler should select a good process **mix** of I/O-bound and CPU-bound processes that the system will be balanced.

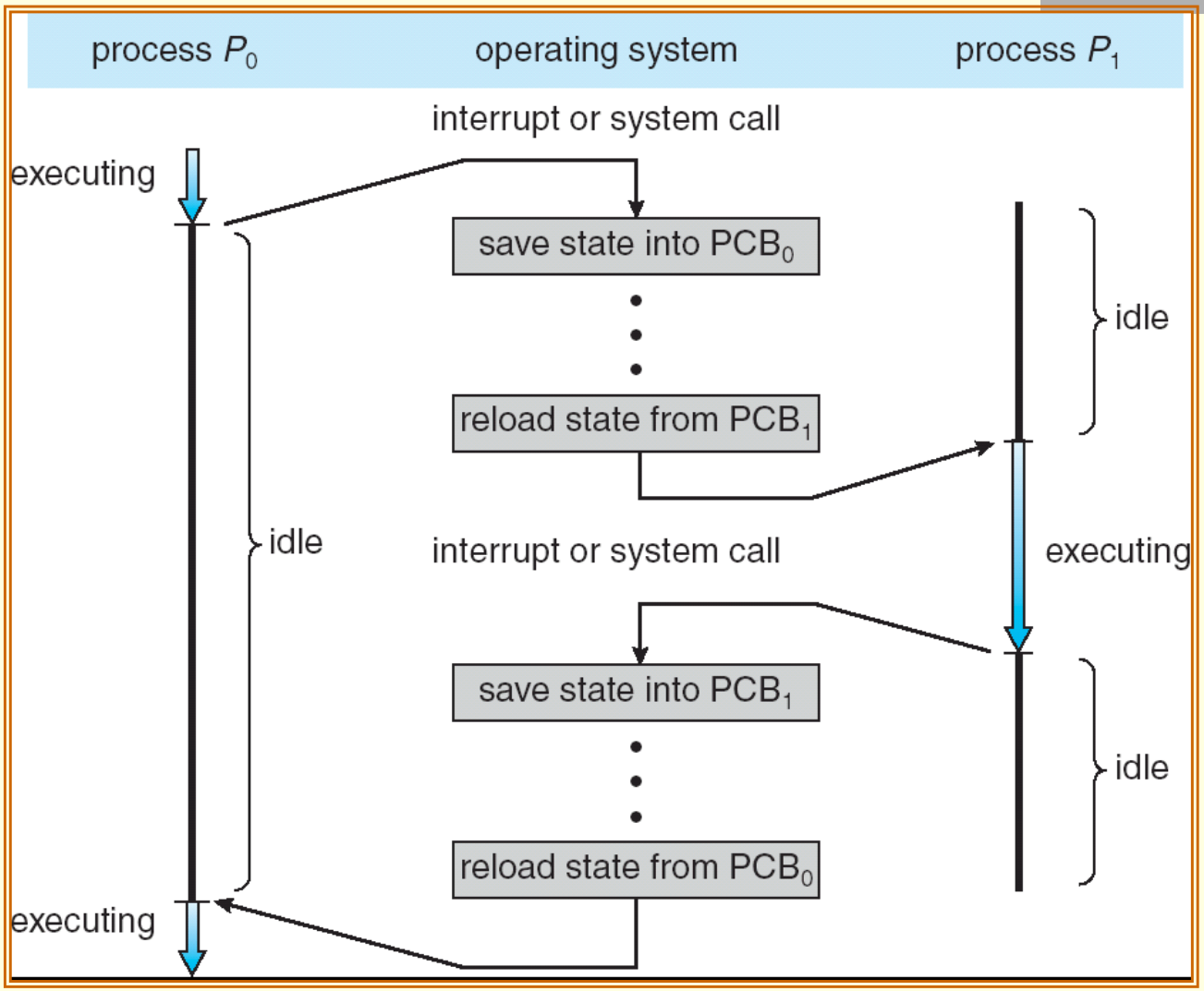
Scheduling Queues and Schedulers (7/7)

- For systems with no long-term schedulers:
  - The stability of the systems may depend on the self-adjusting nature of human users.
    - If the performance declines to unacceptable levels on a multi-users system, some users will simply quit.

Context Switch (1/3)

- When CPU switches to another process, the system must **save** the **context** of the old process and **load** the saved **context** for the new process.
  - E.g., interrupts cause the operating system to change a CPU from its current task to run a kernel routine.
- Such a switching is known as a ***context switch***.
- The context is represented in the PCB of the process.
  - The value of the CPU registers.
  - The process state.
  - Program counter.
  - …

Context Switch (2/3)



Context Switch (3/3)

- **Context-switch time is overhead**; the system does no useful work while switching.

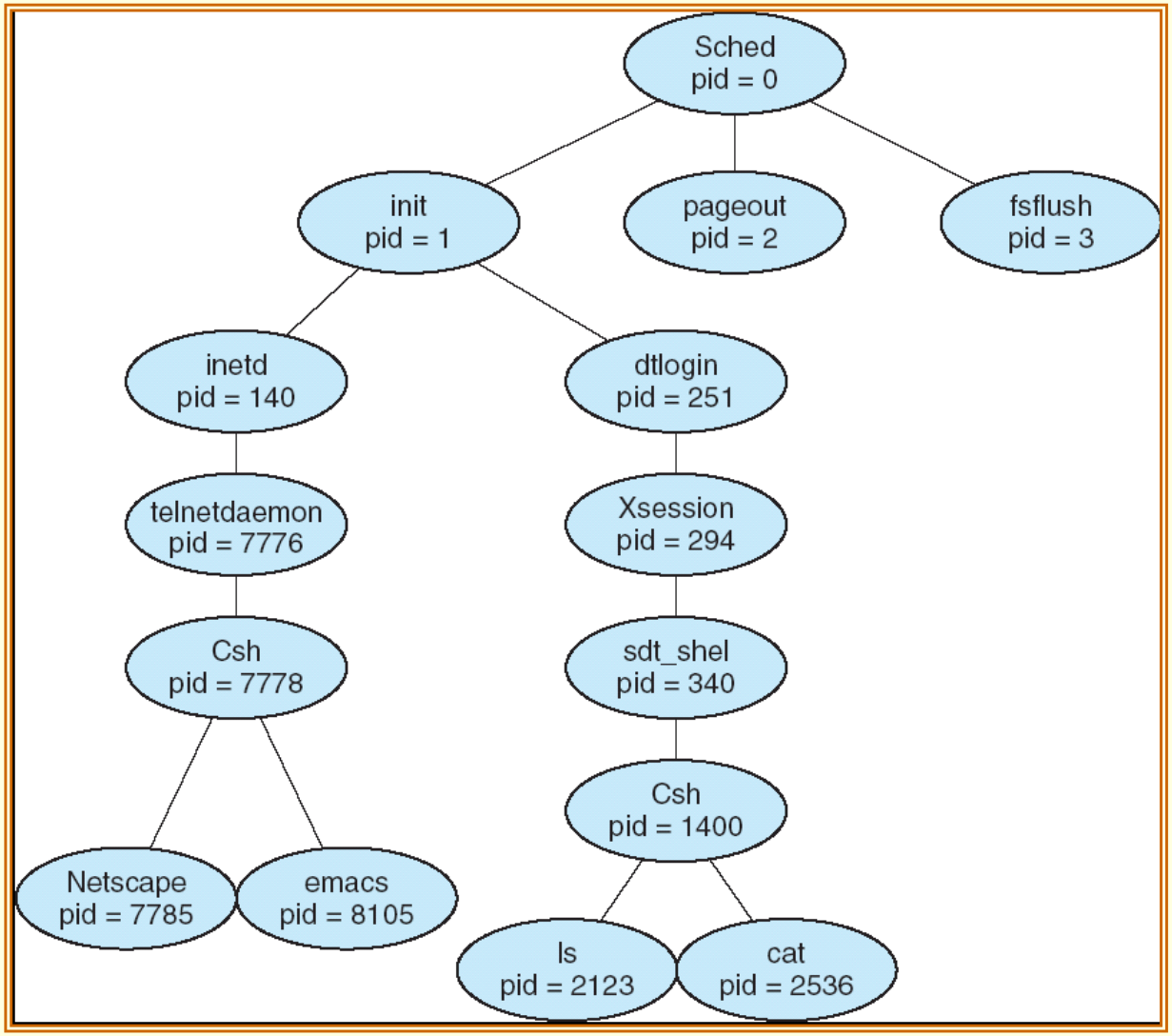
- Dependent on hardware support.
  - Some processors provide multiple sets of registers.
    - A context switch simply requires changing the pointer to the current register set.

Operations on Processes

Process Creation (1/8)

- A process may create several new processes, via a create-process system call.
- The creating process is called a **parent** process, and the new processes are called the **children** of that process.
  - New processes may in turn create other processes, forming a **tree** of processes.
- Most operating systems (UNIX and MS Windows) identify processes according to a unique process identifier (pid).
  - Usually an integer number.

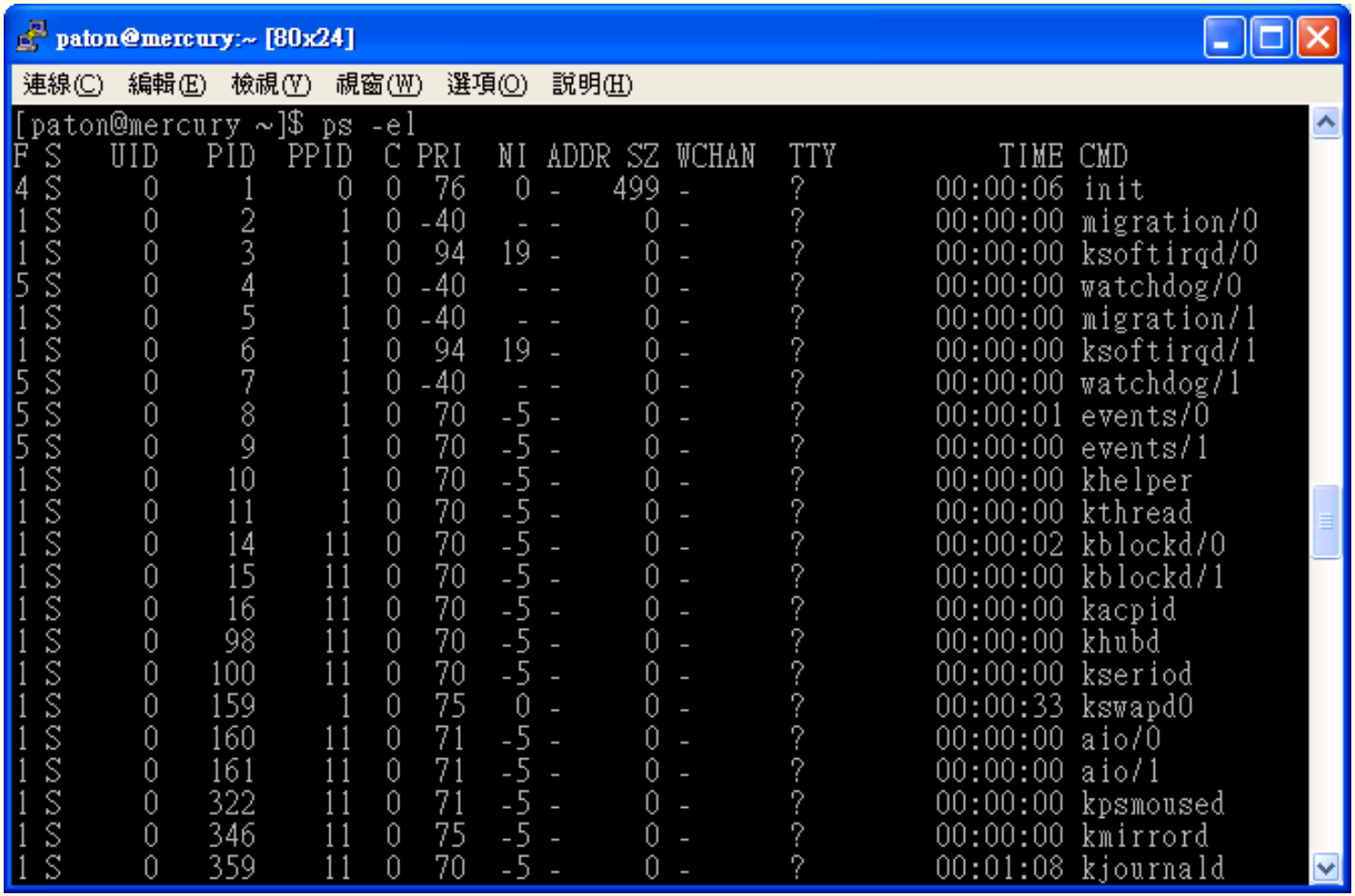
Process Creation (2/8)



A tree of processes on a typical Solaris

Process Creation (3/8)

- On UNIX, the `ps` command can be used to obtain the information of all process.



Process Creation (4/8)

- When parent process holds certain resources (files, I/O devices).
  - Parent and children share all resources.
  - Children share subset of parent's resources.
  - Parent and child share no resources.
- When a process creates a new process, two possibilities exist in terms of execution:
  - Parent and children execute concurrently.
  - Parent waits until children terminate.
- There are also two possibilities in terms of the address space of the new process:
  - Child duplicate of parent.
  - Child has **a program loaded into it**.

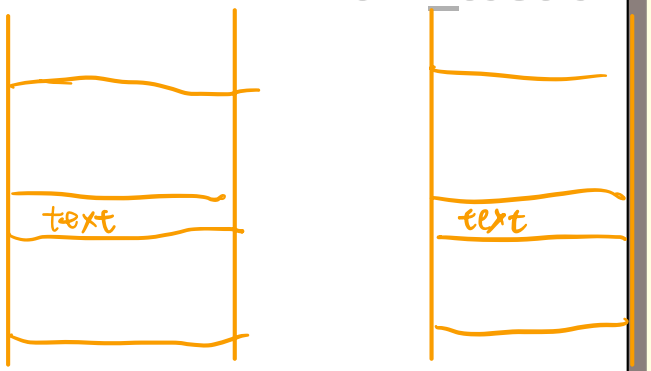
Process Creation (5/8)

- UNIX examples:
  - `fork()` system call creates new process (POSIX).
  - The new process consists of **a copy of the address space of the original process**.
  - Both process **continue (concurrently) execution** at the instruction after the `fork()`.
  - But … how to distinguish?
    - The return code for the `fork()` is zero for the new (child) process.
    - The process identifier (PID) of the child is returned to the parent.

Process Creation (6/8)

fork_test.c

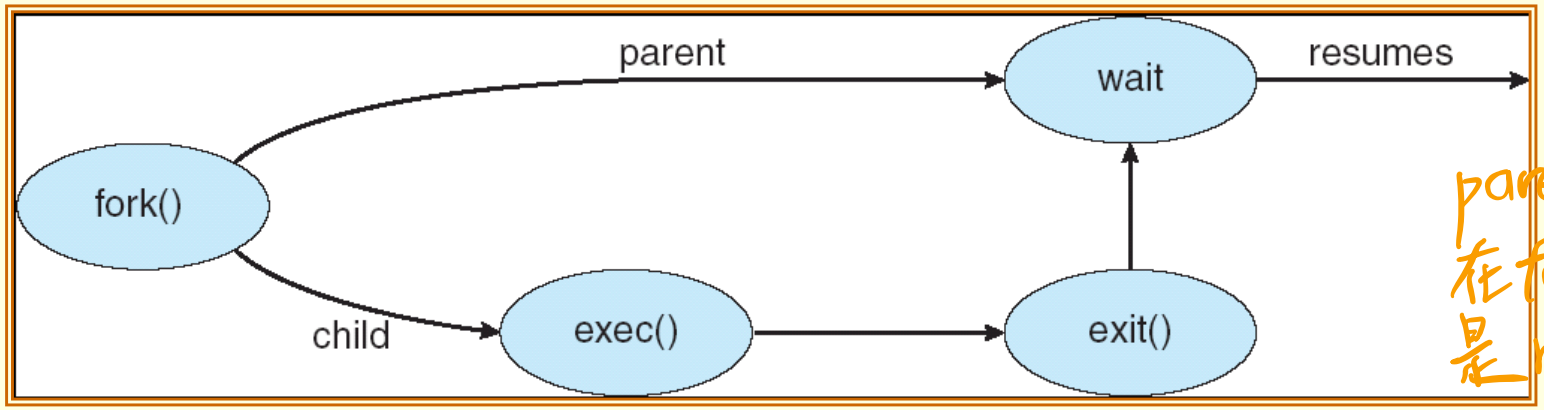
```
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>


int main()
{
    pid_t  pid;

    pid = fork(); /* fork another process */
    if (pid < 0) { /* error occurred */
          fprintf(stderr, "Fork Failed\n");
          return 1;  // exit(1);
    }
    else if (pid == 0) { /* child process */
          execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
            /* parent will wait for the child to complete */
          wait(NULL);
          printf ("Child Complete\n");
    }
    return 0; // exit(0);
}
```



Process Creation (7/8)

- Typically, the **`exec()`** (or its family, such as **`execlp()`**) system call (POSIX) is used after a `fork()` by **one of** the two processes to **replace the process's memory space with a new program**.
  - For example, the child process overlays its address space with UNIX command `/bin/ls` using the `execlp()` system call.
- The parent can wait for the child process to complete with the **`wait()`** system call (POSIX).
  - When the child process completes, the parent process resumes from the call to `wait`.
- **`exit()`** system call: cause normal process termination (POSIX).
  - Or indirectly called by a `return` statement in `main()`.

Process Creation (8/8)



parent & child 在 fork 之下都是 ready

也

parent 會 wait child 跑完才能回到 ready

For OS Course [執行中] - Oracle VM VirtualBox

檔案 機器 檢視 輸入 裝置 說明

paton@paton-VirtualBox: ~/OS_exercises/CH03

```
paton@paton-VirtualBox:~/OS_exercises/CH03$ gcc -o fork_test.out fork_test.c
paton@paton-VirtualBox:~/OS_exercises/CH03$ ./fork_test.out
總計 72
drwxrwxr-x 2 paton paton 4096  3月 14 10:38 .
drwxrwxr-x 3 paton paton 4096  3月 13 19:28 ..
-rw-rw-r-- 1 paton paton  394  3月 13 19:38 consumer.c
-rwxrwxr-x 1 paton paton 8768  3月 13 19:47 consumer.out
-rw-rw-r-- 1 paton paton  499  3月 14 10:38 fork_test.c
-rwxrwxr-x 1 paton paton 8856  3月 14 10:38 fork_test.out
-rw-rw-r-- 1 paton paton  885  3月 13 19:51 pipe.c
-rwxrwxr-x 1 paton paton 9096  3月 13 19:51 pipe.out
-rw-rw-r-- 1 paton paton  806  3月 13 19:37 producer.c
-rwxrwxr-x 1 paton paton 8816  3月 13 19:47 producer.out
Child Complete
paton@paton-VirtualBox:~/OS_exercises/CH03$
```

Process Termination

- Process executes last statement and asks the operating system to delete it by using the **`exit()`** system call.
  - Can return a status value (typically an integer) to its parent process (via the **`wait()`** system call).
  - Process' resources are deallocated by operating system.

- Parent may terminate execution of children processes (**`kill()`**, **POSIX**) when: parent 可以主動殺 child process
  - Child has exceeded allocated resources.
  - Task assigned to child is no longer required.
  - If parent is exiting.
    - Some operating systems do not allow child to continue if its parent terminates.
    - All children terminated - ***cascading termination***.

Interprocess Communication

Cooperating Processes (1/4)

- Processes may be **independent** processes or **cooperating** processes.
  - **Independent** process cannot affect or be affected by the execution of another process.
  - **Cooperating** process can affect or be affected by the execution of another process.

- Reasons for process cooperation:
  - Information sharing.
  - Computation speed-up.
    - Can be achieved only if the system have multiple CPUs.
  - Modularity.
  - Convenience.
    - Users may work on many tasks (or sub-tasks) at the same time.

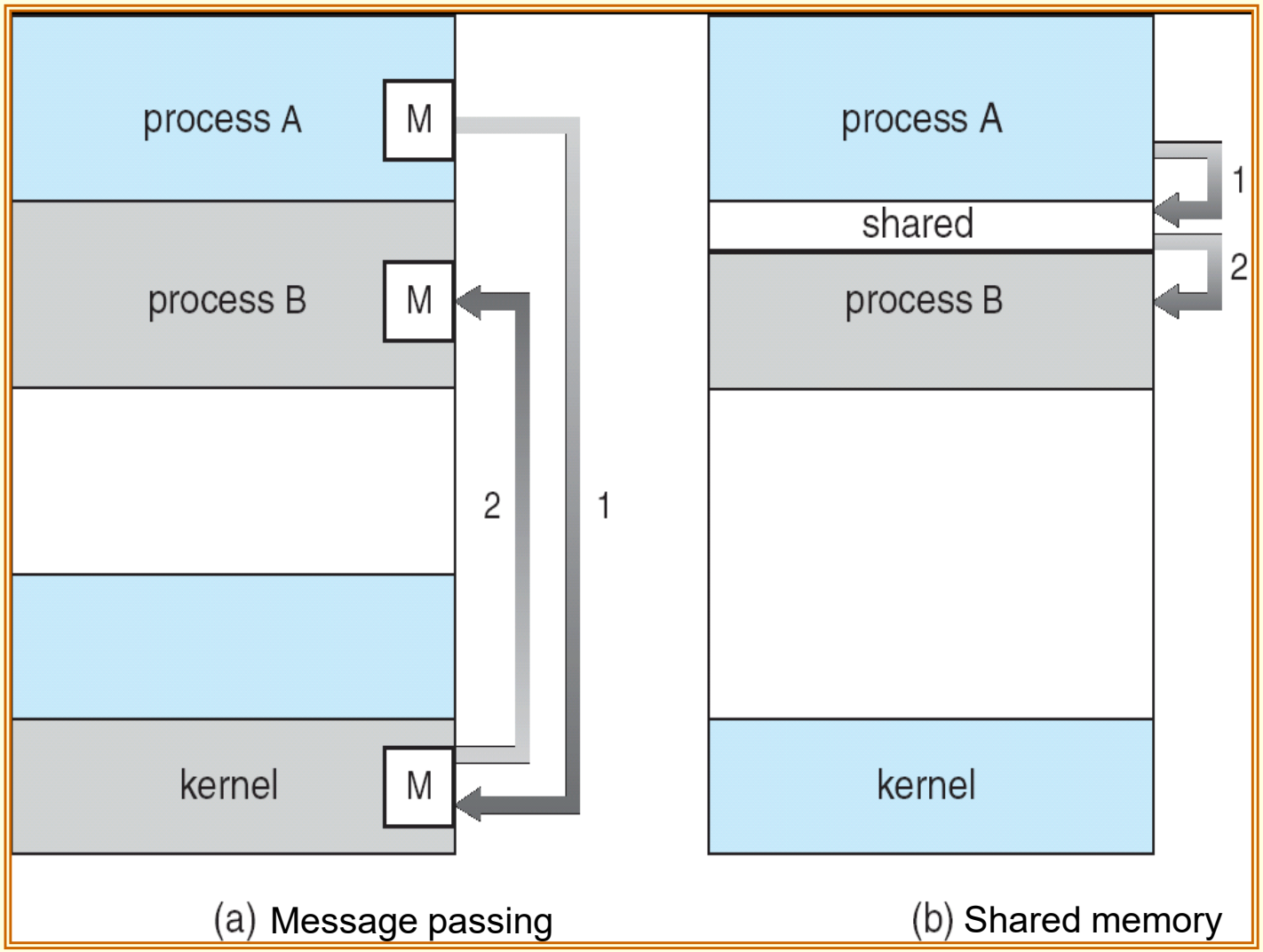
Cooperating Processes (2/4)

- ***Interprocess communication*** (IPC) is a mechanism that allows cooperating processes to exchange data and information.
- Two typical models of IPC:
  - ***Shared memory***.
    - A region of memory is shared by cooperating processes.
    - Processes can exchange information by reading and writing data to the region.
  - ***Message passing***.
    - Communication takes place by means of messages exchanged between the processes.
  - Both of the models are common and implemented in many operating systems.

Cooperating Processes (3/4)

- Message passing is useful for exchanging **smaller amounts of data**.
  - Because **no conflicts** need be avoided.
  - Usually **slow**, because message-passing systems are generally implemented as system calls.
    - Require the more time-consuming task of kernel intervention.
- Shared memory allows **maximum speed** and convenience of communication.
  - It can be done at memory speeds within a computer.
  - Is **faster** than message passing.
    - Once shared memory is established, all accesses are treated as routine memory access.
    - No assistance from the kernel is required.

Cooperating Processes (4/4)



Shared-Memory Systems (1/4)

- A process creates a shared-memory region residing in its address space.
- Other processes that wish to communicate using this shared-memory segment must attach it to their address space.
- Then, they can exchange information by reading and writing data in the shared area.
- The processes are responsible for ensuring that they are not writing to the same location simultaneously.
  - Chapter 6 will discuss synchronization to synchronize concurrent accesses.

Shared-Memory Systems (2/4)

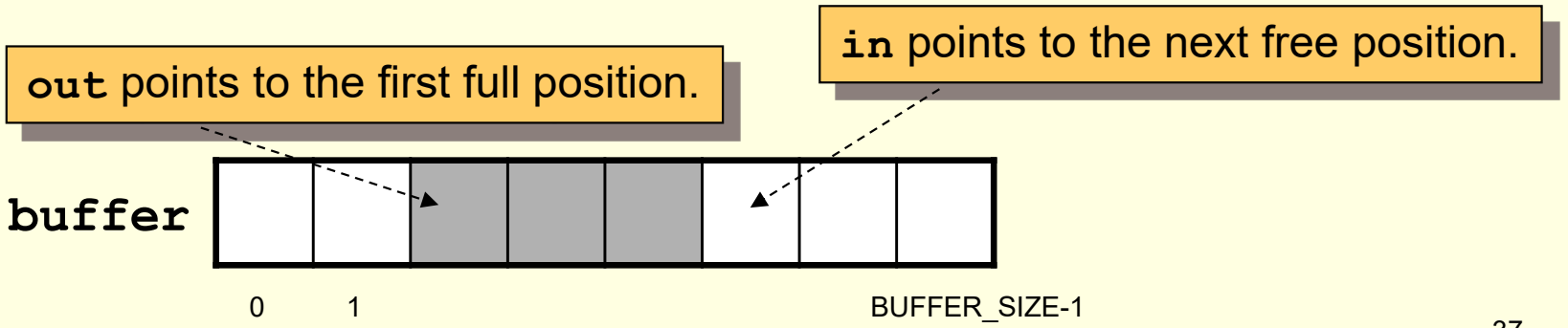
- Shared memory as a ***producer-consumer problem***:
  - **producer** process produces information that is consumed by a **consumer** process.
  - A **buffer** (shared memory) can be filled by the producer and emptied by the consumer.
- Two types of buffers:
  - *unbounded-buffer* places no practical limit on the size of the buffer.
    - The consumer may have to wait for new items.
    - But the producer can always produce new items.
  - *bounded-buffer* assumes that there is a fixed buffer size.
    - The consumer may have to wait for new items.
    - The producer have to wait if the buffer is full.

Shared-Memory Systems (3/4)

- Implementation:

```
#define BUFFER_SIZE 10
typedef struct {
        . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

- The buffer is implemented as a **circular array**:
  - `in=(in++)%BUFFER_SIZE`
  - `out=(out++)%BUFFER_SIZE`
- At most `BUFFER_SIZE-1` items in the buffer at the same time.
- Empty: `in == out`
- Full: `((in+1)%BUFFER_SIZE) == out`



Shared-Memory Systems (4/4)

- The producer process:

```
item nextProduced;

while (true) {
     /* Produce an item */
     while (((in + 1) % BUFFER_SIZE)  == out)
               ;    // do nothing
     buffer[in] = nextProduced;
     in = (in + 1) % BUFFER SIZE;
}
```

- The consumer process

```
item nextConsumed

while (true) {
     while (in == out)
               ;    /* do nothing
     nextConsumed = buffer[out];
     out = (out + 1) % BUFFER SIZE;
}
```

→一把鎖，in≠out後才能通過

POSIX Shared Memory Example (1/7)

- POSIX shared memory is organized using **memory-mapped files**.
  - Associate the region of shared memory with a **file**.

- A process **create** a shared memory object using the `shm_open()` system call.
  - Example:

```
segment_fd = shm_open(name,
             O_CREAT | O_RDWR, 0666);
```

The name of shared-memory object

integer file descriptor for the shared-memory object

create if it does not yet exist

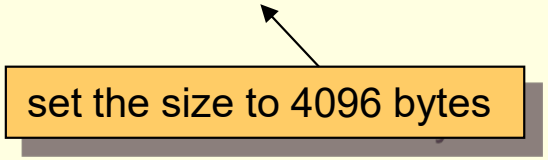
now, open for reading and writing

access permission

POSIX Shared Memory Example (2/7)

- Then, the `ftruncate()` is used to configure the size of the shared-memory object (in bytes).
  - Example:

```
ftruncate(shm_fd,4096);
```



POSIX Shared Memory Example (3/7)

- Finally, the function `mmap()` map the memory-mapped file into memory.
  - Example:

```
ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED,
                    shm_fd, 0);
```

be written, or PROT_READ be read

starting address of mapping, NULL(0), then kernel decides the mapping

offset

changes to the shared-memory object will be visible to all processes sharing the object.

POSIX Shared Memory Example (4/7)

- Finally, a process invokes `shm_unlink()` system call which removes the shared-memory segment.
  - Example:

```
shm_unlink(name);
```

The name of shared-memory object

POSIX Shared Memory Example (5/7)

producer.c

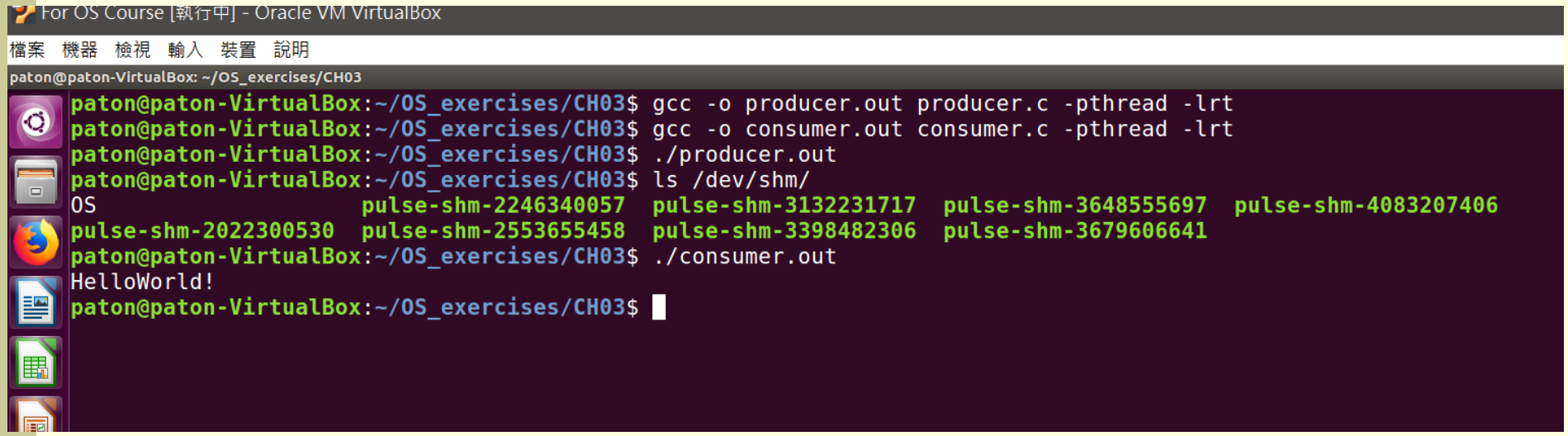
```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <sys/types.h>
#include <unistd.h>

int main()
{
   const int SIZE = 4096;
   const char *name = "OS";
   const char *message_0 = "Hello";
   const char *message_1 = "World!";

   int shm_fd;       // shared memory file descriptor
   void *ptr;        // pointer to shared memory object

   shm_fd = shm_open(name, O_CREAT |O_RDWR, 0666);

   ftruncate(shm_fd,SIZE);

   ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);

   sprintf(ptr, "%s", message_0);
   ptr += strlen(message_0);
   sprintf(ptr, "%s", message_1);
   ptr += strlen(message_1);

   return 0;
}
```

POSIX Shared Memory Example (6/7)

consumer.c

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>

int main()
{
   const int SIZE = 4096;
   const char *name = "OS";
   int shm_fd;
   void *ptr;

   shm_fd = shm_open(name, O_RDONLY, 0666);
   ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);

   printf("%s\n", (char *) ptr);
   shm_unlink(name);

   return 0;
}
```

POSIX Shared Memory Example (7/7)



Message-Passing Systems (1/7)

- Processes communicate with each other **without** resorting to **shared variables**.
- The communication actions are synchronized.
- Useful in a distributed (network) environment.

- A message-passing facility provides at least two operations:
  - ***send(message)***
  - ***receive(message)***
  - message size can be **fixed** or **variable**.
    - **fixed-sized messages** are easy to implement, but makes the programming task difficult.
    - **Variable-sized messages** require a more complex system-level implementation, but the programming task becomes simpler.

Message-Passing Systems (2/7)

- Processes that want to communicate must have a way to refer to each other.
- ***Direct-symmetry-communication***:
  - processes that want to communicate must explicitly name the **recipient** or **sender** of the communication.
  - `send(P, message)` – send a message to process `P`.
  - `receive(Q, message)` – receive a message from process `Q`.
- ***Direct-asymmetry-communication***:
  - Only the sender names the recipient.
  - `send(P, message)` – send a message to process `P`.
  - `receive(id, message)` – receive a message from **any** process; the variable `id` is set to the name of the sender.

Message-Passing Systems (3/7)

- The disadvantage in both of these ***hard-coding*** schemes (symmetric and asymmetric).
  - Changing the identifier of a process may necessitate examining all other processes.
  - All references to the old identifier must be modified to the new identifier.

- ***Indirect communication***:
  - The messages are sent to and received from ***mailboxes***, or ***ports***.
  - Each mailbox has a unique identification.
  - A process can communicate with some other process via **a number of** different mailboxes.
  - A mailbox may be owned either by a process of by the operating system.

  - `send(A, message)` – send a message to mailbox `A`.
  - `receive(A, message)` – receive a message from mailbox `A`.

Message-Passing Systems (4/7)

- In indirect communication scheme, a communication link has the following properties:
  - A link is established between a pair of processes only if both members of the pair have a shared mailbox.
  - A link may associated with more than two processes.
  - Between each pair of communicating processes, there may be a number of different links.

Message-Passing Systems (5/7)

- A practical problem of mailbox sharing:
  - P_1, P_2, and P_3 share mailbox A.
  - P_1, sends; P_2 and P_3 receive.
  - Who gets the message?

- Solutions
  - Allow a link to be associated with at most two processes.
  - Allow only one process at a time to execute a receive operation.
  - Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.

Message-Passing Systems (6/7)

- There are different design options for implementing the `send()` and `receive()` primitives.
  - **Blocking** (***synchronous***) **send**: the sending process is blocked until the message is received by the receiving process or by the mail box.
  - **Nonblocking** (***asynchronous***) **send**: the sending process sends the message and resumes operation.
  - **Blocking receive**: the receiver blocks until a message is available.
  - **Nonblocking receive**: the receiver retrieves either a valid message or a null.
  - Different combinations of `send()` and `receive()` are possible.
    - E.g., **rendezvous**: `send()` and `receive()` are blocking, a trivial solution to the producer-consumer problem.

Message-Passing Systems (7/7)

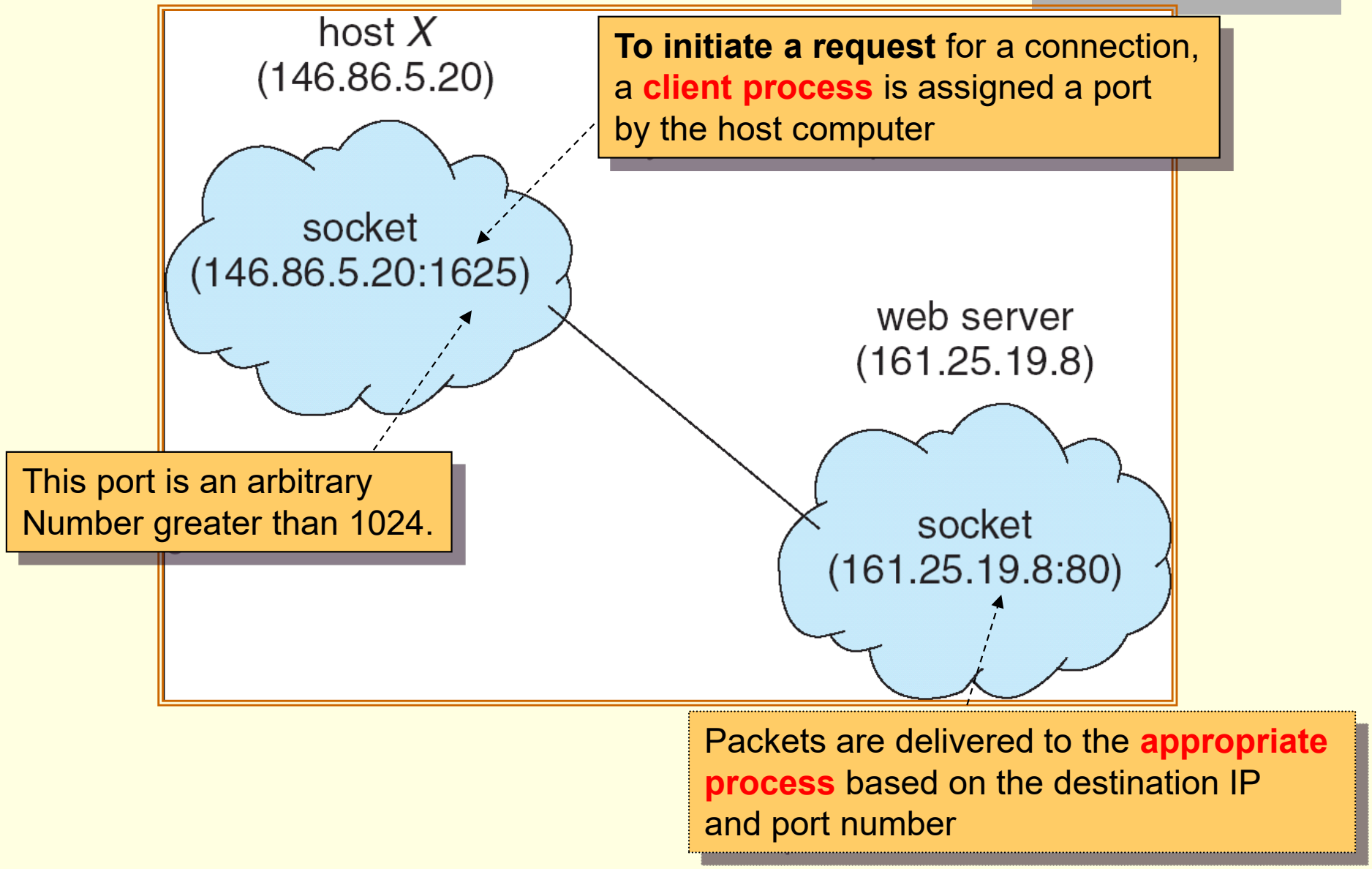
- ***Buffering***:
  - Whether communication is direct or indirect, messages exchanged by communicating processes reside in a temporary **queue**.
  - Queue can be:
    - **Zero capacity**: can not have any messages waiting in it.
      - The sender must block until the recipient receives the message.
    - **Bounded capacity**: the queue has finite length *n*.
      - If the queue is full, the sender must block until space is available in the queue.
    - **Unbounded capacity**: the sender never blocks.

Communication in Client-Server Systems

Sockets (1/2)

- A ***socket*** is defined as an ***endpoint*** *for communication.*
- A socket is identified by an ***IP address*** and a ***port number***.
  - The socket **161.25.19.8:1625** refers to port **1625** on host **161.25.19.8**.
- A pair of processes communicating over a network employ a pair of sockets – one for each process.
  - The server waits for incoming client requests by listening to a specified port.
  - Once a request is received, the server accepts (constructs) a connection from the client socket to complete the connection.
- Why port?
  - A way to communicate is to know the process ID of the hosts.
  - However, IDs change frequently.
  - Each port represents a network services.
  - All ports below 1024 are considered ***well known***.
    - http – 80, telnet – 23, ftp – 21 …

Sockets (2/2)


host X
(146.86.5.20)
To initiate a request for a connection, a client process is assigned a port by the host computer
socket
(146.86.5.20:1625)
web server
(161.25.19.8)
This port is an arbitrary Number greater than 1024.
socket
(161.25.19.8:80)
Packets are delivered to the appropriate process based on the destination IP and port number

Java Sockets Example (1/4)

- Java provides three different types of sockets:
  - ***Connection-oriented*** (TCP) sockets – implemented with the `Socket` class.
  - ***Connectionless*** (UDP) sockets – implemented with the `DatagramSocket` class.
  - ***Multicast sockets*** – implemented with `MulticastSocket` class.

- Example Server:
  - Use connection-oriented TCP sockets.
  - Listen to port 6013.
  - When a client request is received, the server returns the data and time to the client.

Java Sockets Example (2/4)

DateServer.java

```
import java.net.*;
import java.io.*;

public class DateServer
{
  public static void main(String[] args)   {
   try {
     ServerSocket sock = new ServerSocket(6013);

     // now listen for connections
     while (true) {
         Socket client = sock.accept();
         // we have a connection

         PrintWriter pout = new PrintWriter(client.getOutputStream(), true);
         // write the Date to the socket
         pout.println(new java.util.Date().toString());

         // close the socket and resume listening for more connections
         client.close();
     }
   } catch (IOException ioe) {
      System.err.println(ioe);
   }
  }
}
```

Creates a server socket object, bound to the specified port.

Listens for (and **block till**) a connection to be made to this socket and accepts it.

Return a socket that the server can use to communicate with the client.

The server closes the socket to the client and resumes listening for more requests.

http://java.sun.com/j2se/1.4.2/docs/api/java/net/ServerSocket.html

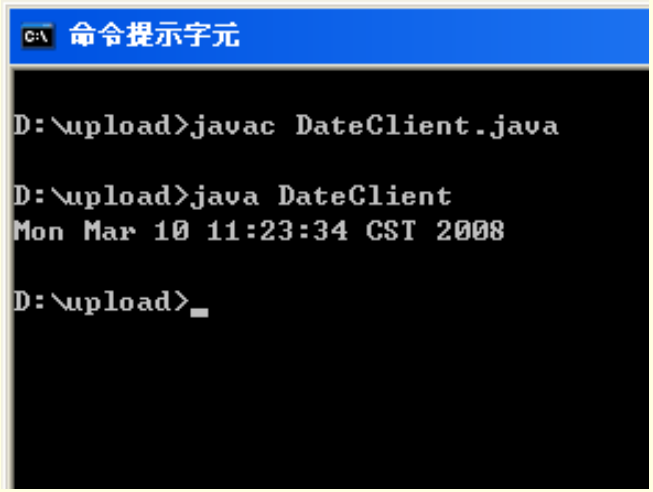
Java Sockets Example (3/4)

DateClient.java

```
import java.net.*;
import java.io.*;

public class DateClient
{
   public static void main(String[] args)  {
     try {
         // this could be changed to an IP name
         // or address other than the localhost
         Socket sock = new Socket("127.0.0.1",6013);
         InputStream in = sock.getInputStream();
         BufferedReader bin = new BufferedReader(new
                                          InputStreamReader(in));

         String line;
         while( (line = bin.readLine()) != null)
            System.out.println(line);

         sock.close();
     } catch (IOException ioe) {
         System.err.println(ioe);
     }
   }
}
```

Create a socket and request a connection with the server on port 6013

Java Sockets Example (4/4)



- The IP address 127.0.0.1 is a special IP address known as the ***loopback***.
  - **it is referring to itself!!**
  - The example runs the client and server on the same host to communicate using the TCP/IP protocol.

Pipes (1/6)

- Pipes were one of the first IPC mechanisms in early UNIX systems.
- Two common types of pipes used on both UNIX and Windows systems:
  - **Ordinary pipes** and **named pipes**.

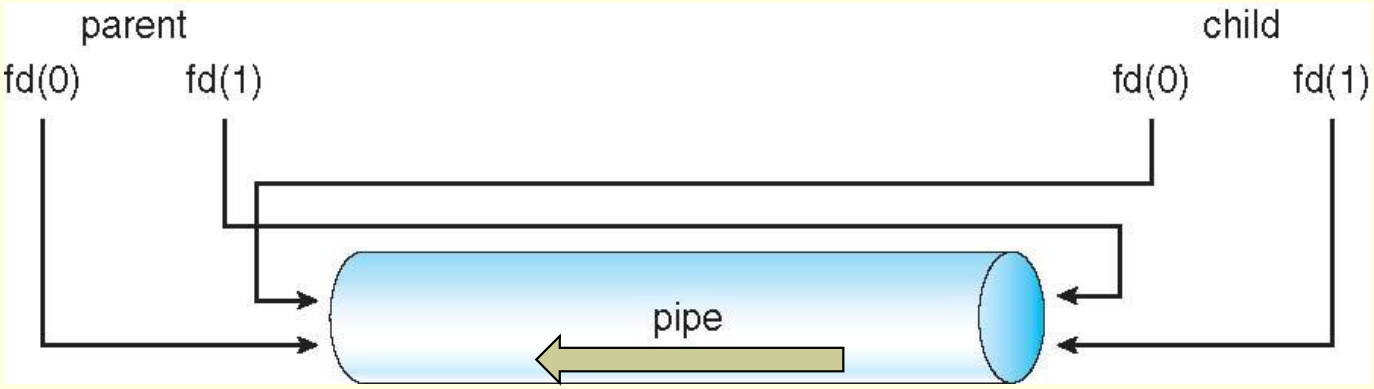
Pipes (2/6)

- Ordinary pipes:
  - Are **unidirectional**.
  - Allow two processes to communicate in standard producer-consumer fashion.
    - The producer writes to one end of the pipe (the **write-end**).
    - The consumer reads from the other end (the **read-end**).
  - On UNIX systems, ordinary pipes are constructed using the **`pipe()`** system call (POSIX).

    `pipe(int fd[])`
  - The function creates a pipe that is accessed through the `int fd[]` file descriptor.
    - `fd[0]` is the read end.
    - `fd[1]` is the write end.
    - UNIX treats a pipe as a file, so pipes can be accessed using `read()` and `write()` system calls.

Pipes (3/6)

- Ordinary pipes:
  - An ordinary pipe **cannot** be accessed from outside the process that created it!!
  - Generally, a parent creates a pipe and uses it to communicate with a child that it creates via `fork()`.
  - A forked child inherits open files from its parent.
    - Pipe is a special type of file, so the child can access the pipe too.

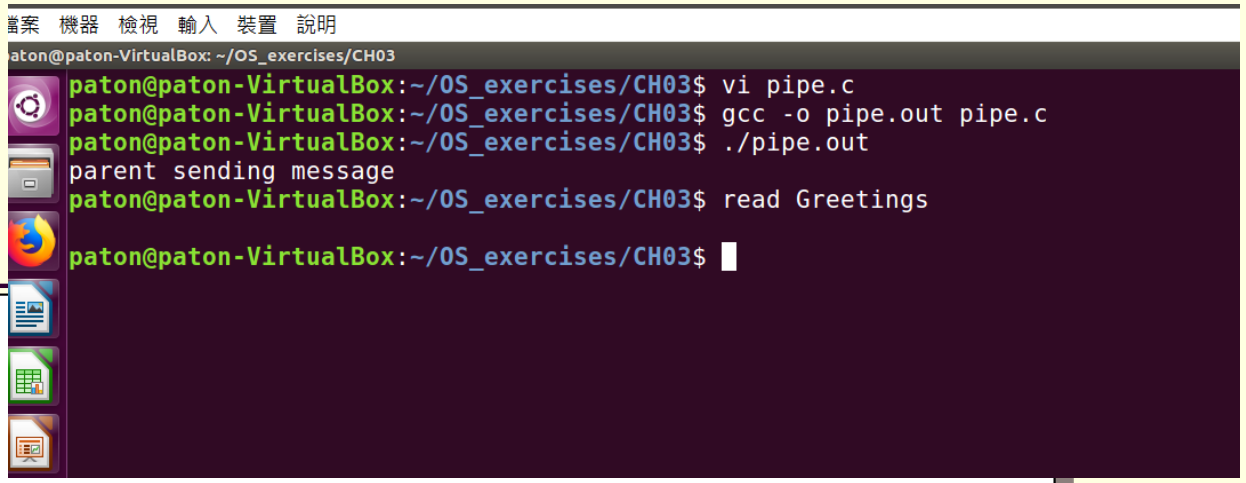


unidirectional communication

Pipes (4/6)

```
#include <sys/types.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

#define BUFFER_SIZE 25
#define READ_END 0
#define WRITE_END 1

int main(void)
{
   char write_msg[BUFFER_SIZE] = "Greetings";
   char read_msg[BUFFER_SIZE];
   int fd[2];
   pid_t pid;

   // create a pipe
   if(pipe(fd) == -1) {
      fprintf(stderr, "Pipe failed");
      return 1;
   }
```

Pipes (5/6)




```
    pid = fork ();

    if (pid < 0) {
        fprintf(stderr, "Fork failed");
        return 1;
    }


    if(pid > 0) {  // parent process
        close(fd[READ_END]);
        printf("parent sending message\n");
        write(fd[WRITE_END], write_msg, strlen(write_msg)+1);
        close(fd[WRITE_END]);

    } else {  // child process
        close(fd[WRITE_END]);
        read(fd[READ_END], read_msg, BUFFER_SIZE);
        printf("read %s\n", read_msg);
        close(fd[READ_END]);
    }

    return 0;
}
```

It is better to close unused ends of the pipe

Pipes (6/6)

- Ordinary pipes require a parent-child relationship between the communicating processes (on both UNIX and Windows).
  - What if we want communication with no parent-child relation?

- Both UNIX and Windows systems support **named pipes**.
  - Referred to as FIFOs in UNIX systems.
  - Once created, they appear as a files in the file systems.
    - Continue to exist until it is explicitly deleted from the file system.
  - You can create named pipes using `mkfifo()` system call (POSIX).

End of Chapter 3

Homework 1:

Chapter 4: Multithreaded Programming

Chien Chin Chen

Department of Information Management
National Taiwan University

Outline

- Overview.
- Multithreading Models.
- Thread Libraries.
- Threading Issues.

Overview (1/6)

- ***Single-threaded*** process – ***multithreaded*** process:
  - A thread is a basic unit of CPU utilization.
  - Traditional process has a single thread of control.
  - Multithreaded process can perform **more than one task at a time**.
- Many applications are multithreaded.
  - A web browser might have one thread display images while another thread retrieves data from the network.
  - A word processor may have a thread for responding to keystrokes from the users, and another thread for performing spelling and grammar checking in the background.

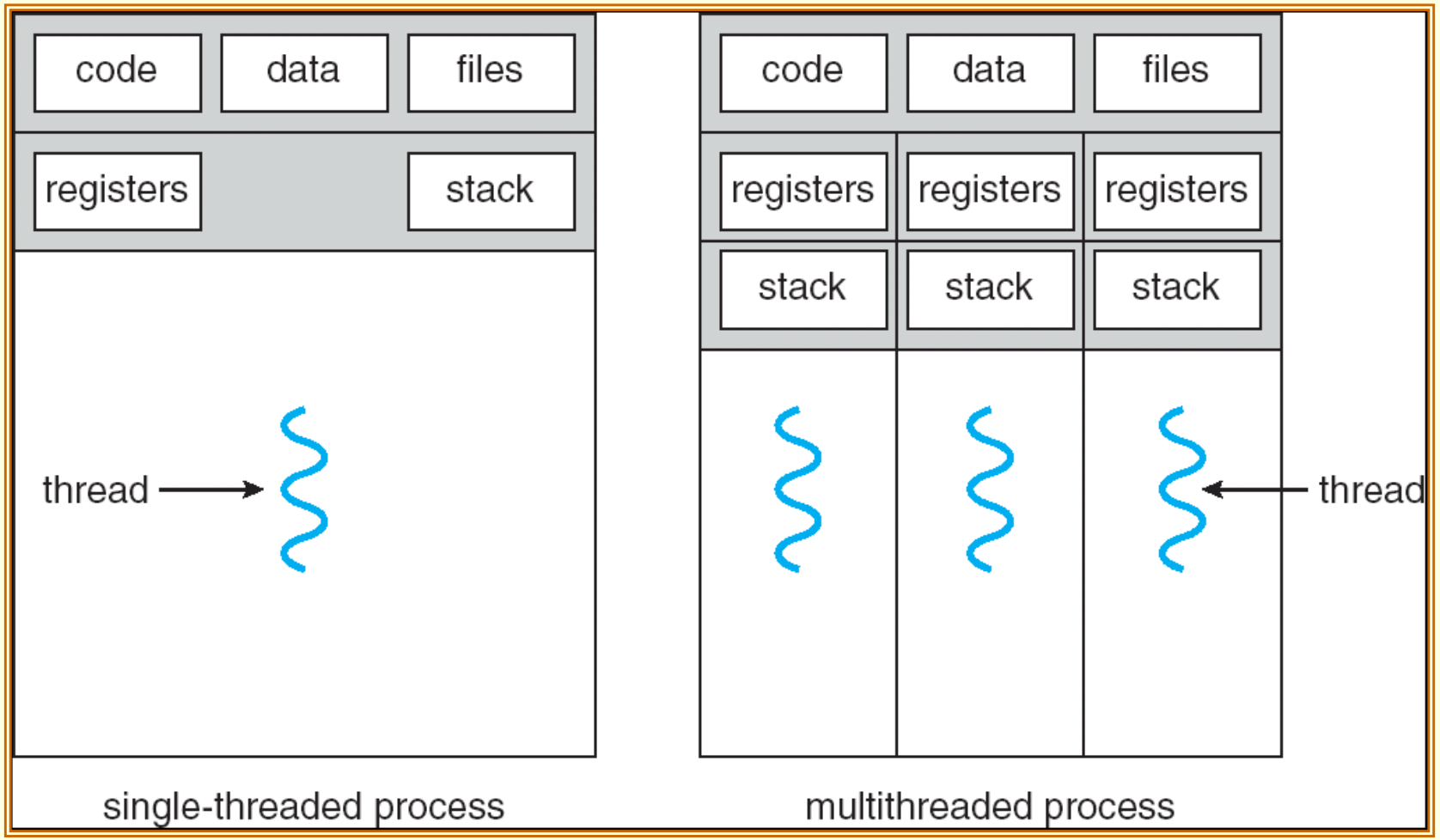
Overview (2/6)

- In many situations, an application may be required to perform several similar tasks.
  - A web server accepts several client requests for web pages.
  - If the web server ran as a traditional single-threaded process … it would be able to service only one client at a time.
- Solution 1:
  - When the server receives a request, it **creates a separate process** to service that request.
    - E.g., `fork()`.
  - This process-creation method was in common use before threads became popular.
  - **Process creation is time consuming and resource intensive**.
- Solution 2:
  - It is more efficient to use one process that contains multiple threads.
  - When a request was made, the server (a thread) would **create another thread to service the request**.

Overview (3/6)

- A thread comprises:
  - A **thread ID**.
  - A **program counter**.
  - A **register set**.
  - A **stack**.
- It shares with other threads belonging to the same process:
  - Code section.
  - Data section.
  - Other operating-system resources, such as open files.

Overview (4/6)


code
data
files
registers
stack
thread
single-threaded process
code
data
files
registers
registers
registers
stack
stack
stack
thread
multithreaded process

Overview (5/6)

- **Benefits**:
  - **Responsiveness**:
    - Application can continue running even if part of it is blocked or is performing a lengthy operation.
    - A multithreaded web browser allow user interaction in one thread while an image was being loaded in another thread.
  - **Resource sharing**:
    - Threads share the memory and the resources of the process to which they belong.
  - **Economy**:
    - Allocating memory and resources for process creation is costly.
    - In Solaris, creating a process is about thirty times slower than is creating a thread. Context switching is about five times slower.
  - **Utilization of multiprocessor architectures**:
    - Threads may be running in parallel on different processors.
    - A single-threaded process can only run on one CPU, no matter how many are available.

Overview (6/6)

- Many operating system kernels are now multithreaded.
  - Several threads operate in the kernel.
  - Each thread performs a specific task.
  - For example, Linux uses a kernel thread for managing the amount of free memory in the system.

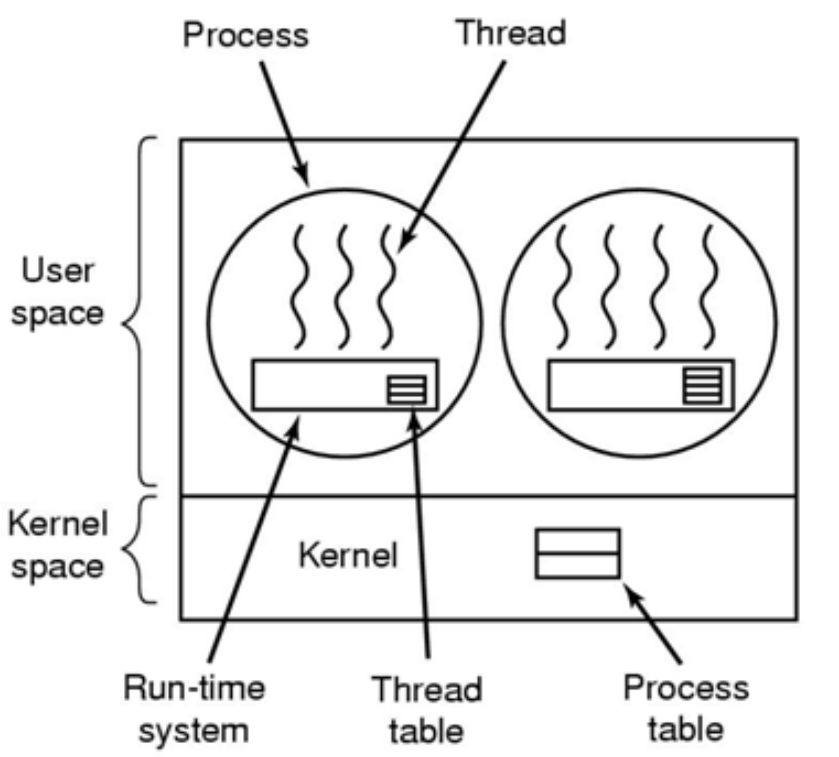
Multithreading Models (1/8)

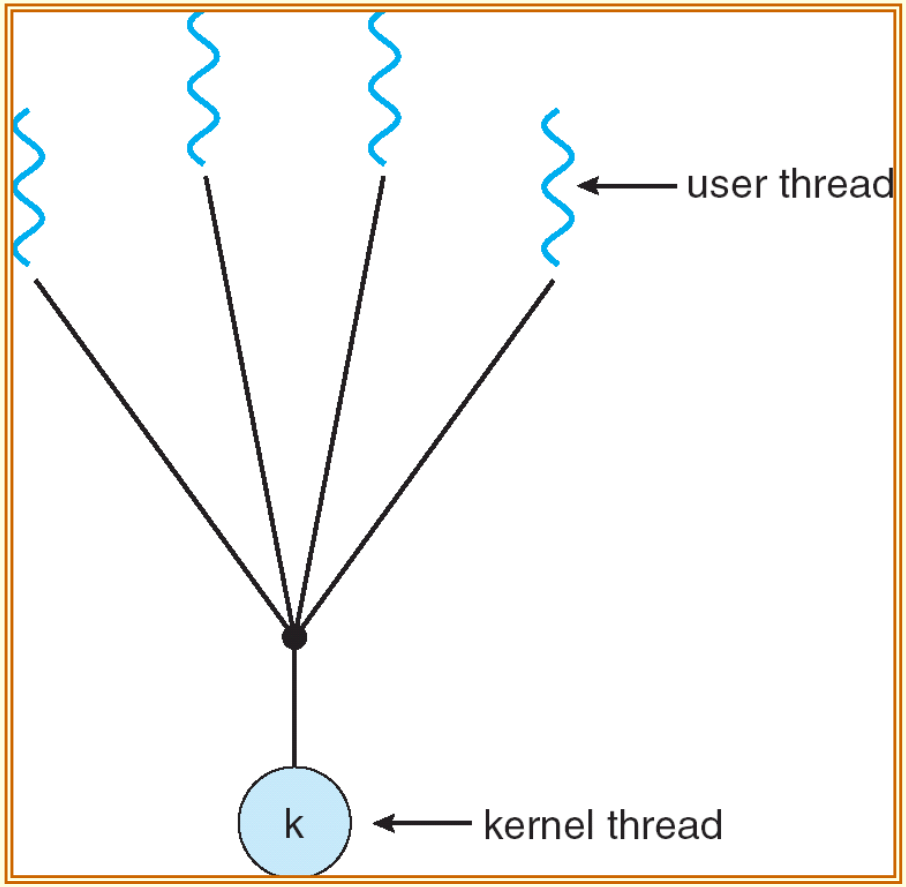
- **User threads** – **kernel threads**:
  - ***User threads*** are supported above the kernel and are managed without kernel support.
  - ***Kernel threads*** are supported and managed directly by the operating system.
  - All contemporary operating systems support kernel threads.
  - There must exist a relationship between user threads and kernel threads.

Multithreading Models (2/8)

- ***Many-to-One model***:
  - Map many user-level threads to one kernel thread.
  - Thread management is done by the thread library **in user space**.
    - The library provides the supports of thread creation, scheduling …
  - **Is efficient**, because there is no kernel intervention.
  - But … as the kernel is not aware of user threads …
  - The entire process will block if a thread makes a blocking system call.
  - Multiple threads are unable to run in parallel on multiprocessors.

Multithreading Models (3/8)





Many-to-One model

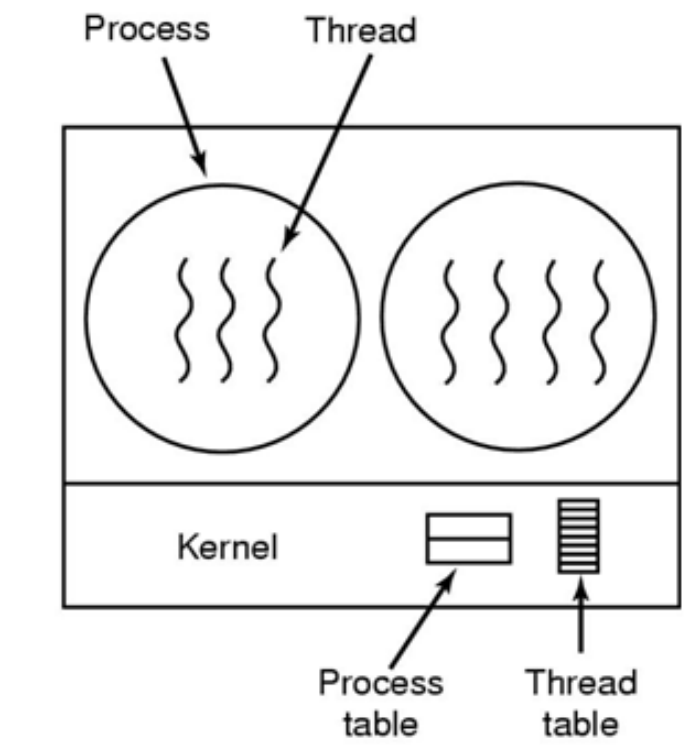
Multithreading Models (4/8)

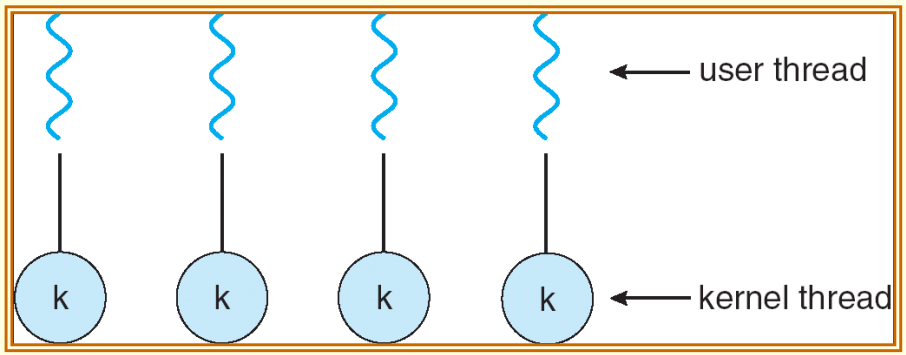
- ***One-to-One model***:
  - Map each user thread to a kernel thread.
  - Allow another thread to run when a thread makes a blocking system call.
  - Allow multiple threads to run in parallel on multiprocessors.

  - But … creating a user thread requires creating the corresponding kernel thread.
  - The creation overhead can burden the performance of an application.
  - Most implementations of this model restrict the number of threads supported by the system.

  - Supported by Linux and Windows family.

Multithreading Models (5/8)



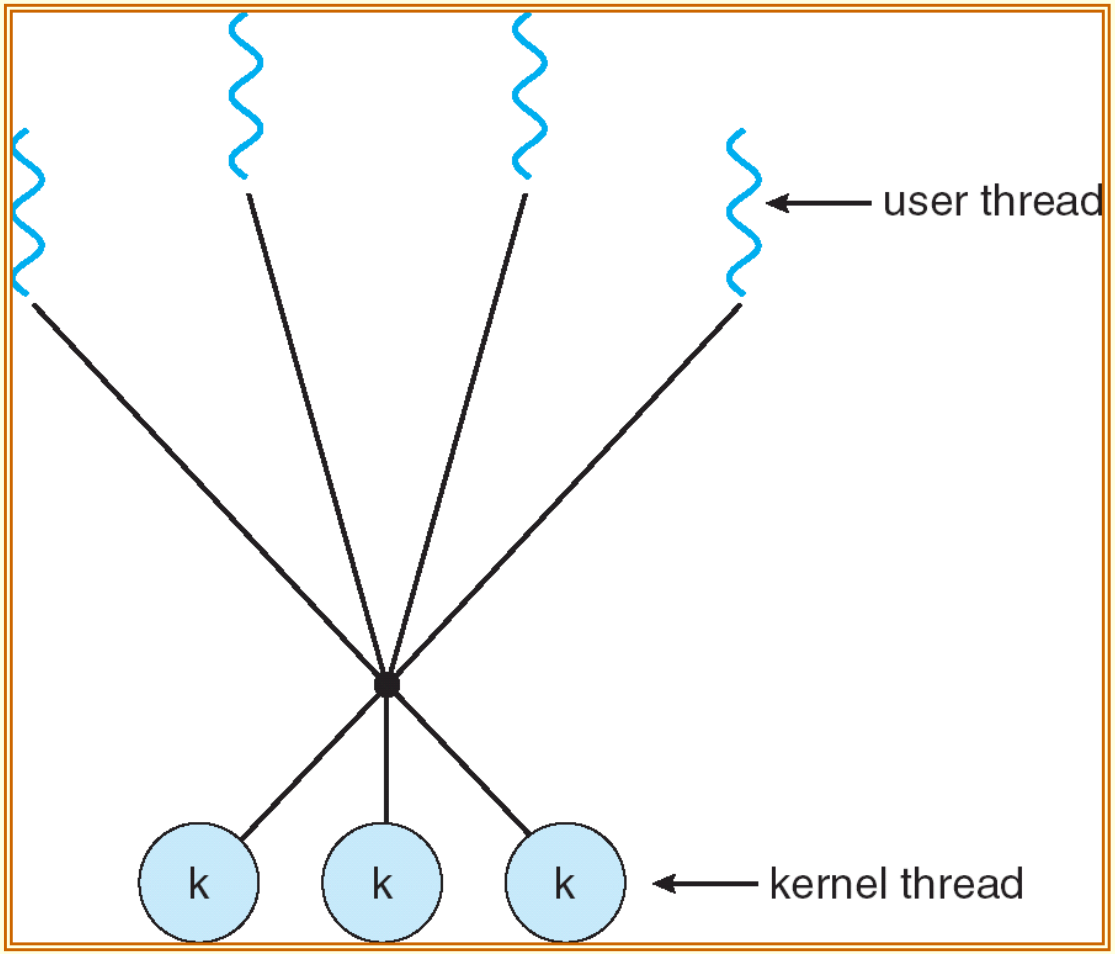


One-to-One model

Multithreading Models (6/8)

- ***Many-to-Many model***:
  - Multiplex many user-level threads to a smaller or equal number of kernel threads.
  - The number of kernel threads may be specific to either a particular application or a particular machine.
  - The many-to-many model **does not** suffer from the shortcomings of the previous two models.
    - Many-to-One: the kernel can schedule only one thread at a time (on a multiprocessor system).
    - One-to-One: the number of threads is limited.
    - Many-to-many model [1.] allows as many user threads as necessary, and [2.] the corresponding kernel threads can run in parallel on a multiprocessor. [3.] When a thread is blocked, the kernel can schedule another thread for execution.

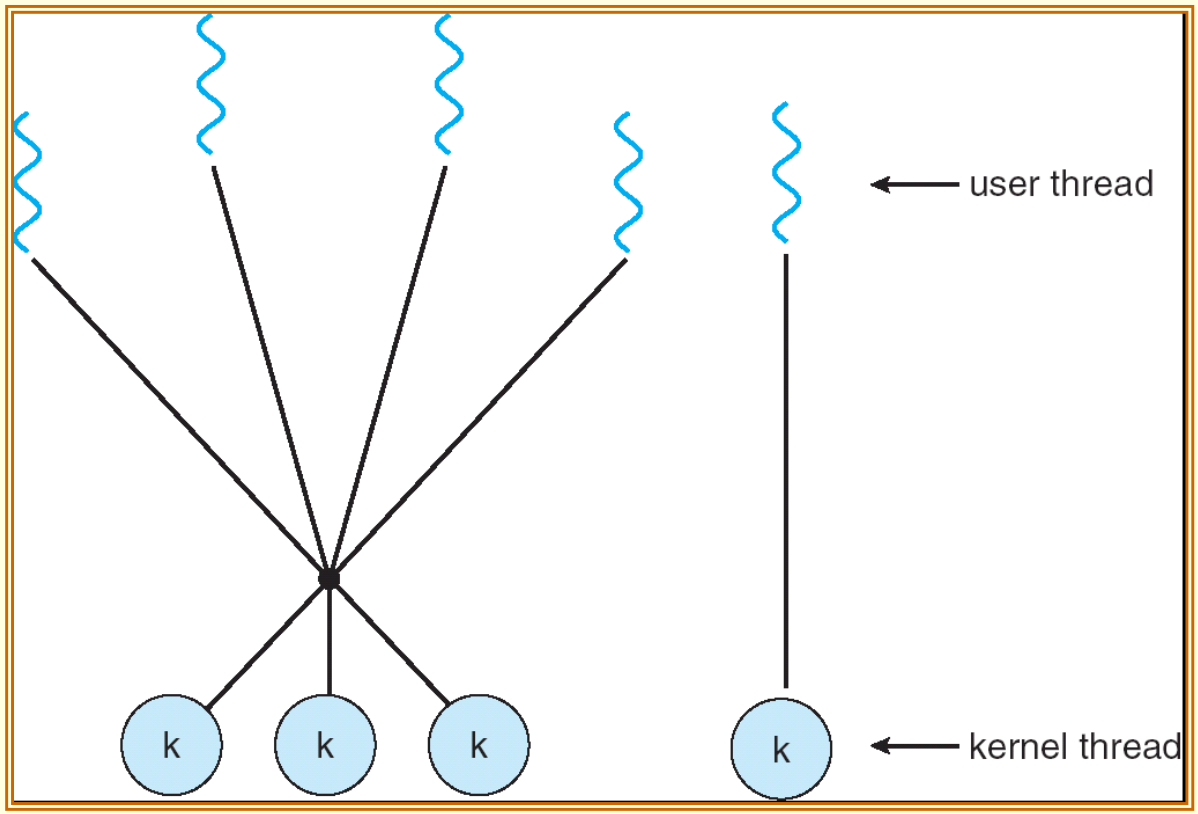
Multithreading Models (7/8)



Many-to-Many model

Multithreading Models (8/8)

- ***Two-Level model*** (hybrid thread model):
  - Similar to many-to-many model.
  - But allow a user-level thread to be bound to a kernel thread.



Two-Level model

Thread Libraries (1/8)

- A thread library provides the programmer an **API** for creating and managing threads.
- Two ways of implementing a thread library:
  - To provide a library entirely in **user space** with no kernel support.
    - All code and data structures for the library exist in user space.
    - All function calls result in local function calls in user space; and no system calls.
  - To implement a **kernel-level** library:
    - Supported by the operating system.
    - Code and data structures exist in kernel space.
    - A function call in the API for the library results in a system call to the kernel.

Thread Libraries (2/8)

- Three primary thread libraries: (很多，我這三種)
  - ***POSIX Pthreads***. (UNISPACE)
    - May be provided as either a user- or kernel-level library.
  - ***Windows threads***.
    - Kernel-level library, available on Windows systems.
  - ***Java threads***.
    - JVM is running on top of a host operating system, the implementation depends on the host system.
    - On Windows systems, Java threads are implemented using the Windows API;
    - UNIX-based systems often use Pthreads.

- The following multithreaded examples perform the summation of a non-negative integer in a separate thread.

$$sum = \sum_{i=0}^{N} i$$

User specified

Thread Libraries – **Pthreads** (3/8)

- Pthreads refers to the POSIX standard, defining an API for thread creation and synchronization.
  - **Is a specification** for thread behavior, **not an implementation**.
  - Operating system designers implement the specification in any way they wish.
  - When a program begins, a single thread of control begins in `main()`.
  - `main()` then creates a second thread.
  - In a Pthreads program, separate threads begin execution in a specified function – in this example, the `runner()` function.

Thread Libraries – Pthreads (4/8)

datatype 不能改

```
void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;

    if (upper > 0) {
        for (i = 1; i <= upper; i++)
            sum += i;
    }

    pthread_exit(0);
}
```

A global variable shared with main()

Thread termination

= return 0

```c
#include <pthread.h>
#include <stdio.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* the thread */

int main(int argc, char *argv[])
{
   pthread_t tid;       /* the thread identifier */
   pthread_attr_t attr; /* set of attributes for the thread */

   if (argc != 2) {
      fprintf(stderr,"usage: a.out <integer value>\n");
      return -1;
   }

   if (atoi(argv[1]) < 0) {
      fprintf(stderr,"Argument %d must be non-negative\n",atoi(argv[1]));
      return -1;
   }

   /* get the default attributes */
   pthread_attr_init(&attr);

   /* create the thread */
   pthread_create(&tid,&attr,runner,argv[1]);

   /* now wait for the thread to exit */
   pthread_join(tid,NULL);
   printf("sum = %d\n",sum);
}
```

All Pthreads programs must include this header file.

thrd-posix.c

Use default thread attributes.

Create a separate thread

function

The parent thread waits for child to complete.

Thread Libraries – **Windows** (6/8)

- A Win32 multithreaded operation will usually be embedded inside a function which returns a `DWORD` and takes a `LPVOID` as a parameter.
  - The `DWORD` data type is an unsigned 32-bit integer.
  - `LPVOID` is a pointer to a `void`.
- Data shared by the separate threads are declared globally.

Thread Libraries – Windows (7/8)

```
/* the thread runs in this separate function */
DWORD WINAPI Summation(LPVOID Param)
{
        DWORD Upper = *(DWORD *)Param;

        for (DWORD i = 0; i <= Upper; i++)
                Sum += i;

        return 0;
}
```

Is identical to `void *`

int pointer 依值取值

Unsigned 32-bit integer

A global variable, shared by the thread(s)

void pointer : a point that can point to any data

```
#include <stdio.h>
#include <windows.h>

DWORD Sum; /* data is shared by the thread(s) */

int main(int argc, char *argv[])
{
        DWORD ThreadId;
        HANDLE ThreadHandle;
        int Param;

        ... // do some basic error checking
        Param = atoi(argv[1]);
        if (Param < 0) {
                fprintf(stderr, "an integer >= 0 is required \n");
                return -1;
        }

        // create the thread
        ThreadHandle = CreateThread(NULL, 0, Summation, &Param, 0, &ThreadId);

        if (ThreadHandle != NULL) {
                WaitForSingleObject(ThreadHandle, INFINITE);
                CloseHandle(ThreadHandle);
                printf("sum = %d\n",Sum);
        }
}
```

Must include this header file

A windows system resources, such as file and thread, are represented as ***kernel object***.
Objects are accessed in Windows programs by ***handles***

Default security attributes

Default stack size

Creation flags, default values make it eligible to be run by the CPU scheduler

Thread function

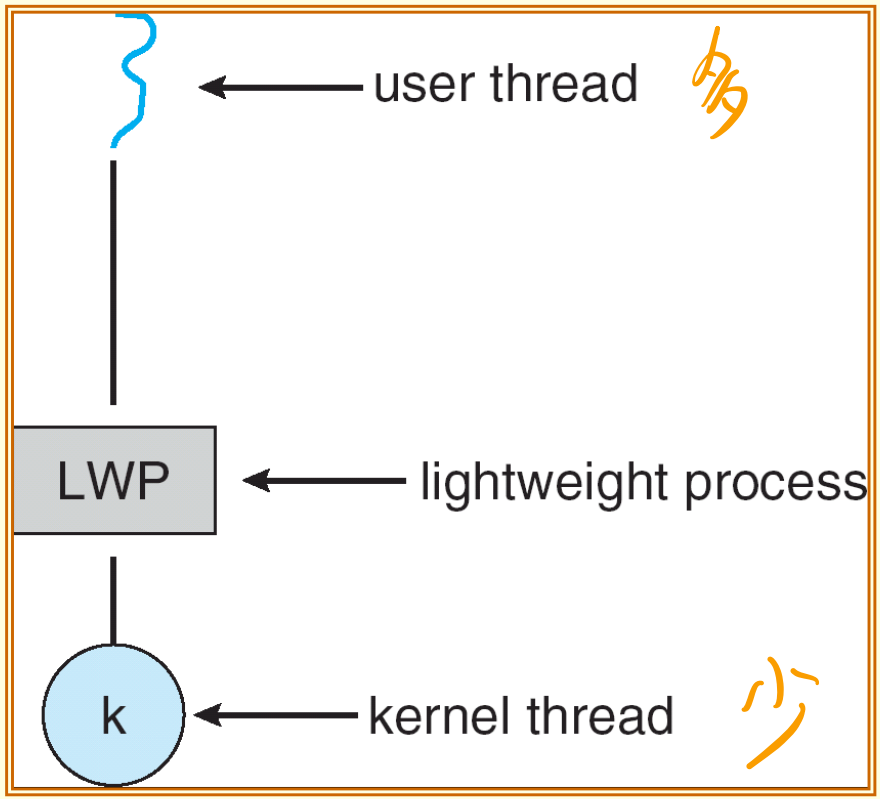
Parameter to thread function

Threading Issues – Scheduler

Activations (1/5)

每個 kernel 都有個 LWP，對到一個 user thread 就能執行

- Many systems implementing the *many-to-many model* place an **intermediate data structure** between the user and kernel threads.
  - ***Lightweight process*** (or LWP, virtual processor).
- The kernel provides an application with a set of virtual processors.
- Each LWP is attached to a kernel thread, and is scheduled by the operating system to run on physical processors.
- To the user-thread library, a LWP can be used to schedule a user thread (to run).

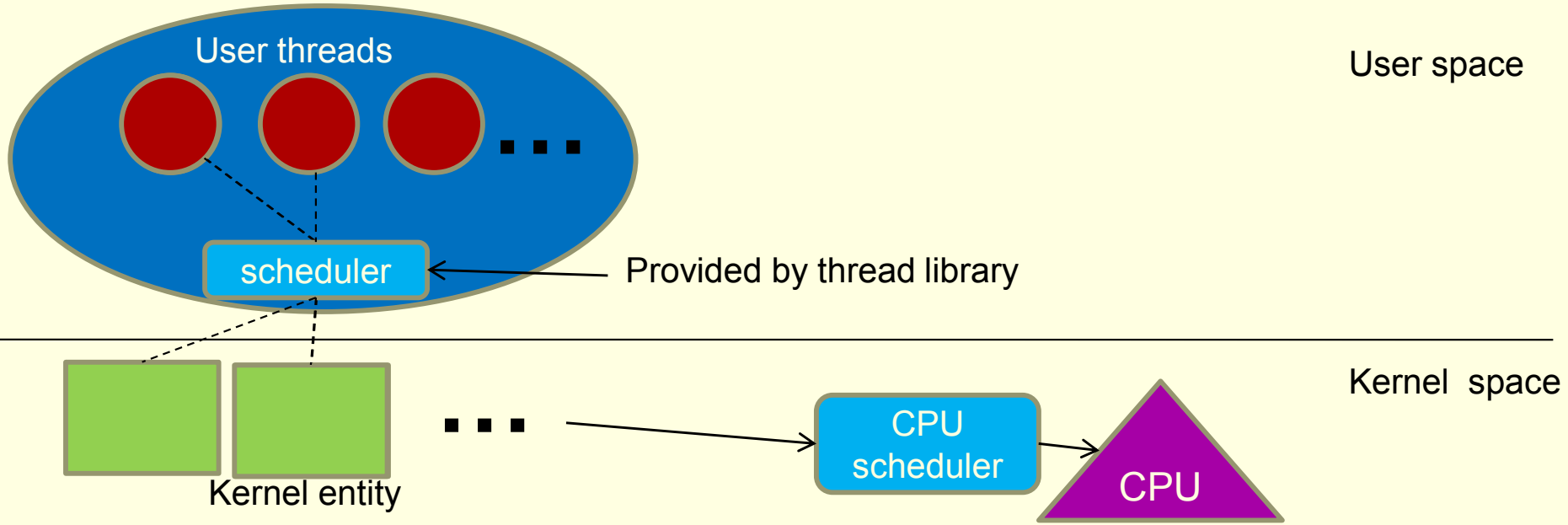


① scheduling：哪些 user thread 能排到對應的 kernel entity

Threading Issues – Scheduler Activations (2/5)

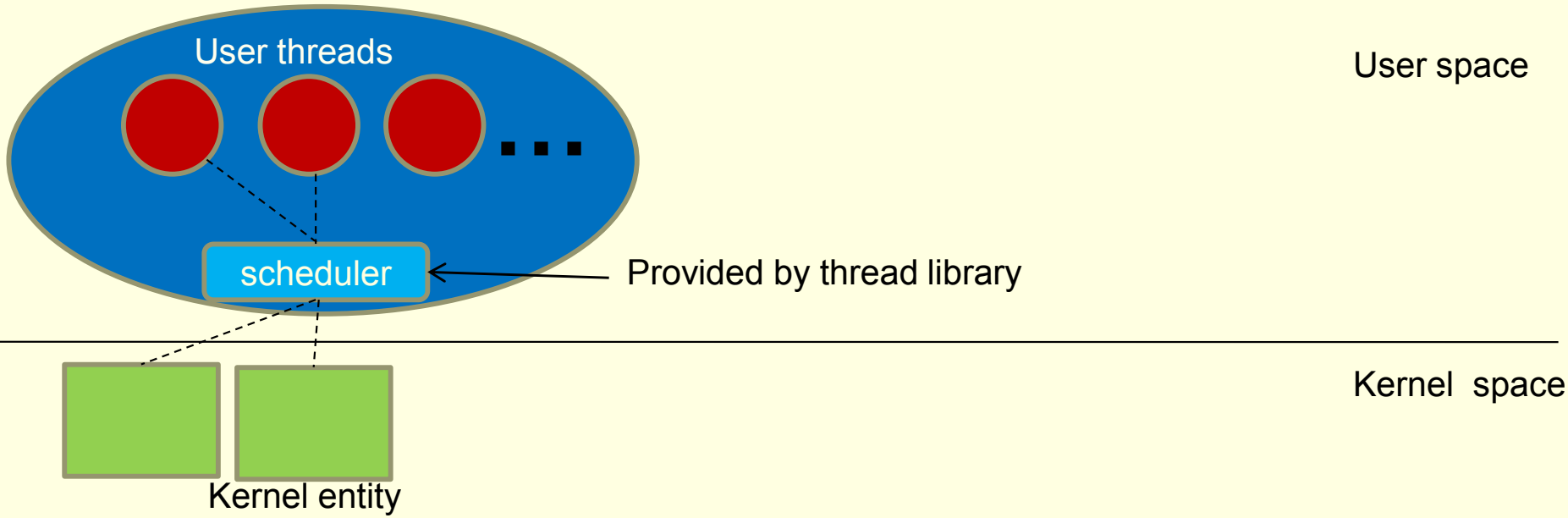
- A Problem of many-to-many model.
  - A thread will block a kernel entity if it makes a blocking system call.

entity 被 block 後可用的愈來愈少



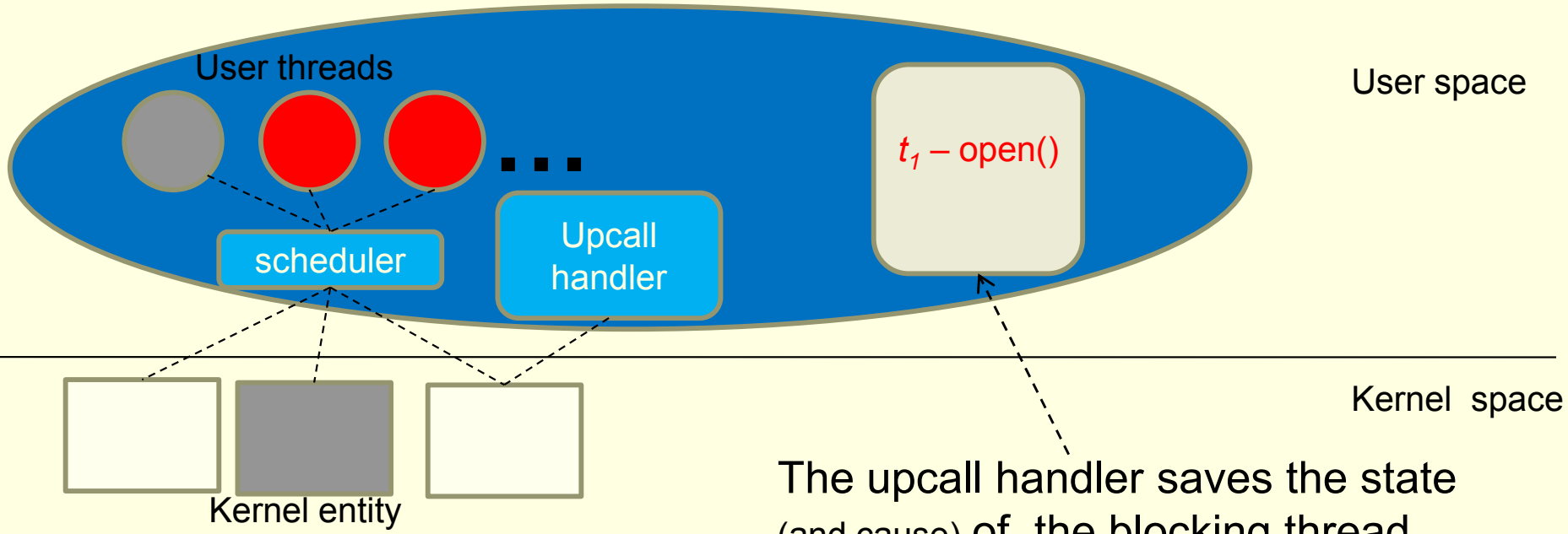
Threading Issues – Scheduler Activations (3/5)

- ***Scheduler activation*** – an efficient scheme for running many-to-many model.
  - When an user thread is about to block … kernel **activates** the **scheduler** supplied by thread library to schedule another user thread.



Threading Issues – Scheduler Activations (4/5)

- How ???
- The kernel makes an **upcall** to **inform** an application about certain events.
  - Upcalls are handled by the thread library with an ***upcall handler***.



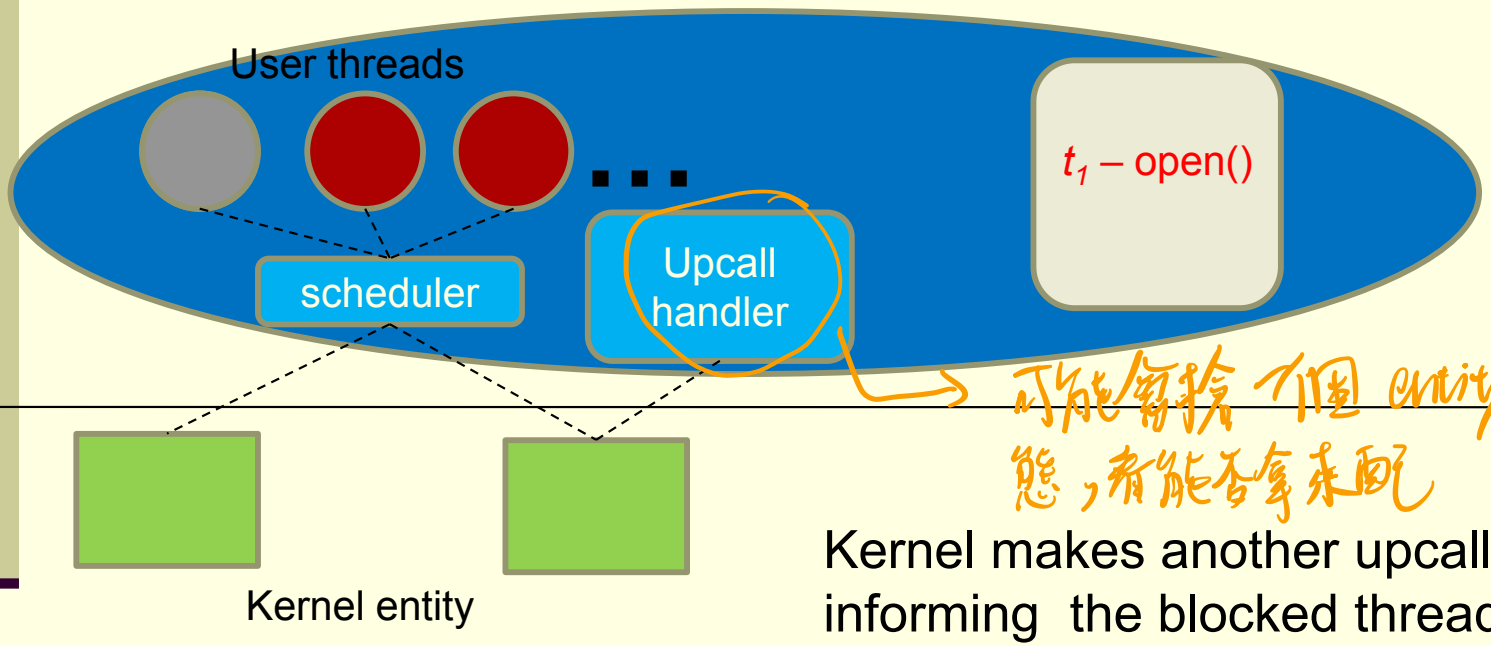
Kernel first allocates a new virtual process (or kernel entity) for executing the upcall handler.

The upcall handler saves the state (and cause) of the blocking thread.

Then it release the blocking kernel entity and **activate** the **scheduler** to run a ready thread

Threading Issues – Scheduler Activations (5/5)

- When the waiting event completes …



Kernel makes another upcall, informing the blocked thread is ready

It may preempt one users thread (or allocate a new virtual processor) to run the upcall handler.

Then **active** the **scheduler** again to schedule a ready thread.

End of Chapter 4

Homework 2:
Exercise:
Due date:

Reference: **Programming with POSIX(R) Threads (Addison-Wesley Professional Computing Series)**
by David R. Butenhof… **400 pages**

Chapter 5: Process Scheduling

Chien Chin Chen

Department of Information Management
National Taiwan University

Outline

- Basic Concepts.
- Scheduling Criteria.
- Scheduling Algorithms.
- Multiple-Processor Scheduling.
- Thread Scheduling.
- Operating Systems Examples.
- Algorithm Evaluation.

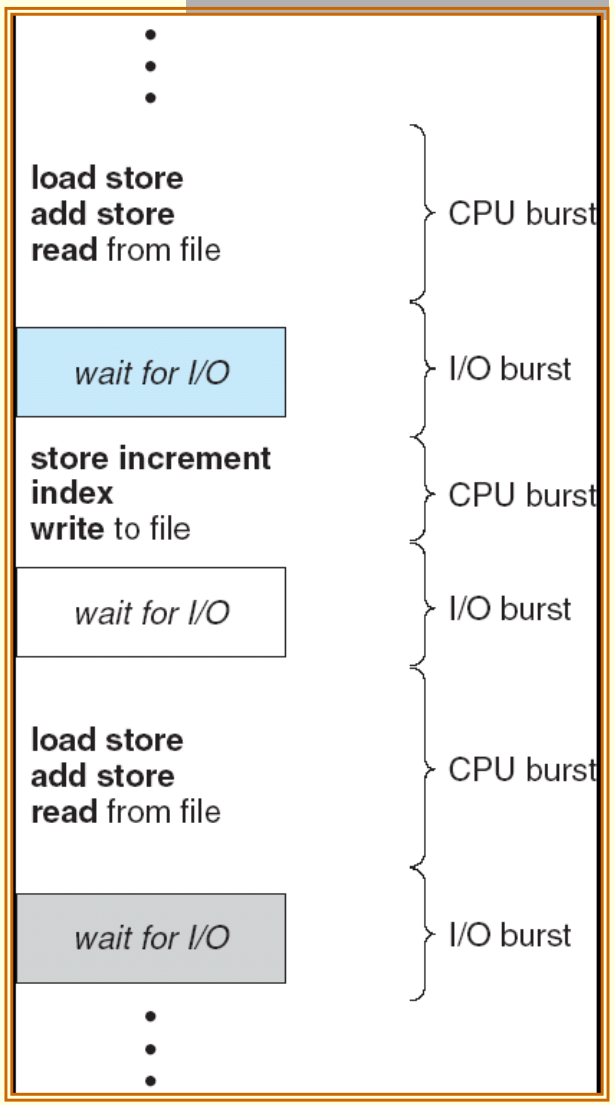
Basic Concepts (1/3)

- In a single-processor, only one process can run at a time; any others must wait until the CPU is free and can be rescheduled.
- ***Multiprogramming*** is to have some process running at all times, to maximize CPU utilization.
  - Several processes are kept in memory at one time.
  - A process is executed until it must wait (for example I/O).
  - When one process has to wait, the operating system **takes** the CPU **away** from that process and **gives** the CPU **to another** process.
- ***Scheduling*** of this kind is a fundamental operating-system function.
  - Almost all computer resources are scheduled before use.
  - Here, we talk about **CPU scheduling** as it is central to operating-system design.

☆ 只要有競爭資源的地方，就需要 scheduling

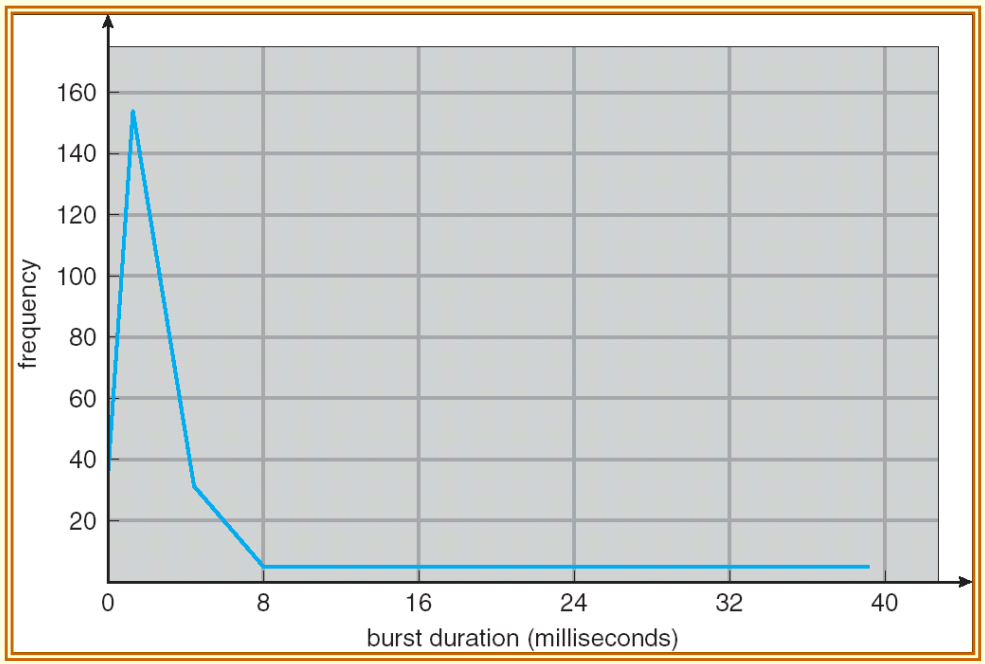
Basic Concepts (2/3)

- Process execution consists of a **cycle** of **CPU execution** and **I/O wait**.
  - This is, processes alternate between these two states.
- Process execution generally begins with a CPU burst, followed by an I/O burst.
  - Which is followed by another CPU burst, and so on.
  - Eventually, the final CPU burst ends with a request to terminate execution.



Basic Concepts (3/3)

- To have a successful CPU scheduler, we have to know properties of process execution.
- The figure shows the durations of CPU bursts.
  - Processes generally have a large number of short CPU bursts and small number of long CPU bursts.



- An I/O-bound program typically has many short CPU bursts.
- A CPU-bound program might have a few long CPU bursts.

CPU Scheduler (1/4)

- ***CPU scheduler*** (short-term scheduler) select one of the processes in the ready queue to be executed, whenever the CPU becomes idle.
- The ready queue is **not necessarily** a *first-in, first-out* (FIFO) queue.
  - With various scheduling algorithms, a ready queue can be implemented as a FIFO queue, a priority queue, a tree, or simply an unordered linked list.
- However, all the processes in the ready queue are waiting for a chance to run on the CPU.
  - The records in the queues are generally *process control blocks* (PCBs) of the processes.

CPU Scheduler (2/4)

- CPU scheduling decisions may take place when a process:
  1. Switches from running to waiting state (I/O or wait for child processes).
  2. Switches from running to ready state (interrupted).
  3. Switches from waiting to ready (completion of I/O).
  4. Terminates.
- When scheduling takes place only under circumstances 1 and 4, we say that the scheduling scheme is ***nonpreemptive***. (不會搶人)
  - Otherwise, it is is ***preemptive***.

↳ 較有效率

CPU Scheduler (3/4)

- Under nonpreemptive scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or by switching to the waiting state.
  - Windows 3.x.
  - Windows 95 and all subsequent versions of Windows have used preemptive scheduling.
- Preemptive scheduling usually has better scheduling performances. However, **it incurs data inconsistency**.
  - Consider the case of two processes that share data.
  - While one is updating the data, it is preempted so that the second process can run.
  - The second process then tries to read the data, which are in an inconsistent state.
  - Chapter 6 discuss mechanism of coordination.

CPU Scheduler (4/4)

- During the processing of a system call, the operating-system kernel may be busy with an activity on behalf of a process.
  - The activity may involve changing important kernel data (e.g., I/O queues).
- If the process is preempted in the middle of these changes and the kernel (or the device driver) need to read or modify the same structure, chaos ensues.
- Therefore, certain operating systems (UNIX) deal with this problem by waiting for system call to complete before doing a context switch.

Dispatcher

- ***Dispatcher***: a module gives control of the CPU to the process selected by the short-term scheduler; this involves:
  - switching context.
  - switching to user mode.
  - jumping to the proper location in the user program to restart that program.
- The dispatcher should be *as fast as possible*, since it is invoked during every process switch.
- ***Dispatch latency*** – time it takes for the dispatcher to stop one process and start another running

Scheduling Criteria (1/4)

- Different CPU scheduling algorithms have different properties, and favor one class of processes over another.
- Many criteria have been suggested for comparing CPU scheduling algorithms.
  - Different criteria are used for comparison can make a substantial difference in which algorithm is judged to be best.
- ***CPU utilization***:
  - The utilization of CPU.
  - Can range from 0 to 100 percent.
  - It should range from 40 percent (lightly loaded system) to 90 percent (heavily used system).

Scheduling Criteria (2/4)

- ***Throughput***:
  - The number of processes that complete their execution per time unit.
  - For example – for long processes, the rate may be one process per hour. For short processes, it may be 10 processes per second.
- ***Turnaround time***:
  - Amount of time to execute a particular process.
  - Is the sum of the periods spent waiting *in the ready queue*, *executing on the CPU, doing I/O*, …

Scheduling Criteria (3/4)

- ***Waiting time***:
  - Since a CPU scheduling algorithm only select processes in ready queue for execution, it doe not affect the amount of time during which a process executes or does I/O.
  - Therefore, the cost of an algorithm is the amount of time a process has been **waiting in the ready queue**.
- ***Response time***: (不代表結束，只算feedback)
  - In an interactive system, turnaround time may not be the best criterion.
  - Often, a process can produce some output fairly early and can continue computing new results.
  - This measure is the amount of time it takes from when a request was submitted until the **first response** is produced.

Scheduling Criteria (4/4)

- In most cases, the goal of a scheduling algorithm is to optimize the **average** measure (e.g., the average throughput).
  - The average comes from the results of many empirical (simulated) processes.
- But .. Investigators have suggested that, for interactive system (such as time-sharing systems), it is more important to minimize the **variance** in the response time than to minimize the average response time.
  - A system with reasonable and **predictable** response time may be considered more desirable than a system that is faster on the average but is highly variable.
- However, little work has been done on minimizing variance.

Scheduling Algorithms (1/33)

- For simplicity, we consider only one CPU burst per process in the following examples.
  - In reality, each process can have several hundred CPU bursts and I/O bursts.

- The measure of comparison is the **average waiting time**.

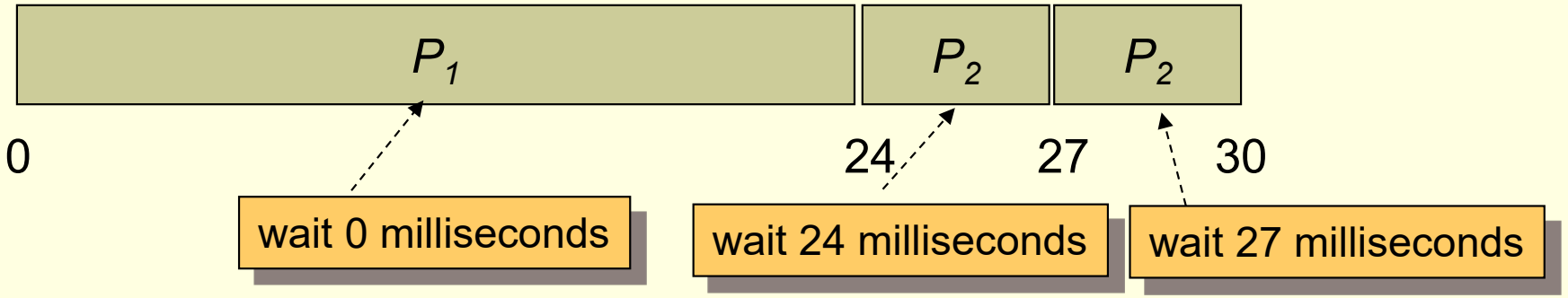
Scheduling Algorithms – **First-Come, First-Served Scheduling** (2/33)

- The simplest CPU-scheduling algorithm is the ***first-come, first-served*** (FCFS) scheduling algorithm.
- The process that requests the CPU first is allocated the CPU first.
- Can be easily managed with a FIFO queue.
  - When a process enters the ready queue, its PCB is linked onto the **tail** of the queue.
  - When the CPU is free, it is allocated to the process at the **head** of the queue.

Scheduling Algorithms – First-Come, First-Served Scheduling (3/33)

Process	Burst Time (ms.)
P_1	24
P_2	3
P_3	3

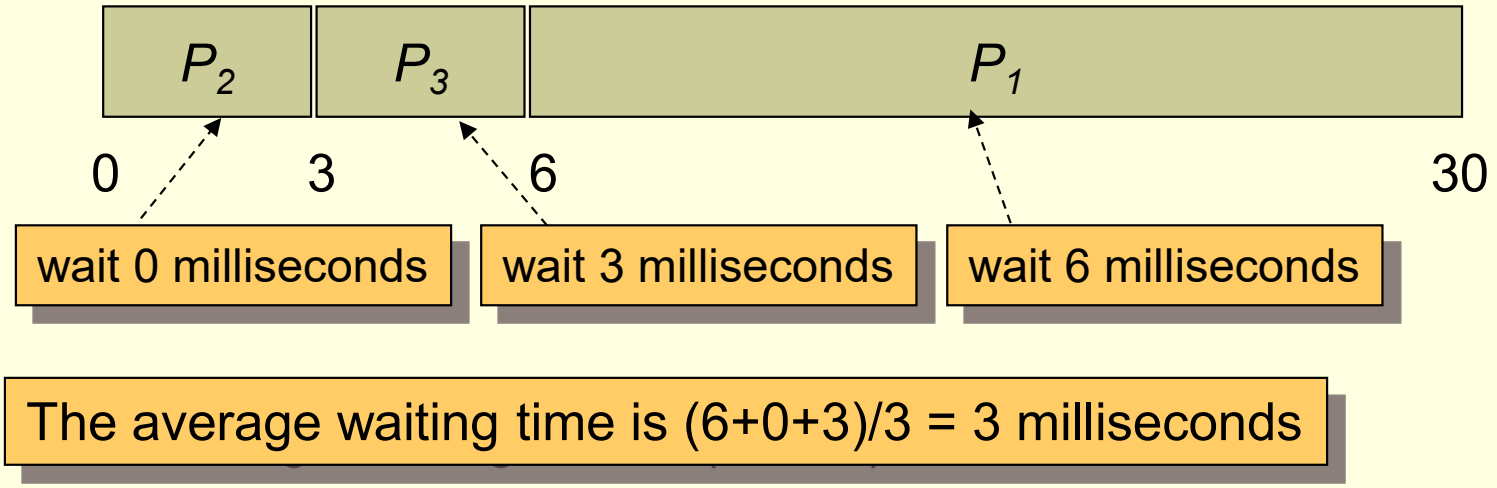
- If the process arrive in the order P_1, P_2, P_3 at time 0. Then, we get the result shown in the following Gantt chart:



The average waiting time is (0+24+27)/3 = 17 milliseconds

Scheduling Algorithms – First-Come, First-Served Scheduling (4/33)

- If the process arrive in the order P_2, P_3, P_1, the results will be as follows:



- The average waiting time under an FCFS policy is generally **not minimal** and **may vary substantially**.

Scheduling Algorithms – First-Come, First-Served Scheduling (5/33)

- The FCFS scheduling algorithm is **nonpreemptive**.
  - Once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU, either by terminating or by requesting I/O.
- Consider FCFS scheduling in a dynamic situation where we have one CPU-bound process and many I/O-bound process.
  - The CPU-bound process will get and hold the CPU.
  - All the other processes will finish their I/O and will move into the ready queue, waiting for the CPU.
    The I/O devices are idle.
  - Eventually, the CPU bound process moves to an I/O devices.
  - All the I/O-bound process execute quickly and move back to the I/O queues.
    The CPU sits idle.
  - Again, the CPU-bound process will then move back and hold the CPU, and all the I/O processes have to wait in the ready queue.

Scheduling Algorithms – First-Come, First-Served Scheduling (6/33)

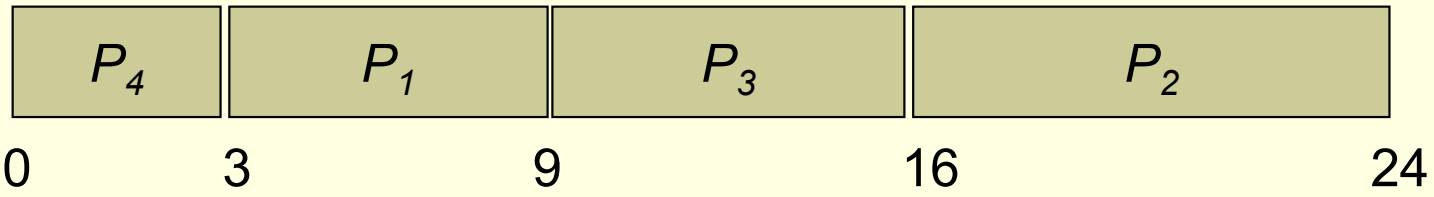
- The above situation is called a ***convoy effect***.
  - All the other processes wait for the one big process to get of the CPU.
  - Result in **lower CPU** and **device utilization**.

Scheduling Algorithms – **Shortest-Job-First Scheduling** (7/33)

- The shortest-job-first (SJF) scheduling algorithm associates with each process the length of the process's next CPU burst.
  - Another more appropriate term – ***shortest-next-CPU-burst algorithm***.
- When the CPU is available, it is assigned to the process that has the smallest next CPU burst.
  - If the next CPU bursts of two processes are the same, FCFS scheduling is used to break the tie.

Scheduling Algorithms – Shortest-Job-First Scheduling (8/33)

Process	Burst Time (ms.)
P_1	6
P_2	8
P_3	7
P_4	3



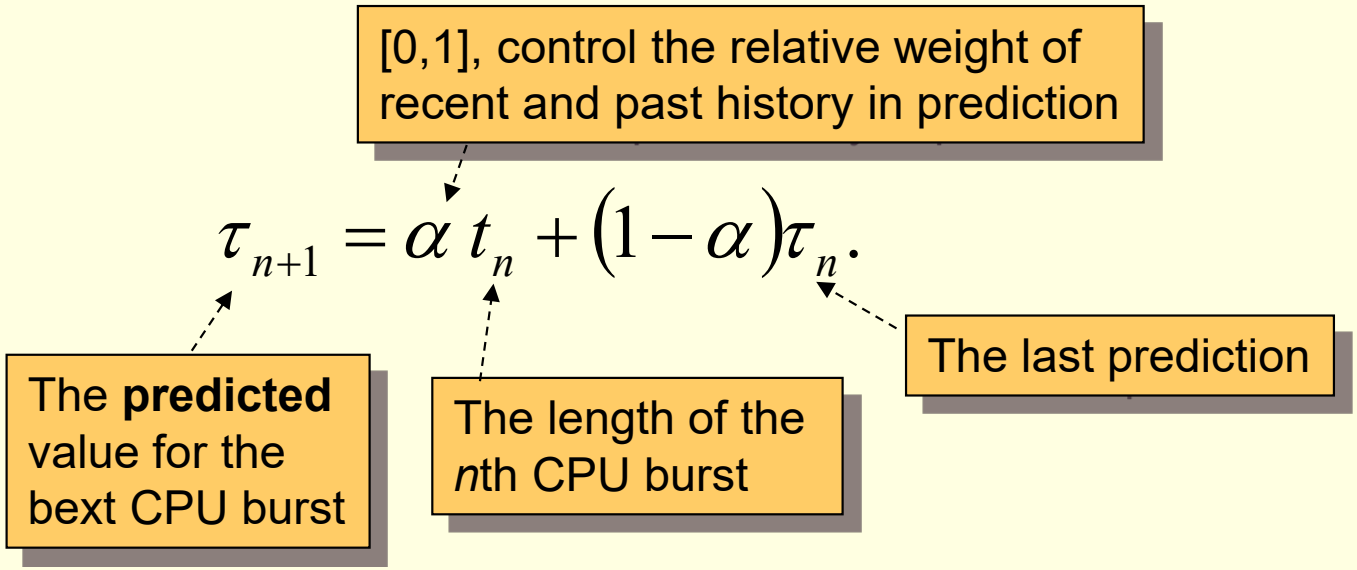
- The average waiting time is (3+16+9+0)/4 = 7 ms.
- By comparison, if we were using the FCFS, the average waiting time would be 10.25 ms.

Scheduling Algorithms – Shortest-Job-First Scheduling (9/33)

- **The SJF scheduling algorithm is optimal!!**
  - It gives the minimum average waiting time for a given set of processes.
  - Why?
    - Moving a short process before a long one decreases the waiting time of the short process more than it increases the waiting time of the long process.
- But … how can we obtain the length of the next CPU burst??
  - **It is impossible to do so …**
- However, we can try our best to **predict** the length.
  - We expect that the next CPU burst will be similar in length to the previous one**s**.
  - And pick the process with the shortest predicted CPU burst.

Scheduling Algorithms – Shortest-Job-First Scheduling (10/33)

- **Prediction** – the ***exponential average*** of the measured lengths of previous CPU bursts.



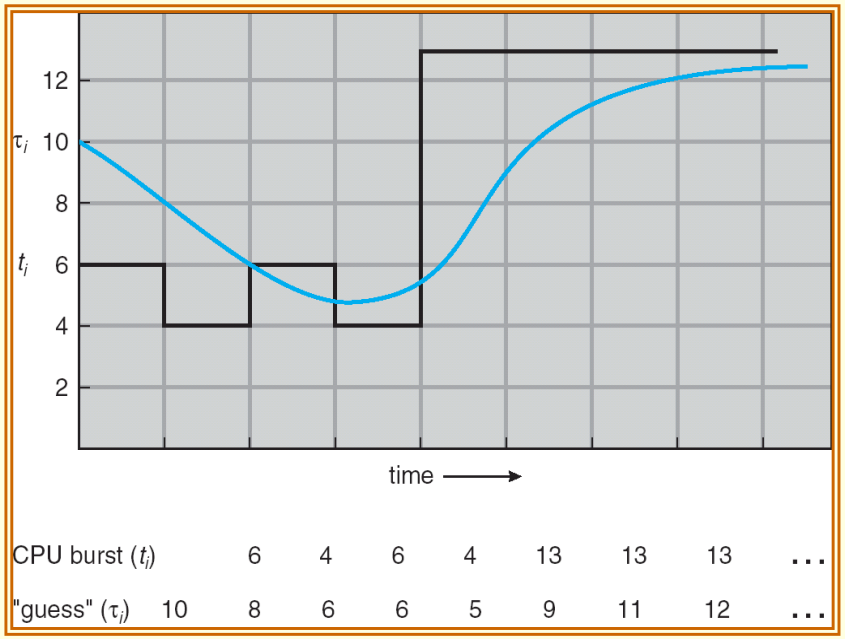
  - If α=1, then only the most recent CPU burst matters.
  - More commonly =0.5; recent history and past history are equally weighted.

Scheduling Algorithms – Shortest-Job-First Scheduling (11/33)

- We can expand the formula to find:

$$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\alpha t_{n-1} + (1-\alpha)^2 \alpha t_{n-2} + \ldots + (1-\alpha)^j \alpha t_{n-j} + \ldots + (1-\alpha)^{n+1} \tau_0$$

  - τ_0 can be defined as a constant or as an overall system average.
  - Since both α and (1-α) are **less than or equal to 1**, each successive term has less weight than its predecessor.



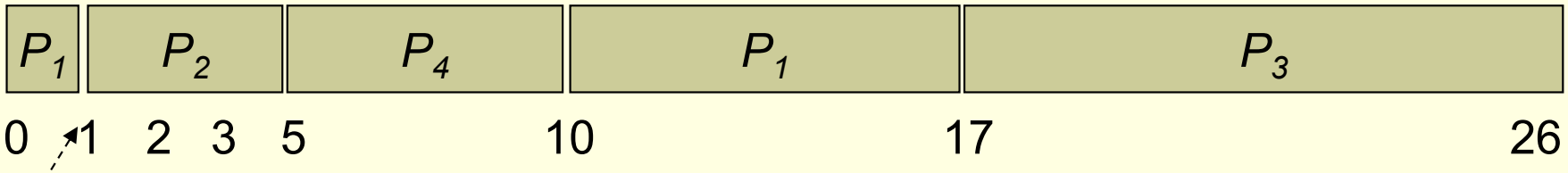
Scheduling Algorithms – Shortest-Job-First Scheduling (12/33)

- The SJF algorithm can be either **preemptive** or **nonpreemptive**.
  - A preemptive SJF algorithm will preempt the currently executing process, if the next CPU burst of the newly arrived process is shorter than **"what is left"** of the currently executing process.
  - Whereas a nonpreemptive SJF algorithm will allow the currently running process to finish its CPU burst.

Scheduling Algorithms – Shortest-Job-First Scheduling (13/33)

- An example of **preemptive** SJF algorithm:

Process	Arrival Time	(remaining) Burst Time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5



Preempted!!

This algorithm is also known as ***shortest-remaining-time-first scheduling***

Scheduling Algorithms – Shortest-Job-First Scheduling (14/33)

- The average waiting time for preemptive algorithm is ((10-1) + (1-1) + (17-2) + (5-3)) / 4 = 6.5 ms.
- The average waiting time for nonpreemptive algorithm is 7.75ms.
- SJF scheduling is used frequently in **long-term scheduling**, where users need to specify **"process time limit"** of the processes.
  - The algorithm encourages users have accurate estimations because:
    - A lower value mean faster response.
    - And too low a value will cause a time-limit-exceeded error and require resubmission.

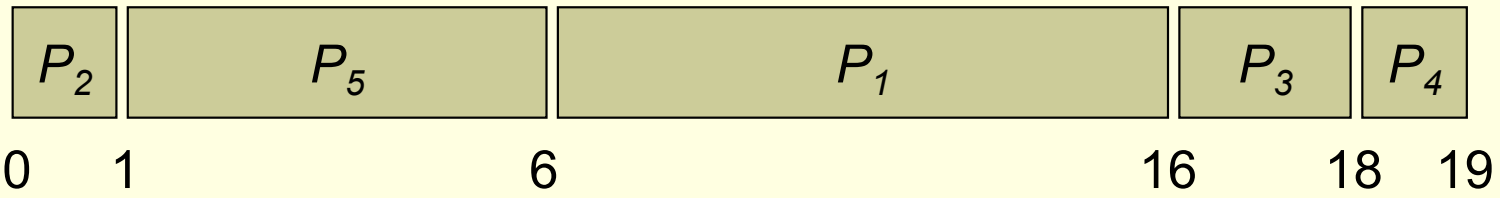
Scheduling Algorithms – **Priority Scheduling** (15/33)

- A priority is associated with each process, and the CPU is allocated to the process with the highest priority.
  - Equal-priority processes are scheduled in FCFS order.
  - The SJF algorithm is a special case of the priority scheduling algorithm.
    - The larger the CPU burst, the lower the priority, and vice versa.
- Priorities are generally indicated by some fixed range of numbers.
  - In this text, we assume that low numbers represent high priority.

Scheduling Algorithms – Priority Scheduling (16/33)

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

- If the process arrive in the order P_1, P_2, …, P_5 at time 0, we would schedule these processes as follows:



  - The average waiting time is 8.2 ms.

Scheduling Algorithms – Priority Scheduling (17/33)

- Priorities can be defined either **internally** or **externally.**
  - Internally – use measurable quantity in terms of time limits, memory requirements …
  - Externally – set by criteria outside the operating system, such as importance, funds, …*often political factors*.
- Priority scheduling can be either **preemptive** or **nonpreemptive**.
  - Preemptive – can preempt the CPU if the priority of the newly arrived process is higher than the priority of the currently running process.
  - Nonpreemptive – simply put the new process at the head of the ready queue.

Scheduling Algorithms – Priority Scheduling (18/33)

- ***Starvation***:
  - **Low-priority processes can be blocked indefinitely**.
  - A major problem with priority scheduling algorithms.
  - May occur in a heavily loaded computer system with steady stream of higher-priority processes.
  - *Rumor: when the IBM 7094 at MIT shut down in 1973, administrators found a low-priority process that had been submitted in 1967, and had not yet been run.*

  - Solution: ***aging***.
    - Gradually increase the priority of processes that wait in the system for a long time.
      - E.g., by 1 every 15 minutes.
    - A process would eventually arrive the highest priority and would be executed.

Scheduling Algorithms – **Round-Robin Scheduling** (19/33)

- Is designed especially for time sharing systems.
- Is similar to FCFS scheduling, but **preemption** is added to switch between processes.
- The ready queue is treated as a **circular queue**.
  - New processes are added to the tail of the ready queue.
- A small unit of time, called a ***time quantum*** or ***time slice***, is defined.
  - Generally 10 to 100 ms.
  - A timer is interrupted after 1 time quantum.
  - The scheduler goes around the ready queue, allocating the CPU to each process for a time interval of up to 1 time quantum.

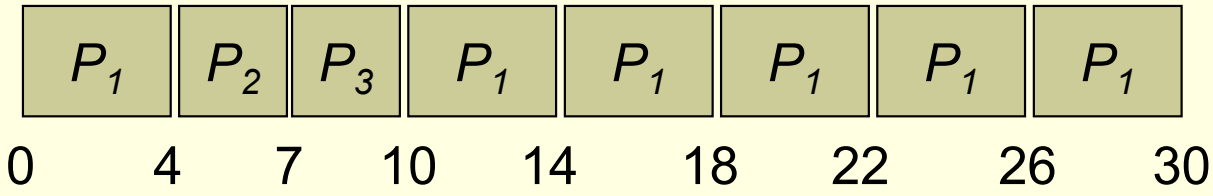
Scheduling Algorithms – Round-Robin Scheduling (20/33)

- If the CPU burst of the selected process is less than 1 time quantum …
  - The process itself will release the CPU.
  - The scheduler will then proceed to the next process in the ready queue.
- If the CPU burst is longer than 1 time quantum …
  - The timer will go off and will cause an interrupt to create context switch.
    - **That is, the process is preempted.**
  - The process will be put at the tail of the ready queue.
  - The scheduler will then select the next process in the ready queue.

Scheduling Algorithms – Round-Robin Scheduling (21/33)

Process	Burst Time (ms.)
P_1	24
P_2	3
P_3	3

- If the process arrive in the order P_1, P_2, P_3 at time 0 and a time quantum is 4 ms. Then the process schedule is:



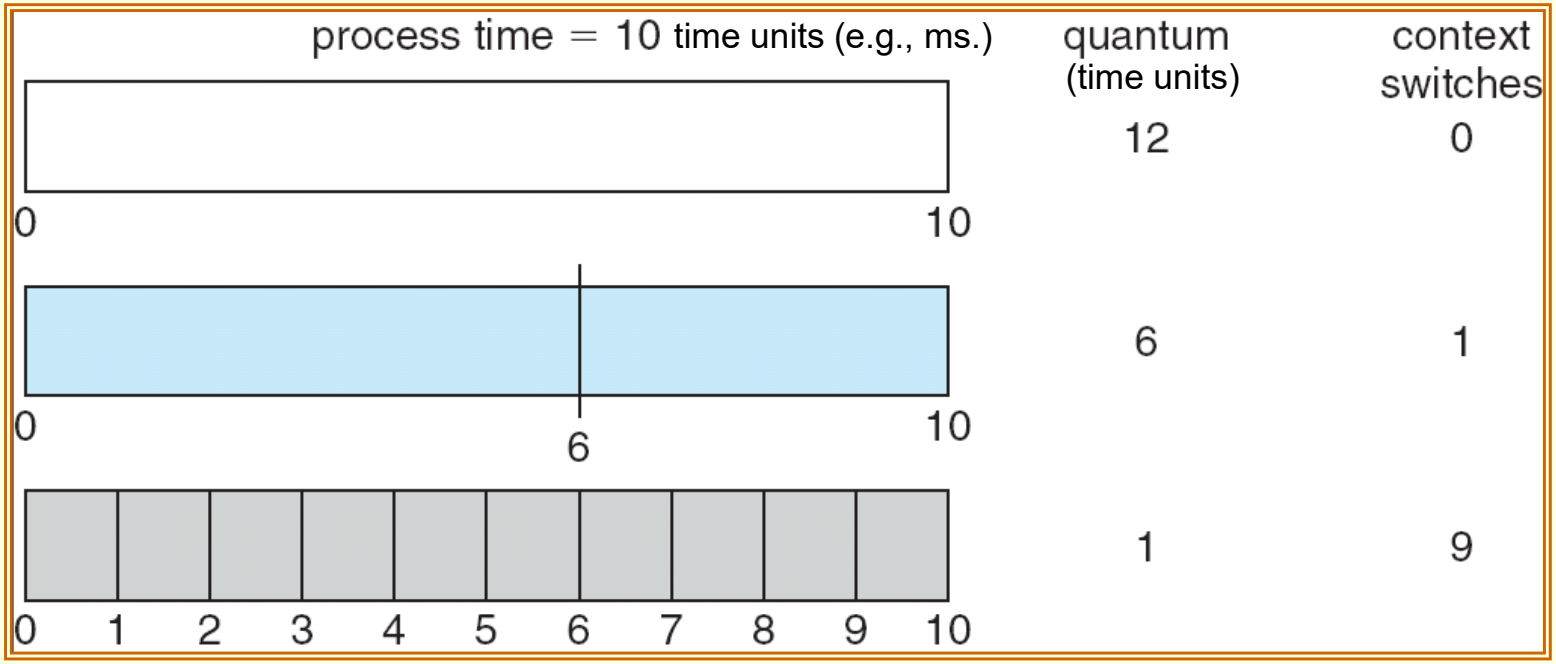
- The average waiting time is (6 + 4 + 7) / 3 = 5.66 ms.

Scheduling Algorithms – Round-Robin Scheduling (22/33)

- If there are *n* processes in the ready queue and the time quantum is *q*,
  - Each process must wait **no longer** then (*n*-1)x*q* time units unit its next time quantum.
- The average waiting time under the RR policy is often long.
  - And the waiting time is proportional to the number of processes.
  - But the waiting is used to trade for time sharing phenomenon.
- The performance of the RR algorithm depends heavily on the size of the time quantum.
  - If the time quantum is extremely large, the RR policy is the same as the FCFS policy.
  - If the time quantum is extremely small (say, 1 ms), the RR approach is called processor sharing and (in theory) creates the appearance that each of *n* processes has its own processors running at 1/*n* the speed of the real processor.

Scheduling Algorithms – Round-Robin Scheduling (23/33)

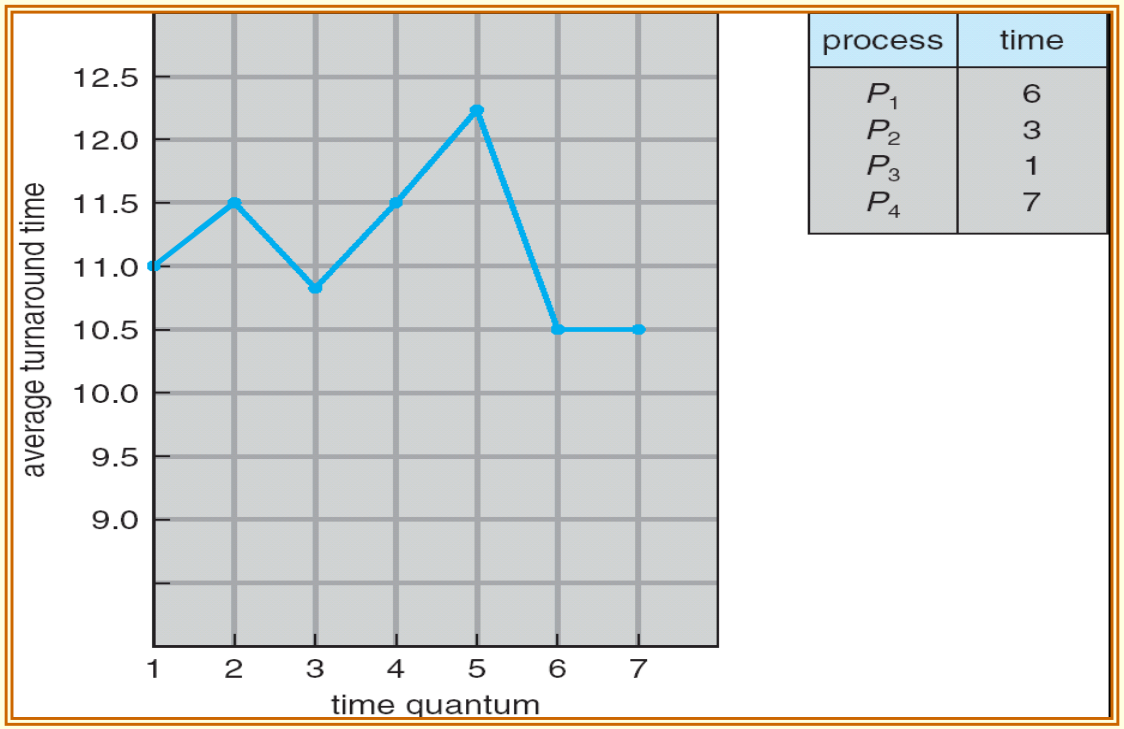
- **But … do not forget the effect of context switching.**



- We want the time quantum to be large with respect to the context switch time.
  - In practice, operating systems have time quanta ranging from 10 to 100 milliseconds (10^{-3}).
  - The time of context switch is typically less than 10 microseconds (10^{-6}).
  - Only a small fraction of the CPU time spent in context switching.

Scheduling Algorithms – Round-Robin Scheduling (24/33)

- The average turnaround time of a set of processes **does not** necessarily improve as the time-quantum size increases.



Scheduling Algorithms – Round-Robin Scheduling (25/33)

- In general, the average turnaround time can be improved if most processes finish their **next** CPU burst in a single time quantum.
  - This implies the time quantum should be long.
  - For example, given 3 processes of 10 time units.
    - The average turnaround time is 29 for a quantum of 1 time unit.
    - However, the average turnaround time drops to 20 if the time quantum is 10.
    - Moreover, if the cost of context-switch is added in, the average turnaround time **increases for a smaller time quantum**.

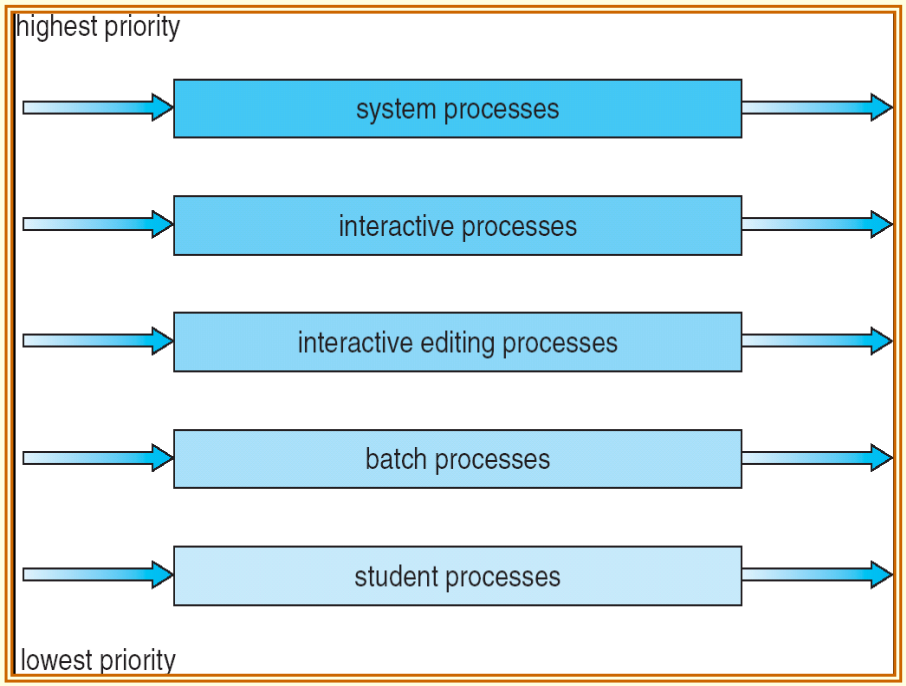
- But remember that … the time quantum should not be too large.
  - Otherwise RR scheduling degenerates to FCFS policy.
  - A rule of thumb is that 80 percent of the CPU bursts should be shorter than the time quantum.

Scheduling Algorithms – **Multilevel Queue Scheduling** (26/33)

- Processes are classified into different groups.
  - Different types of processes require different response-time and have different scheduling needs.
  - For examples, foreground (interactive) and background (batch) processes.
- The scheduling algorithm partitions the ready queue into several separate queues.
  - The processes are assigned to one queue, based on some property of the process (memory size, process priority …).
  - Each queue has its own scheduling algorithm.
    - The foreground queue might be scheduled by an RR algorithm.
    - The background queue is scheduled by an FCFS algorithm.
  - Moreover, there must be scheduling among the queues.
    - Usually a fixed-priority preemptive scheduling.

Scheduling Algorithms – Multilevel Queue Scheduling (27/33)

- No process in batch queue could run unless the queues for system processes, interactive processes, and interactive editing processes were all empty.
- If an interactive editing process entered the ready queue while a batch process was running, the batch process would be preempted.



Scheduling Algorithms – Multilevel Queue Scheduling (28/33)

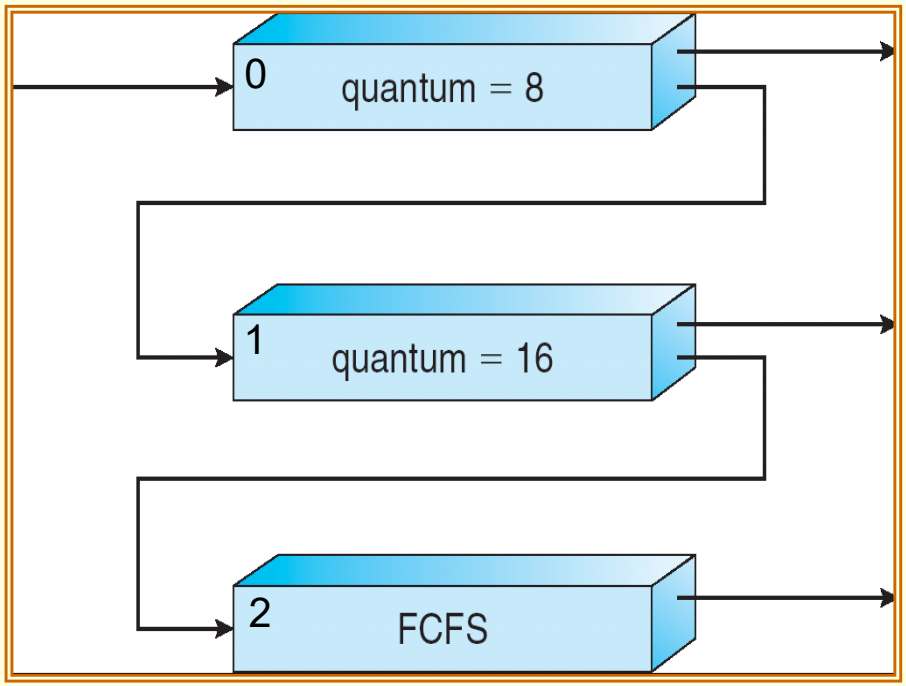
- Another possibility is to time-slice among the queues.
  - Each queue gets a certain portion of the CPU time.
    - Which it can then schedule among its various processes.
  - For instance, in the foreground-background queue example:
    - The foreground queue can be given 80 percent of the CPU time for RR scheduling among its processes.
    - The background queue receives 20 percent of the CPU to give to its processes on an FCFS basis.

Scheduling Algorithms – **Multilevel Feedback-Queue Scheduling** (29/33)

- The scheduling algorithm allows a process to move between queues.
- The idea:
  - If a process uses too much CPU time, it will be moved to a lower-priority queue.
  - A process that waits too long in a lower-priority queue may be moved to a higher-priority queue.

Scheduling Algorithms – Multilevel Feedback-Queue Scheduling (30/33)

- In this example, the scheduler first executes all processes in queue 0.
- Only when queue 0 is empty will it execute processes in queue 1.
  - Similarly, processes in queue 2 will only be executed if queues 0 and 1 are empty.
- A process in queue 1 will in turn be preempted by a process arriving for queue 0.



Scheduling Algorithms – Multilevel Feedback-Queue Scheduling (31/33)

- A process (formally, a CPU burst) entering the ready queue is put in queue 0.
  - It will be given a time quantum of 8 ms.
  - If it (the burst) does not finish within this time, it is moved to the tail of queue 1.
- If queue 0 is empty, the process at the head of queue 1is given a quantum of 16 ms.
  - If it (the burst) does not complete, it is preempted and is put into queue 2.
- Processes in queue 2 are run on an FCFS basis.
  - But are run only when queues 0 and 1 are empty.

Scheduling Algorithms – Multilevel Feedback-Queue Scheduling (32/33)

- This example gives highest priority to processes with CPU bursts of 8 milliseconds or less.
  - Such a process will quickly get the CPU.
  - Finish its CPU burst.
  - And go off to its next I/O burst.
- Processes (CPU bursts) that need more than 8 but less than 24 milliseconds are also served quickly.
- Long processes automatically sink to queue 2 and are served in FCFS order with any CPU cycles left over from queues 0 and 1.

Scheduling Algorithms – Multilevel Feedback-Queue Scheduling (33/33)

- A multilevel-feedback-queue scheduler is generally defined by the following parameters:
  - Number of queues.
  - Scheduling algorithms for each queue.
  - Method used to determine when to upgrade a process.
  - Method used to determine when to demote a process.
  - Method used to determine which queue a process will enter when that process needs service.
- The flexibility makes it the most general CPU-scheduling algorithm.
  - Can be configured to match a specific system.
  - Unfortunately, it is also the most complex algorithm (parameter setting).

Multiple-Processor Scheduling (1/5)

- ***Asymmetric multiprocessing***:
  - Has all scheduling decision, I/O processing, and other system activities handled by a single processor – **the master server**.
- ***Symmetric multiprocessing*** (SMP):
  - Each processor is self-scheduling.
  - All processes may be in a common ready queue, or each processor may have its own private queue of ready processes.
  - Scheduling schedules each processor to examine the ready queue and selects a process to execute.
  - Virtually all modern operating systems support SMP.

Multiple-Processor Scheduling (2/5)

- **Cache** in storage hierarchy:
  - The data most recently accessed by a process populates the cache for a processor.
  - Successive memory access by the process are often satisfied in cache memory.
- If the process migrates to another processor …
  - The contents of cache memory must be **invalidated** for the processor being migrated from.
  - The cache for the processor being migrated to must be **re-populated**.
- To avoid the cost, most SMP systems try to avoid migration of processes between processors.
  - This is known as ***processor affinity***, meaning that a process has an affinity for the processor on which it is currently running.
  - **Soft affinity**: try to, but not guarantee that it will do so.
  - **Hard affinity**: provide system calls for the forbiddance.

Multiple-Processor Scheduling (3/5)

- ***Load balancing***:
  - Keep the workload balanced among all processors.
  - Fully utilize the benefits of multi-processors.
  - Is often **unnecessary** on systems **with a common ready queue**.
    - Once a processor becomes idle, it immediately extracts a runnable process from the common ready queue.
    - However, in most SMP systems, each processor does have a private queue of eligible processes.

Multiple-Processor Scheduling (4/5)

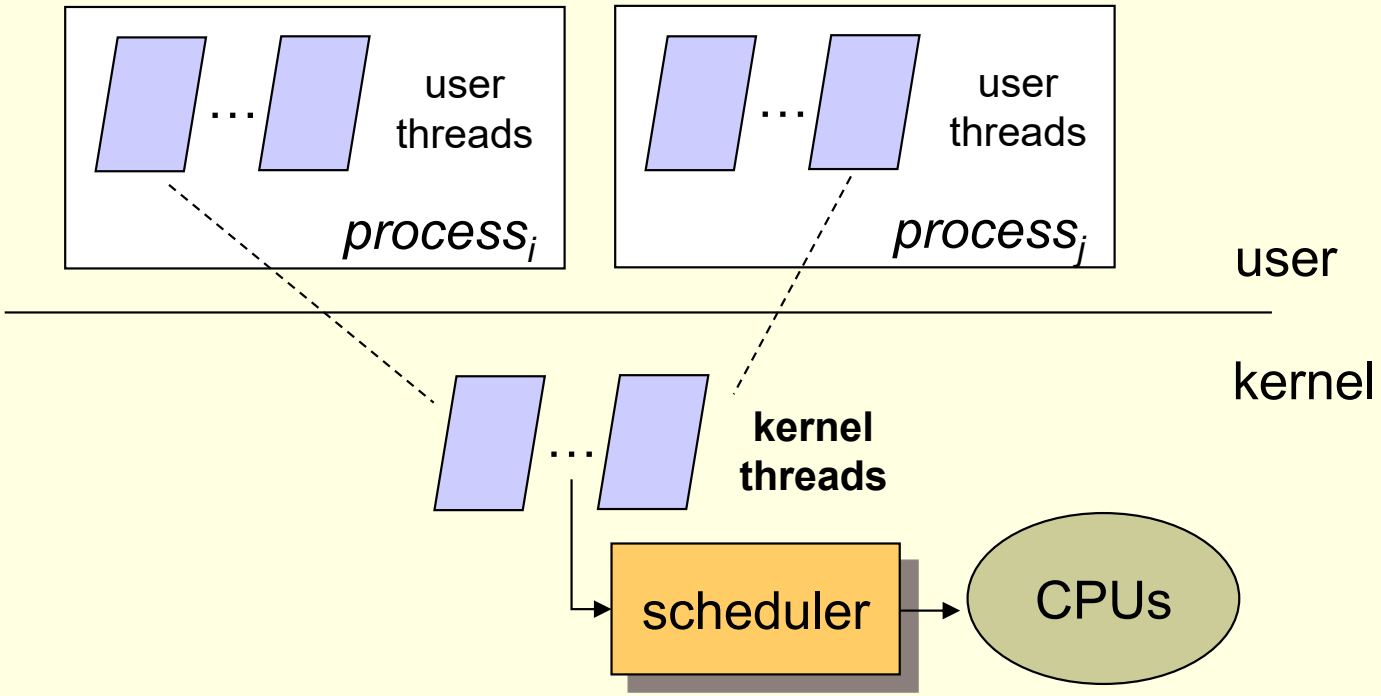
- Two general approaches to load balancing:
  - ***Push migration***:
    - A specific task (kernel process) periodically checks the load on each processor.
    - Evenly distributes the load by moving (or **pushing**) processes from overloaded to idle or less-busy processor.
  - ***Pull migration***:
    - An idle processor pulls a waiting task from a busy processor.
- Push and pull need not be mutually exclusive.
  - Linux runs its load-balancing algorithm every 200 ms (push) or whenever the run queue for a process is empty (pull).

Multiple-Processor Scheduling (5/5)

- Interestingly, load balancing often counteracts the benefits of processor affinity.
  - Either pulling or pushing a process from one processor to another will invalidate the benefit of cache memory.
  - Some systems only move processes if the imbalance exceeds a certain threshold.
- In practice, there is no absolute rule concerning what policy is best.

Thread Scheduling (1/2)

- On a **multithreaded** system:
  - User-level threads must ultimately be mapped to an associated kernel-level thread, to run on a CPU.
  - **It is kernel-level threads that are being scheduled by the operating system.**

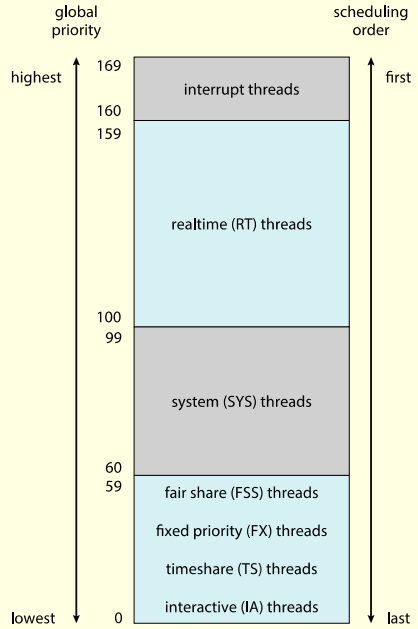


Thread Scheduling (2/2)

- For **many-to-one** and **many-to-many** models:
  - The kernel is **unaware** of user level threads.
  - User-level threads are **managed by a thread library**.
    - The library schedules user-level threads to run on an available LWP.
    - The competition is known as ***process-contention scope*** (PCS), since competition for the CPU takes place among threads belonging to the same process.
  - When the thread library schedules user threads onto available LWPs, **it does not mean that the thread is actually running on a CPU**.
  - The operating system schedules the kernel thread onto a physical CPU.
    - ***System-contention scope*** (SCS) – kernel threads compete with each other for CPU time.
- Systems using the **one-to-one model** (such as Windows XP) schedule threads using only SCS.

Operating System Examples – **Solaris** (1/13)

- Solaris uses **priority-based** thread scheduling:
  - Six classes of scheduling:
    - Real time.
    - System.
    - Fair share.
    - Fix Priority
    - Time sharing (default class for a process).
    - Interactive (e.g., windowing applications).
  - Each class includes a set of priorities.
  - The scheduler converts the class-specific priorities into global priorities.



- The thread with the highest global priority is selected to run.
  - If there are multiple threads with the same priority, the scheduler uses a **round-robin queue**.
  - Thus, the selected thread runs unit it (1) blocks, (2) uses its time slice, or (3) is **preempted** by a higher-priority thread.

Operating System Examples – Solaris (2/13)

- Threads in the real-time class are given the highest priority.
  - A real-time process will run before a process in any other class.
  - In general, **few** processes belong to the real-time class.
- Kernel processes (such as the scheduler) are run in the system class.
  - The priority of a system process does not change, once established.

Operating System Examples – Solaris (3/13)

- The fixed-priority and fair-share classes were introduced with Solaris 9.
  - Threads in the fixed-priority class have the same priority range as those in the time-sharing classes.
  - However, their priorities are not dynamically adjusted.

Operating System Examples – Solaris (4/13)

- The scheduling policy for time sharing (and interactive) class **dynamically alters priorities** of threads using a **multilevel feed back queue**.

priority	time quantum	time quantum expired	return from sleep
0 (low)	200	0	50
5	200	0	50
10	160	0	51
15	160	5	51
20	120	10	52
25	120	15	52
30	80	20	53
35	80	25	54
40	40	30	55
45	40	35	56
50	40	40	58
55	40	45	58
59 (high)	20	49	59

There is an **inverse** relationship between priorities and time slices.

CPU-bound processes would have lower priorities.

Interactive processes would have good response time.

The new priority of a thread that has used its entire time slice.

Penalize CPU-bound processes.

The priority of a thread that is returning from sleeping (e.g., I/O).

Provide good response time for interactive processes

Operating System Examples – **Windows** (5/13)

- Windows scheduler is a **priority-based**, **preemptive** scheduling algorithm.
- Priority classes:
  - `REALTIME_PRIORITY_CLASS`
  - `HIGH_PRIORITY_CLASS`
  - `ABOVE_NORMAL_PRIORITY_CLASS`
  - `NORMAL_PRIORITY_CLASS`
  - `BELOW_NORMAL_PRIORITY_CLASS`
  - `IDLE_PRIORITY_CLASS`
- Within each class is a relative priority:
  - `TIME_CRITICAL`
  - `HIGHEST`
  - `ABOVE_NORMAL`
  - `NORMAL`
  - `BELOW_NORMAL`
  - `LOWEST`
  - `IDLE`

	real-time	high	above normal	normal	below normal	idle priority
time-critical	31	15	15	15	15	15
highest	26	15	12	10	8	6
above normal	25	14	11	9	7	5
normal	24	13	10	8	6	4
below normal	23	12	9	7	5	3
lowest	22	11	8	6	4	2
idle	16	1	1	1	1	1

Fixed, once determined

The priority of each thread (process) is based on the priority class it belongs to and its relative priority within that class

Base priority for each class

Operating System Examples – Windows (6/13)

- The Windows scheduler (also called dispatcher) :
  - Uses **a** queue **for each** scheduling priority (32-level).
  - Traverses the set of queues form highest to lowest until it finds a thread that is ready to run.

- The highest-priority thread will always run … until:
  - It is preempted by a higher-priority thread.
  - It terminates.
  - It calls a blocking system call (such as for I/O).
  - Its time quantum ends.

Operating System Examples – Windows (7/13)

- Processes are typically members of the `NORMAL_PRIORITY_CLASS`.
  - **You can specify the class when you create a process.**
  - The initial priority of a thread is typically the **base priority** of the process the thread belongs to.
- When a thread's time quantum runs out … and it is in the variable-priority class.
  - Its priority is lowered.
  - To limit the CPU consumption of compute-bound thread.
- When a variable-priority thread is released from a wait operation …
  - Its priority is boosted.
  - Tend to give good response times to interactive threads.
    - I/O waiting for mouse and keyboard.

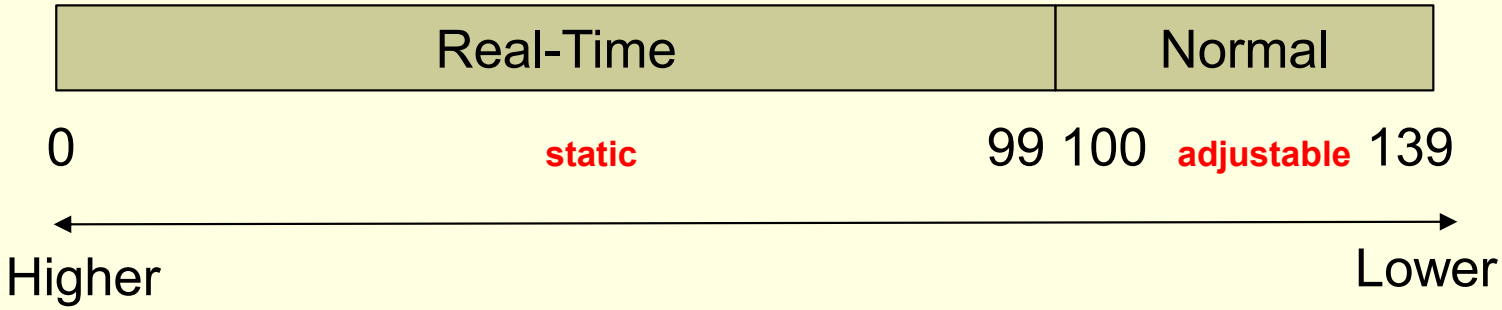
This strategy (multilevel-feedback) is used by many time-sharing operating system.

Operating System Examples – Windows (8/13)

- To provide especially good performance for a running interactive program …
  - Windows distinguishes between the *foreground process* (selected on the screen) and the *background processes*.
  - The scheduling quantum of the foreground process is increased by some factor to give it longer time to run.
  - This special scheduling rule only for process in the `NORMAL_PRIORITY_CLASS`.

Operating System Examples – **Linux** (9/13)

- In release 2.6.23 of the kernel, the ***Completely Fair Scheduler*** (CFS) became the default Linux scheduling algorithm.
- The Linux scheduler is based on **scheduling classes**, and is a **preemptive**, **priority-based** algorithm.
  - Standard Linux kernels implement two scheduling classes: (1) a default (normal) class and (2) a real-time class.



Operating System Examples – Linux (10/13)

- For the normal class:
  - Each task is associated with a ***nice value***.
    - Nice values range from -20 to +19 (0 as default).
    - Generally, a numerically lower nice value indicates a higher relative priority.
  - Unlike Solaris, tasks with lower nice values (higher priorities) receive a higher proportion CPU processing time.

Operating System Examples – Linux (11/13)

- CFS does not use discrete values of time slices!!
  - It identifies a ***targeted latency***, which is an interval of time during which every runnable task should run at least once.
  - The targeted latency has default and minimum values, and can increase if the number of tasks is growing.
- Proportions of CPU time are allocated from the value of targeted latency.

Operating System Examples – Linux (12/13)

- CFS does not directly assign priorities.
  - It records how long each task **has run** using variable ***vruntime*** (virtual run time).
  - The virtual run time is associated with a decay factor based on the priority (nice value) of a task.
    - For tasks at normal priority (nice values of 0), *vruntime* is identical to actual physical run time.
    - If a higher-priority task runs for 200 milliseconds, its *vruntime* will be less than 200 milliseconds.

- To decide which task to run next:
  - CFS simply selects the task that **has the smallest *vruntime* value**!!
  - In addition, a higher-priority task the becomes available to run can **preempt a lower-priority task**.

Operating System Examples – Linux (13/13)

- Assume that two tasks have the same nice values.
  - One task is I/O bound and the other is CPU-bound.
- The I/O-bound task will run only for short periods.
  - The value of *vruntime* will eventually be lower for the I/O-bound task than for the CPU-bound task.
  - **This gives the I/O-bound task higher priority!!**
- If the CPU-bound task is executing when the I/O-bound task becomes eligible to run…
  - **The I/O-bound task will preempt the CPU-bound task!!**

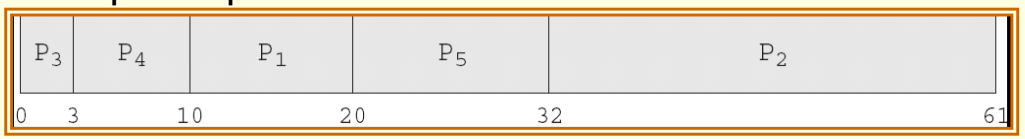
Algorithm Evaluation – **Deterministic Modeling** (1/9)

- The evaluation method takes a particular **predetermined workload** and defines the performance of each algorithm for that workload.
- For example, given the following workload where all processes arrive at time 0 in the order given, which algorithm would give the minimum average waiting time?

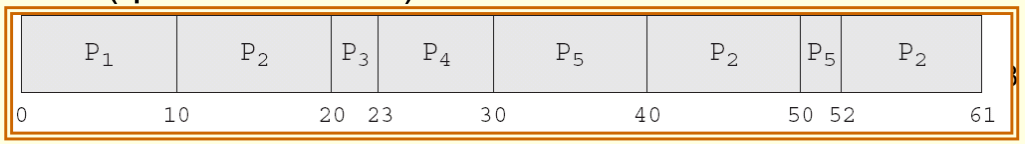
Process	Burst Time (ms.)
P_1	10
P_2	29
P_3	3
P_4	7
P_5	12

FCFS – 28ms

Nonpreemptive SJF – 13ms



RR (quantum=10ms) – 23ms



Algorithm Evaluation – Deterministic Modeling (2/9)

- Deterministic modeling is simple and fast.
- It also gives us exact numbers, allowing us to compare the algorithms.
- **However**, it requires exact numbers for input (predetermined workload), and its answers apply only to those cases.
  - On many systems the processes vary from day to day, so there is no static set of processes to use for deterministic modeling.
  - Therefore, the main uses of this method are in describing scheduling algorithms and providing examples.

Algorithm Evaluation – **Queueing Models** (3/9)

- The computer system is described as **network of servers**.
  - Each server has a queue of waiting processes.
    - The CPU is a server with its ready queue, as is the I/O system with its device queue.
  - ***Queueing-network analysis***: knowing arrival rates and service rates, we can compute utilization, average waiting time …
  - If the system is in a **steady state** —— the number of processes leaving the queue must be equal to the number of processes that arrive:
    - ***Little's formula***: $n = \lambda \times W$

The average queue length

The average arrival rate for new process in the queue (e.g., 3 processes/second)

The average waiting time in the queue

Algorithm Evaluation – Queueing Models (4/9)

- We can use Little's formula to compute one of the three variables, if we know the other two.
  - If we know that 7 processes arrive every second (on average), and the there are normally 14 processes in the queue,
  - Then we can compute the average waiting time per process as 2 seconds.
- Queueing analysis can be useful in comparing scheduling algorithms.
  - However, the mathematics of complicated algorithms and distributions can be difficult to work with.
  - Thus the distributions are often defined in **unrealistic** ways and may not be accurate.
  - Therefore, the accuracy of the computed results may be questionable.

Algorithm Evaluation – **Simulations** (5/9)

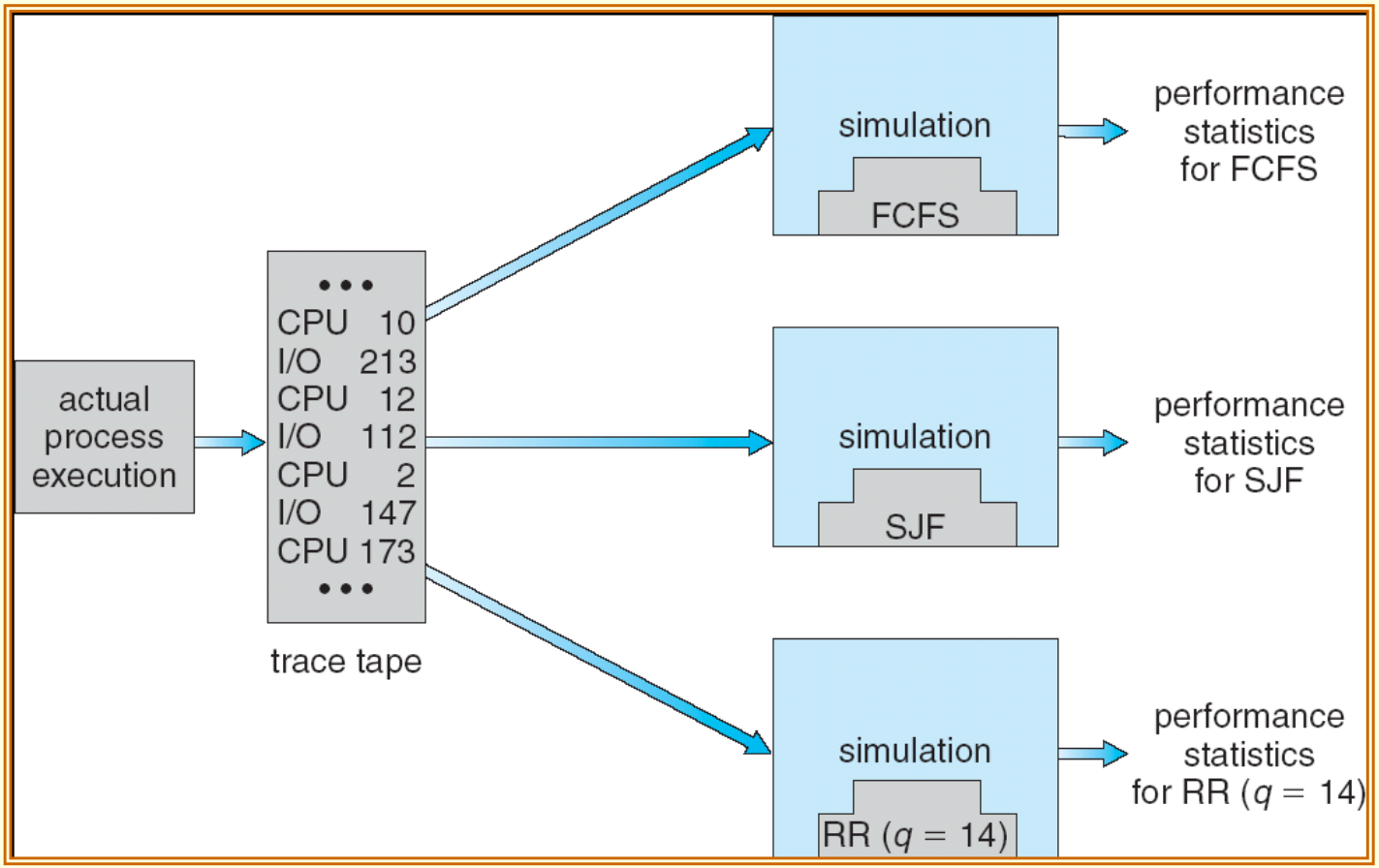
- Simulation involves:
  - Programming a model of the computer system.
  - The programmed system has a *variable* representing a *clock*.
    - As the variable's value is increased, the simulator modifies the **system state** to reflect the activities of the devices, the processes, and the scheduler.
- As the simulation executes, statistics that indicate algorithm performance are gathered and printed.
  - Usually, the evaluation result is more accurate than that of queueing models.

Algorithm Evaluation – Simulations (6/9)

- Simulators require data to drive simulations.
  - The most common method of data generation uses a **random-number generator**.
    - Which generate processes, CPU burst times, arrivals … according to probability distributions.
    - However, a distribution-driven simulation may be inaccurate.
      - Generally, the frequency distribution indicates only how many instances of each event occur; rarely indicate the order of their occurrence.
  - To acquire more accurate result, we can use ***trace tapes***.
    - A trace tape is created by monitoring the real system and recording the **sequence** of actual events.
    - The sequence is then used to drive the simulation.

Algorithm Evaluation – Simulations (7/9)

- Trace tapes provide an excellent way to compare different algorithms on exactly the same set of **real** inputs.



Algorithm Evaluation – Simulations (8/9)

- While simulations can acquire accurate results.
  - The process can be **expensive**, often requiring hours of computer time.
  - Trace tapes can require large amount of storage space.
  - The design, coding, and debugging of the simulator can be a difficult task.

Algorithm Evaluation – **Implementation** (9/9)

- The completely accurate way to evaluate a scheduling algorithm is to **code it up**!!
  - Put it in the operating system, and see how it works.
- The difficult with this approach is the high cost.
  - Coding the algorithm.
  - Modifying the operating system to support it.
  - **The reaction of the users to a constantly changing operating system**.

End of Chapter 5